University of Stuttgart
Institute of Industrial Automation and
Software Engineering

Online Risk-Annotated Scene Graphs for Autonomous Systems

Master Thesis [MT-3986]

Submitted at the University of Stuttgart by
André Schiel

Autonomous Systems

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: M.Sc. Joachim Grimstad

26.03.2026

Abstract

This thesis addresses a key gap in dynamic risk perception for Human-Robot Interaction (HRI). Many existing detection pipelines are closed-set and therefore provide limited contextual understanding in changing scenes. In particular, common object detectors such as YOLO do not natively maintain richer semantic context, additional labels, or pose-related relations over time in a unified world model. To address this, the thesis presents a modular framework for online Scene Graph (SG) generation that combines RGB-D sensing, visual odometry, graph cutout generation, and Visual Language Model (VLM) inference. Additional modules for segmentation, prompt construction, and graph fusion are integrated into a frame-wise processing pipeline designed for interoperability and model exchange. The framework also includes mechanisms to reduce VLM hallucinations and output instability, which are especially relevant for smaller models.

The framework was evaluated in two laboratory test runs. One static environment with a moving camera and a dynamic scenario with a moving person. In the static run, the system achieved around 95% node classification precision and approximately 90% edge precision. In the dynamic run, where viewpoint diversity was lower and reflections introduced depth artifacts, node precision reached around 80% and edge precision approximately 85%-90%. Here, precision is defined as the proportion of correctly classified VLM object predictions relative to all predicted labels by the VLM. The appearance-based tracking mechanism successfully inserted and removed dynamic human nodes over time, demonstrating consistent temporal graph maintenance.

Overall, the thesis demonstrates the technical feasibility of online VLM-driven SG generation for dynamic risk perception. The current scope is perception rather than risk assessment. The system incrementally builds and maintains a semantic SG from visual observations, while global SLAM-based spatial correction is not yet implemented. Nevertheless, revisiting previously seen positions remains functional through persistent node-position storage and tracking. The resulting graph provides a structured and extensible representation that can be used by subsequent modules to perform scene-graph-based risk assessment in future work.

Zusammenfassung

Diese Arbeit adressiert eine zentrale Lücke in der dynamischen Risikowahrnehmung für die Interaktion zwischen Mensch und Roboter. Viele bestehende Detektionspipelines sind Closed Set Ansätze und bieten daher nur ein begrenztes kontextuelles Verständnis in sich verändernden Szenen. Insbesondere gängige Objektdetektoren wie YOLO können reichhaltigere semantische Kontexte, zusätzliche Labels oder posebezogene Relationen über die Zeit nicht nativ in einem einheitlichen Weltmodell aufrechterhalten. Zur Schließung dieser Lücke präsentiert die Arbeit ein modulares Framework zur Online Generierung von Scene Graphs (SG), das RGBD Sensorik, visuelle Odometrie, die Erzeugung von Graph Cutouts und Inferenz mit VLMs kombiniert. Zusätzliche Module für Segmentierung, Prompt Konstruktion und Fusion der Graphen sind in eine bildweise Verarbeitungspipeline integriert, die auf Interoperabilität und Modellaustausch ausgelegt ist. Das Framework umfasst außerdem Mechanismen zur Reduktion von Halluzinationen der VLMs und von instabilen Ausgaben, die insbesondere bei kleineren Modellen relevant sind.

Das Framework wurde in zwei Labortestläufen evaluiert, in einer statischen Umgebung mit bewegter Kamera sowie in einem dynamischen Szenario mit einer sich bewegenden Person. Im statischen Lauf erreichte das System eine Präzision der Knotenklassifikation von circa 95% und eine Kantenpräzision von etwa 90%. Im dynamischen Lauf, in dem die Ansicht geringer war und Reflexionen Tiefenartefakte verursachten, lag die Knotenpräzision bei circa 80% und die Kantenpräzision zwischen 85% und 90%. Die Präzision ist dabei als Anteil korrekt klassifizierter Objektvorhersagen des VLM an allen vom VLM vorhergesagten Labels definiert. Der erscheinungsbasierte Trackingmechanismus fügte dynamische menschliche Knoten im Zeitverlauf konsistent ein und entfernte sie wieder, was eine stabile zeitliche Graphpflege demonstriert.

Insgesamt zeigt die Arbeit die technische Machbarkeit einer Online Generierung von SG auf Basis von VLMs für die dynamische Risikowahrnehmung. Der aktuelle Umfang liegt auf der Wahrnehmung und nicht auf der Risikobewertung. Das System baut aus visuellen Beobachtungen inkrementell einen semantischen SG auf und hält ihn aufrecht, während eine globale auf SLAM basierte räumliche Korrektur bislang nicht implementiert ist. Dennoch bleibt das Wiedererkennen bereits besuchter Positionen durch persistente Knotenpositionsspeicherung und Tracking funktionsfähig. Der resultierende Graph stellt eine strukturierte und erweiterbare Repräsentation bereit, die von nachgelagerten Modulen für eine auf SG basierte Risikobewertung in zukünftigen Arbeiten genutzt werden kann.

Table of Contents

Abstract i

Zusammenfassung ii

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Motivation . . . 1

1.2 Relevance . . . 1

1.3 Goal of the Thesis . . . 2

2 Theoretical background 3

2.1 Human-Robot Interaction (HRI) . . . 3

2.1.1 Safety by separation . . . 3

2.1.2 Safety by supervision and coexistence . . . 4

2.1.3 Full collaboration . . . 5

2.2 Risk management for AS . . . 6

2.2.1 Risk Assessment in AS . . . 6

2.2.2 ISO Safety Standards . . . 7

2.2.3 Limitations of ISO Standards for HRI . . . 7

2.2.4 Risk Assessment Methods . . . 8

2.2.5 SLAM as one Enabling Technology for Safety . . . 9

2.3 Scene Graphs (SGs) . . . 10

2.4 Visual Language Models (VLMs) . . . 11

2.5 Enhancing VLM Performance for Context Recognition . . . 12

2.5.1 Prompt Engineering . . . 12

2.5.2 Few-Shot Learning . . . 12

2.6 Retrieval-Augmented Perception (RAP) for Semantic Memory . . . 13

2.6.1 Embedding-Based Retrieval . . . 13

2.6.2 Application to Safety-Critical AS . . . 13

2.7 Risk indicators . . . 13

2.8 Related Work . . . 14

2.8.1 ConceptGraphs . . . 14

2.8.2 OpenGraph . . . 16

2.9 Comparison with Existing Approaches . . . 17
2.9.1 Real-Time Processing . . . 17
2.9.2 API Independence and Asynchronous Input Handling . . . 17
2.9.3 Model Interchangeability and Extensibility . . . 17
2.9.4 Ease of Deployment and Integration . . . 18
2.9.5 Summary of Comparative Advantages . . . 18
2.10 Cloud and Edge computing . . . 19
2.10.1 Cloud computing . . . 19
2.10.2 Edge computing . . . 20
2.10.3 Hybrid approaches . . . 20
2.11 Ethical Considerations & Safety Compliance . . . 21
2.11.1 Ethical Implications . . . 21
2.11.2 Responsible AI Deployment . . . 21
2.11.3 Explainability & Transparency . . . 22

3 Methodology **23**
3.1 Overall System Architecture . . . 23
3.2 Communication Protocols . . . 24
3.2.1 Comparison with ROS . . . 24
3.2.2 Limitations and Trade-offs . . . 24
3.2.3 Endpoint Design and Functionality . . . 25
3.3 Edge device . . . 25
3.3.1 NVIDIA Jetson Orin . . . 25
3.4 Evaluating VLM models . . . 26
3.4.1 Input data and prompts . . . 27
3.4.2 Llava 1.6 . . . 28
3.4.3 Qwen 2.5 . . . 29
3.4.4 Phi 4 Vision Multimodal . . . 30
3.4.5 Quality comparison . . . 31
3.4.6 Speed comparison . . . 31
3.4.7 Hallucination . . . 32
3.5 Framework design . . . 33
3.6 Data preprocessing . . . 34
3.6.1 Collecting images and IMU data . . . 34
3.6.2 Creating a cutout of the SG . . . 36

3.6.3 Preprocess the SG . . . 38
3.7 SG Representation and Constraints . . . 38
3.8 Formal Graph Definition . . . 39
3.9 Coordinate Conversion . . . 39
3.10 Graph combination strategy . . . 41
3.11 Performance Optimization & Scalability . . . 41

4 Implementation **43**
4.1 Class Diagram . . . 43
4.2 Overall process for one frame . . . 45
4.3 Retrieving additional information . . . 46
4.4 Generating cutout of the SG . . . 47
4.5 Inference process . . . 48
4.6 Combining SGs . . . 49
4.7 Learning unknown objects . . . 50
4.8 Debugging and testing . . . 50
4.9 Dataset Usage . . . 51

5 Results **52**
5.1 Test runs . . . 52
5.1.1 Experimental setup . . . 52
5.1.2 Test run in static environment . . . 53
5.1.3 Test run with moving human . . . 55
5.2 VLM performance . . . 57
5.2.1 Inference performance and output stability . . . 57
5.2.2 Prompt engineering and output format . . . 57
5.2.3 Semantic understanding and limitations . . . 58
5.2.4 Framework implications and benchmark caveats . . . 58
5.3 Handling model limitations . . . 59
5.4 Future Work . . . 60
5.4.1 SLAM . . . 60
5.4.2 Different VLM . . . 60
5.4.3 Risk Assessment . . . 61

6 Summary **61**

Bibliography **63**

Appendix **67**

A VLM Prompts 67

A.1 1. Prompt 67

A.2 2. Prompt 67

A.3 3. Prompt 67

A.4 4. Prompt 68

A.5 Complete prompt for the VLM 68

List of Figures

1 KUKA Automotive Industry (KUKA AG, 2025) 4
2 framework from Mavsar, Simonič and Ude (2025) 6
3 SG proposed by Johnson et al. (2015) 10
4 Layered SG by Armeni et al. (2019) 11
5 Pipeline of ConceptGraphs by Gu et al. (2024) 15
6 OpenGraph hierarchical representation (Deng et al., 2024) 16
7 Cloud vs Edge Computing Architecture 19
8 XAI for image recognition (F. Mumuni and A. Mumuni, 2025; Selvaraju et al., 2017) 22
9 freiburg3_long_office_household dataset 27
10 Llava 1.6 percentages for different prompt and quantization combinations 28
11 Qwen 2.5 percentages for different prompt and quantization combinations 29
12 Phi 4 Vision Multimodal percentages for different prompt and quantization combinations 30
13 Comparison of average and best percentages for each VLM 31
14 Overall framework design 33
15 Difference between distorted and undistorted images in TUM-Dataset 35
16 Difference between distorted and undistorted images in PlantCV example 35
17 Checkerboard image by OpenCV 36
18 Camera frustum illustration 36
19 Class Diagram 43
20 Overall Process of one frame 45
21 State Diagram of frame states 45
22 Sequence diagram for retrieving additional information 46
23 Sequence diagram for generating cutout of the SG 47
24 Inference process of the VLM 48
25 Combining SGs 49
26 Sequence diagram for learning unknown objects 50
27 Scene images from the lab 53
28 SGs from the lab 54
29 Scene images and graphs of a person walking 56
30 Depth and reflection artifacts 57

List of Tables

1 Comparative analysis of framework characteristics across ConceptGraphs, OpenGraph, and the proposed approach 18

2 Llava 1.6 evaluation results 29

3 Qwen 2.5 evaluation results 30

4 Phi 4 Vision Multimodal evaluation results 31

5 Tokens per second comparison 32

6 Confusion Matrix: Classification Outcomes and Definitions 41

1 Introduction

This chapter introduces the motivation, relevance, and goals of the thesis. It outlines the practical safety challenges that arise in Human-Robot Interaction (HRI) and frames why dynamic, context-aware risk perception is needed. The section also summarizes the proposed framework and its intended contributions.

1.1 Motivation

The accelerated deployment of collaborative robots (cobots) and Autonomous Systems (AS) in industrial environments increases the need to evaluate safety mechanisms that operate reliably in close HRI. The International Federation of Robotics reports a 12% increase in industrial robot installations in 2024, with cobots rising from 2.8% of installations in 2017 to 11.9% in 2024 (International Federation of Robotics, 2025). These values indicate greater exposure to collaborative operation, which demands robust safety assessment methods capable of real-time hazard detection and mitigation.

Despite advancements in sensing and compliance technologies, accident analyses continue to show that injuries often occur during non-routine tasks such as maintenance or system setup (Lee, Shin and Lim, 2021). These incidents frequently stem from insufficient environmental understanding and the inability of robots to adapt their risk assessment to dynamic, unstructured surroundings. Existing safety standards largely assume predictable environments and predefined hazard zones, assumptions that no longer hold as AS become increasingly mobile and context-aware (Lee, Shin and Lim, 2021).

This gap motivates the need for AS capable of interpreting their surroundings at a semantic level, identifying emerging hazards, and adjusting their behavior in real time. As AS transition from static workcells to changing, human-centered environments, safety can no longer rely solely on pre-engineered constraints. Developing perception-driven, context-aware risk assessment methods integrated directly into AS control architectures is therefore essential for enabling genuinely safe HRI.

1.2 Relevance

The limitations of current safety approaches become particularly evident when AS operate in environments where human behavior, object configurations, and task demands change continuously. Existing risk assessment frameworks typically rely on static models or predefined hazard maps, which cannot capture the evolving spatial and semantic relationships that characterize real collaborative settings. As a result, AS often lack the ability to detect emerging risks or to adapt their actions proactively (Prof. Dr.-Ing. Andrey Morozov, 2024).

This thesis directly addresses this gap by introducing a framework that leverages dynamic Scene Graphs (SG) enriched with risk-annotated semantics. Unlike traditional methods, this approach enables the AS to maintain an up-to-date representation of its environment and provides a tool to reason about potential hazards at runtime. By embedding risk perception into the AS framework and decision-making pipeline, the framework aims to provide a level of situational awareness that current safety systems do not offer.

The need for such adaptive methods is underscored by recent findings studies that industries tend to invest in advanced robotic systems only after experiencing elevated injury rates (De Simone, Guarascio and Reljic, 2025). This reactive pattern suggests that safety mechanisms for emerging technologies often lag behind their deployment. For AS, still relatively new in many industrial contexts, this means that existing safety frameworks may not yet capture the full spectrum of risks associated with dynamic and human-centered environments. Adressing this gap before injury rates rises is therefore both a technical challenge and an ethical obligation for researchers.

1.3 Goal of the Thesis

This master's thesis proposes a framework for AS that generates a dynamic SG with risk-annotated semantics. An example implementation is demonstrated on different datasets and camera data. The core objective is to design a fully functional system capable of detecting and annotating potential risks in the environment, followed by constructing a complete SG at runtime.

The framework collects and processes data from multiple sensors to build an accurate representation of the environment. This SG can be segmented into subgraphs to reflect the AS current situation. Risk annotations and object detections are based on temporal or static objects in the environment and are performed by a Visual Language Model (VLM), which can be fine-tuned to detect specific types of risk. A dedicated endpoint is provided to access the full graph during runtime.

A secondary focus is on interoperability and modularity. The framework is designed to be easily adaptable to other AS platforms with minimal changes. Components can be replaced or extended to suit different use cases, for example the VLM can be swapped out for another model to benchmark its performance in risk detection and annotation. While a full evaluation of risk assessment using the SG is beyond the scope of this thesis, a basic benchmark is included, along with suggestions for future research.

2 Theoretical background

This chapter establishes the foundational concepts and core technologies for intelligent safety assessment in HRI. Core principles in AS safety and risk assessment methodologies are first reviewed, including relevant ISO standards. Subsequently, the technical foundations are examined, including SGs for spatial representation, VLMs for environmental understanding, and techniques for performance enhancement through Retrieval-Augmented Perception (RAP) and parameter optimization. Computational architectures for edge and cloud deployment are also addressed. Finally, this approach is positioned within existing literature and key ethical considerations for Autonomous Safety Systems (AuSS) in collaborative environments are discussed.

2.1 Human-Robot Interaction (HRI)

The classification of HRI has undergone considerable evolution in recent years. Villani et al. (2018) note that earlier approaches attempted to categorize HRI through extensive taxonomies featuring eleven distinct dimensions, including task characteristics, AS design, role distribution, temporal, and spatial arrangements. However, they argue that such detailed categorizations may no longer adequately represent contemporary developments in the field.

In response to these limitations, De Luca and Flacco (2012) proposed a refined framework that structures HRI into three hierarchical levels: safety, coexistence, and collaboration. According to their conceptualization, these levels form a nested structure where achieving higher levels of interaction presupposes that all requirements of preceding levels are satisfied. The foundational level focuses on ensuring safe operation without physical harm. Building upon this foundation, coexistence describes situations where humans and AS share workspace and may manipulate common objects, yet operate without coordinating their actions or intentions. It must be noted that Villani et al. (2018) also show that this definition from De Luca and Flacco (2012) is not applied consistently across the literature, with different authors drawing the boundary between coexistence and collaboration in varying ways.

Furthermore, Yanco and Drury (2004) draw an important conceptual distinction between HRI and Human-Robot Collaboration (HRC). They explain that HRC specifically describes scenarios characterized by coordinated joint work toward common goals, whereas HRI represents a more general term encompassing any form of interaction between humans and robots, including those without shared objective.

While their framework provides a theoretical hierarchy of three interaction levels, practical implementations of the foundational safety level can take different forms depending on the application context and robot characteristics. In industrial settings, two distinct approaches to achieving safe HRI have emerged: safety by separation and safety by supervision (Villani et al., 2018). These represent different strategies for satisfying the safety requirements that form the foundation of the nested framework. Once this safety foundation is established through either approach, systems can progress to higher levels of coexistence or full collaboration as described in their framework.

2.1.1 Safety by separation

A widely used safety strategy in industrial robotics is based on spatial separation between humans and robots rather than direct physical interaction control. In this approach, a restricted operational area is defined around the robot, and human access to this space is not allowed during operation. If a person enters this predefined safety zone, the robot immediately stops its motion and interrupts its current task to prevent injury. This concept is commonly implemented through clearly marked safety areas or by enclosing the robot within a physical safety cage (Quarta et al., 2017).

This safety method is strongly influenced by the type of robot used. Most industrial robots are large, powerful machines designed for high-speed and high-force operations such as welding, pressing, or handling heavy components. Due to their size, mass, and kinetic energy, these robots

are not designed for direct human collaboration. As a result, safety by separation is often the only feasible and accepted protection strategy for such systems. The intention is to ensure that humans and robots never occupy the same workspace during active operation.

Notably, this approach does not rely on external sensing technologies such as cameras, laser scanners, or distance sensors for harm prevention. All sensors integrated into the system are used exclusively for the robots internal control, including position monitoring, motion execution, and process accuracy. These sensors enable the robot to perform its assigned tasks but do not contribute to human detection or collision avoidance (Prof. Dr.-Ing. Dr. h. c. Michael Weyrich, 2024).

A representative example of this safety concept can be found in the automotive industry, where large industrial robots such as KUKA systems (KUKA AG, 2025) are widely used (Figure 1). These robots handle heavy vehicle body parts, perform spot welding, and execute other force-intensive tasks. Because they are not designed for collaborative operation, they are typically installed in fenced production cells. While the robot is active, human access to the cell is strictly prohibited. If a safety gate is opened or the protected area is breached, the robot automatically stops, ensuring worker safety through enforced spatial separation (Prof. Dr.-Ing. Dr. h. c. Michael Weyrich, 2024).

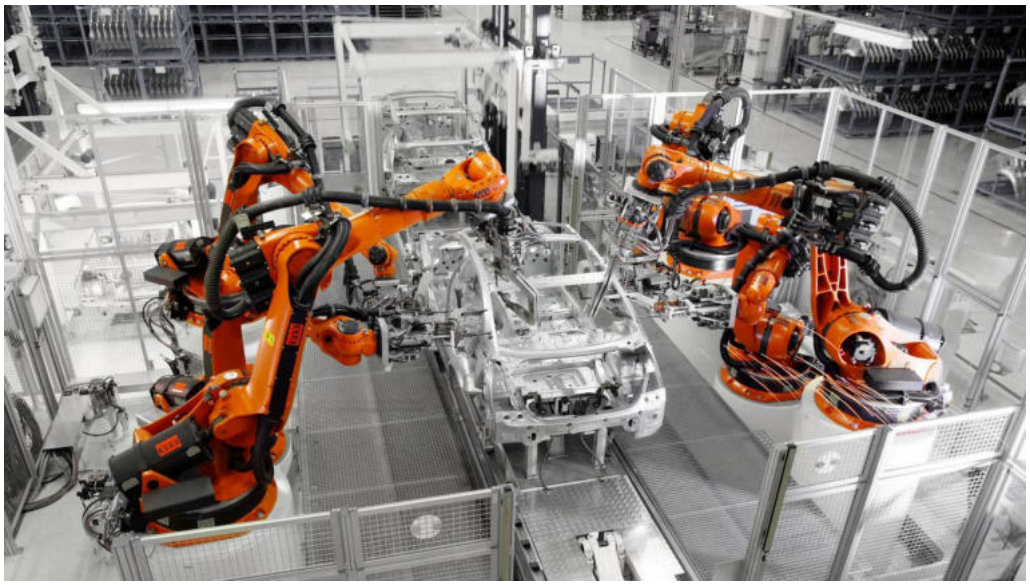


Figure 1: KUKA Automotive Industry (KUKA AG, 2025)

2.1.2 Safety by supervision and coexistence

Safety by supervision enables shared workspaces without strict spatial separation by combining intrinsic robot safety mechanisms with continuous human oversight. Under this approach, humans and robots share the same physical workspace and may work on the same object or in close proximity, yet without direct coordination. This configuration corresponds to what Villani et al. (2018) identify from prior literature as coexistence.

This shift from spatial separation to contact management transforms the safety problem from architectural to operational. Rather than preventing access, the system must manage collision risks through intrinsic sensing. Cobots typically employ joint torque sensing, motor current monitoring, and impedance control to detect unexpected contact and respond through motion modification or controlled stopping (Mohammad, Schappler and Ortmaier, 2023). However, this reactive approach has a fundamental limitation. It can only limit impact severity, not fully prevent contact. Improperly configured thresholds for torque, force, or impedance may still result in harmful interactions (International Organization for Standardization, 2016). For cobots with limited payload, appropriately adjusted parameters can provide sufficient protection without additional hardware,

making this approach suitable for close HRI.

The critical vulnerability in safety by supervision lies in operator performance. Continuous monitoring requires sustained attention, situational awareness, and rapid responses to unexpected events. Studies show that this cognitive burden can reduce operator performance and negatively affect safety outcomes. Therefore, intuitive human-machine interfaces with clear visual feedback, haptic cues, or augmented reality support are essential. Not merely as conveniences, but as necessary safeguards to maintain operational safety (Apraiz et al., 2025).

Rather than relying solely on reactive collision detection, AS integrate a comprehensive range of passive and supervisory safety measures into their mechanical and control design. These include rounded edges, compliant joints, soft surface materials, lightweight structures, limits on speed and force, and mechanical torque limiters or fuses that operate independently of software to provide fail-safe protection (International Organization for Standardization, 2016). This layered approach, combining passive mechanical safeguards with software-based measures, reflects a multi-layered protection strategy where multiple independent mechanisms work together. Modern safety by supervision systems extend beyond purely reactive collision handling by incorporating predictive or preventive strategies such as human motion modeling (Zheng et al., 2022), vision- or depth-based workspace monitoring (Amaya-Mejía et al., 2022), dynamic adaptation of safety zones based on human proximity (Bricher and Müller, 2025), and data-driven anomaly detection (Brockmann et al., 2023). These enhancements enable the system to identify and respond to unusual AS behavior before it escalates to hazardous interactions, significantly improving overall system safety.

This coexistence mode is particularly relevant in scenarios where workspace efficiency is important but full task coordination is not required. For example, in assembly lines where a human operator performs manual tasks on one side of a workpiece while a AS handles automated operations on another side, both share the workspace and the object without direct interaction. In full collaboration, humans and AS coordinate their tasks, but in coexistence, they work independently even when sharing the same space. While the safety requirements remain high and accidental collisions must be prevented through appropriate sensing, motion planning, and control strategies, the system does not need to interpret human intentions or coordinate complex multi-agent task sequences, simplifying the technical implementation compared to full collaboration. Clear task allocation and bounded workspaces allow humans and AS to work in close proximity for flexible, frequently reconfigured assembly or handling tasks. When paired with training and careful parameterization, this mode delivers efficient workspace sharing while maintaining high safety standards.

2.1.3 Full collaboration

Full collaboration represents the most advanced level of HRI, in which humans and AS work jointly toward a shared goal through direct interaction, dynamic task coordination, and continuous mutual adaptation. At this level, AS must not only ensure safety in shared workspaces, but also interpret human intentions and adapt their behavior accordingly.

A key requirement for full collaboration is the AS's ability to understand what the human intends to do and to react proactively. Rather than responding only after human actions occur, the AS must anticipate future actions in order to avoid conflicts, provide assistance, and synchronize task execution. An example of this approach is presented in (Mavsar, Simonič and Ude, 2025), where human pose estimation and learning-based models are used to classify hand trajectories and predict the human's intended goal in real time. The estimated intention is integrated into the AS's control architecture, enabling online adaptation of motion planning and task execution. Their proposed framework is shown in Figure 2.

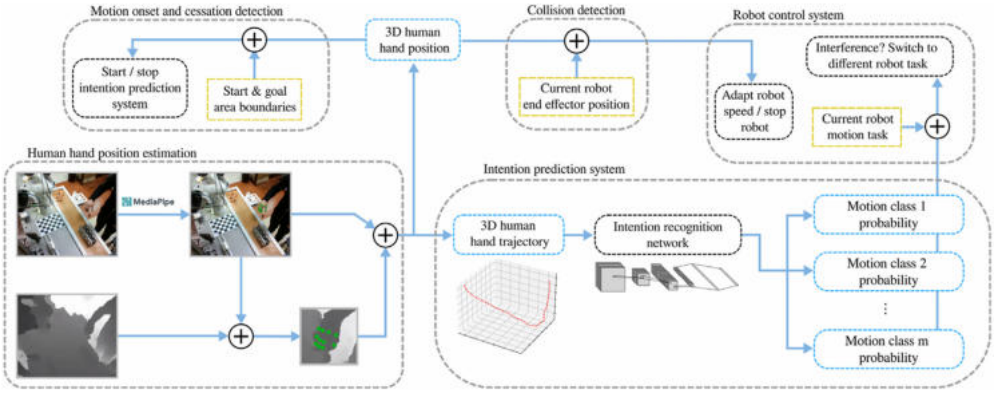


Figure 2: framework from Mavsar, Simonič and Ude (2025)

In such intention-aware collaborative systems, human intention recognition is tightly coupled with motion planning and collision avoidance. When the AS predicts that the human intends to interact with a specific object or workspace, it can adjust its trajectory, yield priority, or modify its task allocation in advance. This anticipatory behavior is a defining characteristic of full collaboration, distinguishing it from lower interaction levels that rely primarily on reactive safety mechanisms.

Full collaboration also requires bidirectional understanding between human and AS. While intention recognition enables the AS to infer human goals, humans must be able to understand the AS's intended actions to maintain trust and ensure smooth coordination. This aspect is addressed in (Zhang, Yan and Y. Jia, 2025), where mixed-reality interfaces are used to visualize AS intentions, such as planned motion paths or upcoming actions, directly to the human collaborator. Experimental results show that explicitly communicating AS intent improves task efficiency, user comfort, and perceived safety in collaborative scenarios (Zhang, Yan and Y. Jia, 2025). By combining human intention recognition with intuitive communication of AS behavior, collaborative systems achieve more natural and efficient interaction.

In summary, the progression from safety-focused approaches through coexistence to full collaboration represents an evolution toward increasingly sophisticated HRI systems. While each level builds upon the safety foundations of its predecessors, full collaboration demands advanced mechanisms for mutual understanding and coordination that transform the HRI from reactive risk mitigation into proactive, goal-oriented partnership.

2.2 Risk management for AS

This subsection covers risk assessment and management strategies for AS. Fundamental risk assessment concepts and traditional safety approaches are first introduced, followed by relevant ISO safety standards and established risk assessment methodologies. The discussion then examines enabling technologies such as SLAM that support runtime-capable safety mechanisms, before addressing the role of perception-based systems in identifying hazards beyond predefined models.

2.2.1 Risk Assessment in AS

In many traditional engineering domains, risk assessment is conducted primarily during system design to identify hazards before deployment (Chu, Yang and Chen, 2024). This design-time approach assumes relatively stable operating conditions and relies on predefined failure modes and expert assumptions. In AS, risk assessment must also continue at runtime (Huck et al., 2021), because safety-relevant conditions can change during operation, especially in HRI and dynamic environments.

Existing approaches to runtime risk assessment also rely on fixed experimental setups and pre-

defined models of human and AS motion. Representative examples include the works of Liu et al. (2020), Dittrich and Woern (2016) and Katranis et al. (2025), which use human and robot trajectory models to predict collisions. Some methods like from Liu et al. (2020) define danger zones or compute predicted paths to assess risk levels in advance. While such approaches are effective in controlled environments, they exhibit limitations when applied to AS operating in highly dynamic and unstructured settings.

For collaborative AS, both design-time and runtime risk assessment approaches must adapt to the inherent complexity of HRI. Traditional design-time assessments are often unable to capture the full range of behaviors and environmental changes encountered during operation, whereas runtime assessments must respond robustly to unforeseen hazards in real time. This dual requirement introduces new challenges for risk management, particularly with respect to adaptability, perception, and computational feasibility in dynamic environments.

Given these constraints, runtime risk assessment is the decisive capability for AS in unstructured, human-centered environments. Design-time analysis remains necessary for baseline safety requirements, but it cannot capture the evolving hazards of collaborative workspaces. Current runtime methods still depend heavily on fixed setups, predefined motion models, and trajectory-based prediction. This dependence motivates semantically aware approaches that maintain a dynamic, context-rich representation of the environment and infer hazards from semantic relations in addition to geometric or kinematic cues.

2.2.2 ISO Safety Standards

Risk assessment for industrial machinery and AS systems is introduced by several ISO standards that ensure safety, reliability, and a structured approach to hazard management. The foundational framework is provided by ISO 12100 (International Organization for Standardization, 2011b), which outlines a systematic process for identifying and reducing risks. This begins with defining the machines operational limits, followed by identifying potential hazards and evaluating both their severity and likelihood. The process is iterative, meaning that each new safety measure must be reassessed, as it may introduce additional risks that require mitigation. An addition to this norm, ISO/TR 14121 (International Organization for Standardization, 2013) offers practical guidance for conducting assessments effectively. It recommends using both top-down and bottom-up approaches to hazard analysis and emphasizes the importance of involving multidisciplinary expert teams to ensure a comprehensive evaluation.

For robotic systems, ISO 10218 (International Organization for Standardization, 2011a) specifies safety requirements for both the robot itself and its integration into the working environment. It mandates a full risk assessment in accordance with ISO 12100 and includes a robot specific hazard checklist to address unique risks associated with automation. Building on this, ISO/TS 15066 (International Organization for Standardization, 2016) focuses on cobots, where humans and robots share workspaces. It defines four distinct modes of collaboration: hand guiding, sensor based monitored stop (e.g. lightbarrier), power and force limiting and speed and separation monitoring. Each mode has tailored safety requirements and risk assessment procedures to ensure safe HRI. Together, these ISO standards provide a robust framework for systematically assessing and mitigating risks in traditional robotic systems.

2.2.3 Limitations of ISO Standards for HRI

While these standards reflect widely accepted best practices for machinery and cobots safety, they also exhibit limitations that become increasingly relevant in modern HRI scenarios and systems incorporating AI-based components. In particular, the following limitations can be identified:

- **Static hazard models:** ISO 12100 assumes relatively stable tasks and environments. In HRI, non-deterministic human behavior and changing context can invalidate preliminary risk estimates and require continuous monitoring and reassessment.

- **Human variability:** The power, force limits and pain threshold data in ISO/TS 15066 are population averages established under controlled conditions. They may not cover vulnerable users, long-term exposure, startle responses, or short duration force spikes during contact-intensive tasks.
- **Sensor reliability and SSM:** Speed and separation monitoring depends on perception systems subject to occlusion, latency, and false positives or negatives. Deriving and maintaining a guaranteed protective separation distance is challenging in cluttered, dynamic workplaces.
- **Coverage gaps:** Cognitive workload, trust, and psychosocial risks in HRI are not explicitly addressed. Learning systems that update online fall outside many validation assumptions, creating a gap between standard guidance and AI-driven behavior.

2.2.4 Risk Assessment Methods

In practice, the risk assessment methods defined by ISO standards are applied using a wide range of techniques and models. While the ISO norms propose general frameworks and procedures for evaluating risks, these are often adapted, extended, or combined with newly developed methods to suit specific applications and industrial contexts. Moreover, the resulting approaches can generally be categorized into qualitative and Quantitative Risk Assessment (QRA) methods, depending on whether they rely on descriptive evaluations or numerical analysis (Prof. Dr.-Ing. Andrey Morozov, 2024):

- **Qualitative:** Methods like FMEA, HAZOP, PrHA, and What-If are used to systematically identify hazard scenarios based on expert knowledge. For example, FMEA evaluates failure modes of components and their effects on the system, while HAZOP investigates deviations from design intent using guide words like "No," "More," or "Reverse." PrHA is applied early in the design phase to prioritize hazards when little operational data is available. What-If methods rely on brainstorming to explore hypothetical failure scenarios. These methods require experienced analysts and structured procedures to ensure completeness and consistency (Primatech Inc., 2017).
- **Quantitative:** Methods such as Fault Tree Analysis (FTA), Event Tree Analysis (ETA), Layer of Protection Analysis (LOPA), and Bayesian Networks (BN) assign numerical probabilities to failure events and model their consequences. These techniques require detailed failure rate data and consequence modeling to calculate risk values. For example, FTA maps out the logical combinations of failures that could lead to a top event, while ETA explores the possible outcomes following an initiating event. Static approaches such as FTA or BN evaluate the system under the assumption that its behavior and environment remain constant over time. While they offer a snapshot of risk, they can be extended with reliability functions to reflect time-dependent degradation. In contrast, dynamic methods like DFT or Discrete-Time Markov Chains (DTMC) model how system states and external conditions evolve, incorporating stochastic processes to capture complex temporal behavior. These dynamic techniques require more advanced modeling and computational resources but provide a more realistic representation of risk over time (Prof. Dr.-Ing. Andrey Morozov, 2024).

Despite their complexity, which arises from extensive state-space modeling and stochastic analysis, quantitative methods rely on a predefined set of system parameters, failure modes, and environmental assumptions established during the models implementation phase. Inputs such as component reliability data, operational procedures, and hazard scenarios are typically fixed and cannot adapt to unforeseen risks or evolving system behaviors unless explicitly modeled in advance (Robla-Gómez et al., 2017). Moreover, dynamic methods like DFTs or DTMCs often involve complex state-space representations and stochastic transitions, which result in significant computational overhead. This makes runtime evaluation impractical in most industrial or household settings, especially when system conditions change rapidly. As a result, conventional risk assessment frameworks struggle to cope with highly dynamic environments without substantial computational resources (Rao et al., 2009). Therefore, directly transferring risk models developed

for traditional industrial robots to AS is insufficient unless the unique challenges of HRI, environmental variability, and runtime adaptability are explicitly addressed in the modeling approach.

For AS, it is difficult to design a model that accounts for all possible risks. Unlike fixed environments, AS operate in highly dynamic settings where multiple hazards can emerge unpredictably. This variability makes it impossible to define a closed set of potential dangers. As a result, unforeseen issues may arise that are not documented within predefined risk categories. These limitations are also acknowledged in ISO 12100 (International Organization for Standardization, 2011b), which implicitly recognizes that not every hazard, particularly those that cannot be reasonably foreseen, can be enumerated. Such rare events also resist probabilistic modeling, for example with Markov chains, because their infrequency provides insufficient data to establish reliable risk indicators.

Furthermore, hazardous situations often emerge from relations between objects rather than from the objects in isolation. For example, a heater and a cloth may be safe individually, but their proximity creates a fire risk. Because the number of possible object configurations and contexts is very large, exhaustive predefined modeling is impractical. This motivates adaptive methods that can infer risk from contextual relations at runtime.

These limitations highlight the need for runtime-capable safety mechanisms that rely on continuous perception of the environment rather than predefined hazard models. One such enabling technology is SLAM, which provides the spatial awareness required for adaptive and proactive safety strategies.

2.2.5 SLAM as one Enabling Technology for Safety

SLAM is widely used in AS for localization in indoor and outdoor environments. It jointly estimates the AS pose and builds an environmental map, which is why it is common in mobile robots, drones, and autonomous vehicles. For long-horizon localization, such map-based correction is essential because purely odometric or inertial estimates accumulate drift over time (Cadena et al., 2017). Despite this maturity, safety-critical deployment remains challenging because practical implementations still suffer from issues such as drift, z-axis inaccuracies, and loop-closure limitations (Feng et al., 2025). As a result, robust integration of SLAM remains an active research topic and depends strongly on application context (Deng et al., 2024; Rosinol et al., 2021; Sturm et al., 2012).

For AS, SLAM provides the geometric foundation for spatial safety. Continuous mapping and localization support obstacle-aware navigation and collision avoidance with static structures such as walls (Deng et al., 2024). Classical SLAM, however, represents geometry rather than semantics and does not inherently classify object types such as humans, tools, or furniture (Cadena et al., 2017). In practice, dynamic object tracking and human detection are therefore added through complementary sensing and vision modules, for example camera- or LiDAR-based perception and detectors such as YOLO (Hughes, Chang and Carlone, 2022).

When these modules are fused with SLAM, the system can react to dynamic hazards during operation. It can replan trajectories, reduce speed near workers, and trigger controlled stops when a human enters the projected path (Mavsar, Simonič and Ude, 2025). This makes SLAM a strong enabler for geometric risk mitigation in shared workspaces, particularly under changing environmental conditions.

Nevertheless, SLAM alone is not a complete safety solution. It primarily addresses geometric risk, while non-geometric factors such as object semantics, human intent, task context, and procedural constraints require additional reasoning layers. Comprehensive risk management therefore requires integration with higher-level perception and risk assessment frameworks, as well as validation within existing safety standards and certification processes. Future work should focus on improving robustness, reducing mapping and localization errors, and strengthening dynamic-scene handling so that SLAM can serve reliably as one component of a broader safety architecture.

2.3 Scene Graphs (SGs)

Current methods in AS increasingly use SGs to achieve a structured and interpretable representation of the environment. The concept of SGs was first introduced by Johnson et al. (2015). These explicitly encode objects, their attributes, and the relationships between them, enabling a more detailed and semantically rich understanding of scenes compared to conventional visual feature representations such as low-level descriptors or convolutional embeddings like GIST or CNN features. An example graph is given in figure 3. Their results demonstrated that this structured representation improves semantic image retrieval, object localization, and contextual reasoning, since it captures how objects interact spatially and semantically rather than treating them as isolated detections (Johnson et al., 2015).

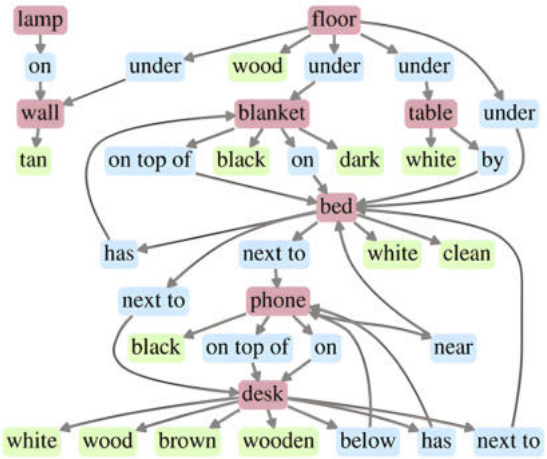


Figure 3: SG proposed by Johnson et al. (2015)

Johnson et al. (2015) also discuss why text-only scene descriptions are insufficient for detailed visual reasoning. Natural language is ambiguous (e.g., "the person near the table") and becomes inefficient when many objects and relations must be described. Converting long text into a structured representation further requires object-reference linking, relation extraction, and visual grounding, each introducing additional error sources. SGs reduce this ambiguity by directly encoding objects, attributes, and relations in a grounded structure derived from the image, which improves reliability for reasoning, retrieval, and scene understanding.

Building on these advantages, more recent approaches have extended scene-graph construction through automatic generation using models such as CLIP and related VLMs (Hughes, Chang and Carlone, 2022; Ji et al., 2020; Schmid et al., 2024; Shang et al., 2021). These methods scale visual-relation annotation and improve multimodal grounding, but many still depend on predefined object categories and relation vocabularies. This closed-set assumption, together with limited runtime generation, constrains practical deployment. When unseen objects are omitted, the graph becomes incomplete and can mislead downstream reasoning, which is especially problematic for AS operating in changing environments with unfamiliar tools or layouts. In addition, several prominent systems, including approaches by Bavle et al. (2022), Rosinol et al. (2021), and Armeni et al. (2019), build graphs after the scene is captured via SLAM (Rosinol et al., 2021) or prerecorded data (Armeni et al., 2019) rather than directly supporting real-time perception in unconstrained operation.

Beyond deployment constraints, SGs also have representational limits. They describe primarily "what" and "where," but only weakly capture causality or intent. A relation such as "robot near human" encodes proximity, but not the reason for that proximity or the associated task context. Scalability is another issue, because graph size grows quickly with object and relation count in dense scenes. Relation semantics are also application-dependent, with no universal threshold for predicates such as "near," "supports," or "grasps." These limits can be mitigated with auxiliary reasoning modules that combine graph structure with current image evidence and global workspace

context. For example, VLMs can help disambiguate relations and refine incomplete graph content.

One structured extension that addresses part of these limitations is the layered scene-graph concept introduced by Armeni et al. (2019). Layered graphs (Figure 4) integrate semantic, geometric, and perceptual information across abstraction levels such as objects, rooms, and cameras. This hierarchy supports modular queries, for example which objects are visible from a viewpoint or which rooms contain specific object classes, and separates local object interactions from global spatial context. Subsequent work combines SLAM with layered graph generation, including Hydra (Hughes, Chang and Carlone, 2022) and Khronos (Schmid et al., 2024). However, current systems still show practical gaps: Hydra has limited dynamic-object handling, while Khronos focuses primarily on a dynamic layer with selected entities (e.g., humans and AS).

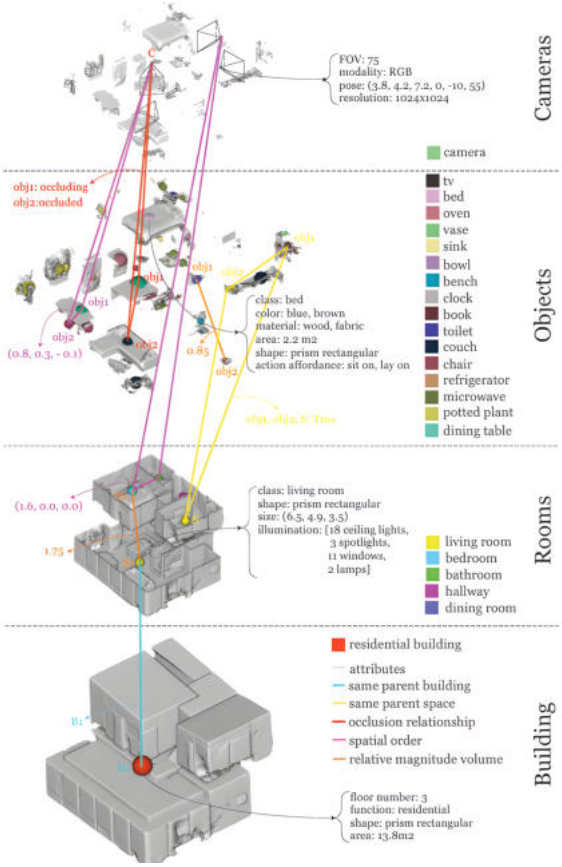


Figure 4: Layered SG by Armeni et al. (2019)

2.4 Visual Language Models (VLMs)

With the rise of Large Language Models (LLMs) and VLMs, including internVL3 (I. Team, 2025), LLava (L. Team, 2025), and GPT-4 (OpenAI, 2023), multimodal methods have become increasingly relevant for scene understanding and safety assessment. These models are trained on large image-text datasets and learn representations that connect visual content with language. As a result, they can support zero-shot classification, captioning, and visual question answering through prompts, which makes them attractive for flexible scene interpretation in changing environments.

For AS, this multimodal grounding is useful for identifying objects, interpreting relations, and flagging potentially unsafe situations. At the same time, model outputs remain context-dependent and not fully interpretable, so safety-critical deployment requires careful validation and system-level safeguards.

For domain-specific use, VLMs can be adapted through supervised fine-tuning or through Retrieval-Augmented Generation (RAG) pipelines. Supervised fine-tuning can improve task alignment, but

it is often resource-intensive and time-consuming because it requires curated labeled data and repeated training cycles (Dettmers et al., 2023). RAG combines a pretrained model with retrieval from external knowledge sources during inference, so runtime information can be incorporated without updating model weights (Y. Gao et al., 2023; P. Lewis et al., 2020). In practical deployments, previously observed objects, scene patterns, and annotations can be stored in the retrieval base and reused later. Such retrieval memory can improve consistency, coverage, and contextual grounding, while reducing the need for frequent retraining (Y. Gao et al., 2023). For edge deployment, this is relevant because edge devices provide only limited compute and memory resources (Jetson AI Lab, 2025). Under these constraints, smaller models enhanced with retrieval can offer a practical trade-off between performance and deployability.

Deployment also depends on inference efficiency. Although large models are resource-intensive, techniques such as quantization and hardware acceleration can reduce latency while preserving useful accuracy (Dettmers et al., 2023; Jetson AI Lab, 2025). These optimizations improve feasibility for near real-time hazard perception and response in AS scenarios.

Despite these advantages, VLM-based perception can still produce misclassification or scene misinterpretation. In safety-critical systems, such errors may propagate to planning and actuation. Therefore, VLM outputs should be integrated into safety architectures that include redundancy, fail-safe mechanisms, and human oversight to bound the operational impact of incorrect predictions.

2.5 Enhancing VLM Performance for Context Recognition

For context recognition tasks, VLM performance can be improved through targeted techniques that increase output precision and consistency. Since VLMs can be used to generate structured scene representations for risk assessment, reliable schema-compliant outputs are critical.

2.5.1 Prompt Engineering

For SG generation, the VLM must produce outputs in a structured format such as JSON or YAML that represents entities, relationships, and attributes for downstream risk assessment. The response must therefore be both semantically correct and schema compliant. Prompt engineering addresses this by explicitly encoding required keys, structural constraints, naming conventions, and allowed value ranges, sometimes with small JSON examples that illustrate the expected field types (Wei et al., 2022; K. Zhou et al., 2022). This improves adherence to a standardized, parseable format for robust downstream processing.

Ensuring consistent format compliance remains challenging. Prompt effectiveness depends on iterative refinement and empirical testing, because VLM behavior cannot be fully predicted analytically. Engineers typically test prompt variants, adjust wording, tune examples, and calibrate constraints until the model reliably emits valid SGs. This underscores that prompt design is an engineering practice grounded in experimentation and domain knowledge (Wei et al., 2022; K. Zhou et al., 2022).

2.5.2 Few-Shot Learning

Few-shot learning enhances VLM performance by providing representative examples in the prompt, enabling adaptation to task-specific patterns with limited labeled data (Alayrac et al., 2022; Brown et al., 2020). For context recognition in AS, this is especially useful for rare or domain-specific objects and interactions that may be underrepresented in pretraining data. Carefully selected examples can improve scene-graph accuracy, making example selection and retrieval a critical part of the method. More advanced RAP techniques for automating this selection process are discussed in Section 2.6.

2.6 Retrieval-Augmented Perception (RAP) for Semantic Memory

While LLMs and VLMs possess extensive parametric knowledge from pretraining, they can still exhibit knowledge gaps and hallucinations in domain-specific safety scenarios beyond their training data. In AS, these failure modes can lead to critical misinterpretations of hazards. RAP mitigates this limitation by augmenting models with external semantic memory: a structured, indexed knowledge base accessed at inference time through retrieval (W. Wang et al., 2025). When the knowledge base is well curated and domain aligned, grounding perception in retrieved evidence can reduce false classifications and hallucinations. RAP is not universally applicable, as its performance is tightly coupled to the quality of the underlying knowledge base, the reliability of retrieval, and the degree of alignment with the target operational domain (Y. Gao et al., 2023; W. Wang et al., 2025).

2.6.1 Embedding-Based Retrieval

Embedding-based retrieval defines the representation and similarity layer of RAP systems. In this approach, both queries and knowledge-base documents are encoded into high-dimensional embeddings using techniques such as neural networks or transformer models. During the retrieval process, the query embedding is compared to the stored document embeddings to identify the most relevant ones based on semantic similarity, for example cosine similarity or euclidean distance (Y. Gao et al., 2023). Compared with keyword retrieval, embedding-based retrieval can better capture semantic relationships and improve matching in domains where exact term overlap is insufficient. With suitable indexing, it can also support efficient search over large knowledge bases (W. Wang et al., 2025). These advantages depend on embedding quality, data curation, and appropriate similarity settings, which must be validated in practice. In RAP pipelines, high-quality retrieval improves access to contextually relevant evidence during perception. Conversely, incomplete coverage, semantic drift, or noisy embeddings can inject misleading context and degrade downstream model behavior (Y. Gao et al., 2023). To support efficient retrieval at scale, vector stores provide the storage and indexing layer for document embeddings, often using Approximate Nearest Neighbor (ANN) search structures that enable efficient similarity searches across large collections of vectors (Prof. Dr.-Ing. Dr. h. c. Michael Weyrich, 2024). In safety-critical settings, retrieval quality must be evaluated together with latency and consistency requirements.

2.6.2 Application to Safety-Critical AS

In collaborative AS, multimodal RAP may improve detection of rare or domain-specific objects when curated safety knowledge bases are available and maintained. Retrieved images, CAD drawings, or hazard labels can reduce ambiguity for VLMs when interpreting uncommon tools, fixtures, or PPE, and can support risk assessment when evidence is accurate and relevant (W. Wang et al., 2025). For example, if a VLM detects an unfamiliar gripper attachment during a HRI task, the RAP system can retrieve technical specifications, safety certifications, and operational constraints and combine this evidence with live sensor data to improve gripper identification, safety-compliance verification, and recommendations for speed or force limits. However, these benefits depend on current and complete documentation: outdated knowledge, incomplete domain coverage, or retrieval failures can still produce unsafe recommendations, making continuous validation essential.

2.7 Risk indicators

In safety assessment for AS, risk quantification follows the relationship $R = S \times P$, where risk (R) is determined by the product of severity (S) and probability (P) of a hazardous event (Prof. Dr.-Ing. Andrey Morozov, 2024). Severity represents the potential magnitude of harm that could result from a collision or unsafe interaction, while probability indicates the likelihood of such an event occurring under current operational conditions. In practice, both terms are operationalized through continuously measured indicators that can be computed during runtime.

Among various indicators, the distance is a reliable metric for risk assessment. It directly influences both terms of the risk equation, because shorter distances increase collision likelihood and reduce the time available for evasive actions or emergency stops, thereby increasing potential severity. Safety standards such as ISO/TS 15066 reflect this relationship by establishing protective separation distances and velocity constraints based on spatial proximity, such as velocities below 250 mm/s for hand-guiding operations (International Organization for Standardization, 2016). The widespread adoption of distance-based thresholds in industrial safety standards demonstrates that spatial separation provides a quantifiable and enforceable basis for risk management in AS.

Higher-resolution indicators are obtained from pose tracking of humans and AS using vision systems such as depth cameras (Zacharaki et al., 2020). From these trajectories, the system can derive relative distance, velocity, and motion direction indicators to detect elevated-risk configurations, for example a human entering the AS workspace during motion. These pose-derived indicators support adaptive control policies that tune AS's speed and trajectory according to current human movement (Katranis et al., 2025; Svenstrup et al., 2009). Compared with threshold-only distance sensing, this provides finer risk estimation and can improve both safety and efficiency in shared workspaces.

Beyond geometric and kinematic indicators, semantic indicators can be extracted from scene understanding. In the work of Ashour et al. (2022), semantic labeling transforms risk assessment from a purely geometric representation into a semantically enriched environmental model. Rather than modeling the operational space only through occupancy or kinematic constraints, the environment is mapped using hazard-aware categories that jointly encode hazard type and associated severity level. Through dense semantic segmentation of visual data, distinct sources of danger such as fire hazards, trip hazards, and combustible materials are identified and classified according to predefined risk levels, enabling precise spatial localization of hazardous regions. This information is subsequently integrated into a three-dimensional voxel-based occupancy map, resulting in a semantic risk representation where danger becomes an intrinsic property of the mapped space. Compared to bounding-box detection methods, pixel-wise semantic labeling provides a more accurate delineation of irregular hazard regions and supports their continuous projection into three-dimensional environments. By embedding hazard semantics directly into the mapping process, the approach enhances situational awareness and facilitates informed decision-making for AS.

The closed-set approach with predefined risk categories provides a structured and interpretable baseline for semantic risk mapping. A next step is to extend this baseline with relational and contextual reasoning so that emergent hazards from inter-object configurations can also be represented. For example, the alignment of obstacles within human traversal paths, may create risk conditions that are not fully captured by fixed taxonomies. Building on the semantic mapping paradigm demonstrated by Ashour et al. (2022), such extensions can improve generalizability to diverse and unforeseen hazard scenarios.

2.8 Related Work

During research for this thesis, only few works were found that combine VLMs with SGs for dynamic environment representation. Most existing works either focus on static SG generation from pre-recorded datasets or videos, or prioritize scene-understanding queries for environment traversal. This section examines two approaches that integrate VLMs with SG representations: ConceptGraphs, which targets indoor environments with rich semantic reasoning capabilities, and OpenGraph, which focuses on hierarchical 3D graph construction for large-scale outdoor scenarios.

2.8.1 ConceptGraphs

The first work to propose the integration of VLMs with SGs is presented by Gu et al. (Gu et al., 2024). In this work, the authors introduce ConceptGraphs, a framework that combines VLMs with SG representations to enhance visual understanding and reasoning. A key contribution of this framework is the use of layered SGs, which represent the environment at multiple levels of abstraction and enable higher-level semantic reasoning over visual scenes.

The pipeline (Figure 5) of ConceptGraphs integrates multiple vision and VLM components to construct open-vocabulary 3D SGs from RGB-D observations. First, generic instance segmentation is applied to each image to extract object regions. For each segmented region, semantic features (e.g., CLIP embeddings) and optional textual descriptions are computed to support open-vocabulary matching. Using camera poses and depth information, these 2D object regions and their features are projected into 3D and incrementally fused across views to form persistent object nodes. Optionally, LLMs can be invoked to generate object captions and infer semantic relationships between objects, which are then encoded as edges in the SG.

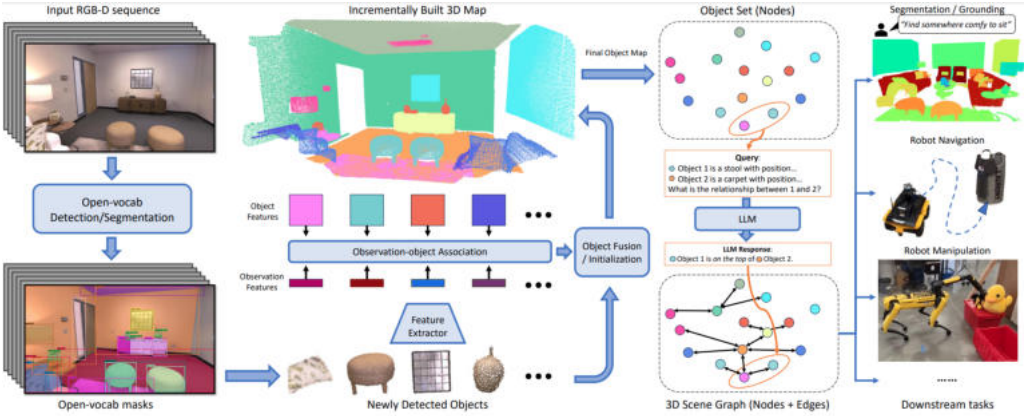


Figure 5: Pipeline of ConceptGraphs by Gu et al. (2024)

Most components of ConceptGraphs can be deployed locally using relatively lightweight models, making the framework suitable for edge computing scenarios. However, for high-level scene understanding and relationship extraction, the authors rely on an API call to OpenAIs GPT-4V. This introduces a dependency on an external service and a stable internet connection, which may limit the applicability of the framework in real-time or resource-constrained environments. Additionally, invoking a large-scale API model for each image or frame can incur significant token costs, particularly in scenarios involving large datasets or continuous visual streams.

Another limitation of ConceptGraphs is that it is not designed to handle real-time input from cameras or sensors. The framework operates on pre-recorded images or videos, which restricts its usability in dynamic settings such as AS that require immediate perception and adaptation. Consequently, it is less suitable for environments where humans or other dynamic agents are present and where rapid hazard detection is critical.

The optional use of an external API is motivated by the desire to offload computationally intensive tasks, such as relationship extraction and holistic scene understanding, to a powerful model capable of providing richer contextual reasoning than locally deployed VLMs. Nevertheless, this design choice reflects the primary focus of ConceptGraphs, which is not direct SG generation for downstream control or safety applications, but rather enhanced visual reasoning for tasks such as image retrieval and visual question answering. For example, the framework can guide a AS toward a target object by querying the SG or assess path traversability by reasoning about obstacles in the environment.

A key limitation of ConceptGraphs is that it does not support fine-grained or domain-specific object detection, such as identifying labeled or semantically critical objects (e.g., a gas can where the label conveys essential information). Instead, it primarily detects generic object categories (e.g., "can" or "bottle"), which limits its applicability in safety-critical or industrial scenarios.

Overall, while ConceptGraphs represents a significant step toward combining VLMs with structured scene representations, its reliance on external APIs, lack of real-time processing, and focus on generic object understanding constrain its suitability for dynamic, safety-critical environments.

2.8.2 OpenGraph

Another approach to integrating semantic understanding with structured scene representations is OpenGraph (Deng et al., 2024), which introduces a hierarchical open-vocabulary 3D graph representation specifically designed for large-scale outdoor environments. Unlike ConceptGraphs, OpenGraph constructs its hierarchical graph from incoming LiDAR and image sequences, producing a multi-layered representation (Figure 6) that includes a point cloud layer, lane graph layer, instance layer, segment layer, and environment layer. The hierarchical structure reflects its outdoor focus: layer 2 represents lane graphs rather than objects, layer 3 captures instances instead of spatial areas, and layer 4 encodes roads rather than rooms as seen in indoor-focused frameworks like Hydra (Hughes, Chang and Carlone, 2022). This layer design demonstrates that OpenGraph is fundamentally tailored for outdoor driving scenarios rather than indoor environments.

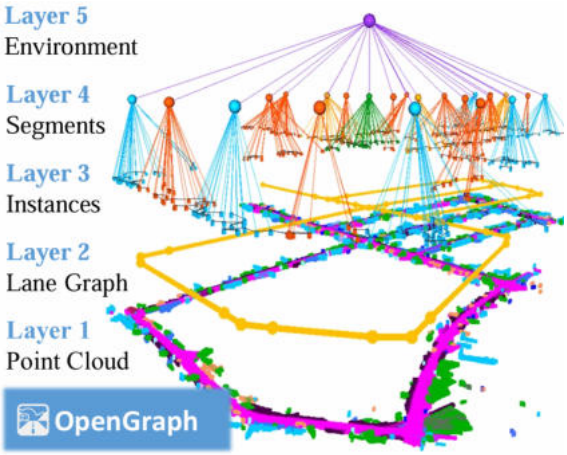


Figure 6: OpenGraph hierarchical representation (Deng et al., 2024)

A key distinction of OpenGraph is its independence from external APIs during inference. While the framework leverages pre-trained VLMs and LLMs to encode object captions and semantic features offline, it does not rely on external services such as GPT-4 at runtime. The caption features are encoded using pre-trained models trained on large open-vocabulary datasets, which enables semantic reasoning without the dependency on, network connectivity, or associated token costs that characterize ConceptGraphs.

OpenGraph is evaluated primarily on the SemanticKITTI dataset, a large-scale outdoor LiDAR dataset captured from vehicles in driving scenarios. This focus on outdoor environments represents both a strength and a limitation. While the framework excels at representing and reasoning about large-scale outdoor scenes, it has not been validated for indoor environments such as offices, warehouses, or manufacturing facilities where safety-critical applications may be deployed.

Another significant limitation of OpenGraph is that it operates primarily as an offline experimental system. The hierarchical graph is generated after processing the complete sensor data, rather than being constructed in real-time during operation. While the framework can incrementally build the graph from sensor streams, it is demonstrated in an offline setup using pre-recorded datasets rather than as a real-time AS system. This constrains its applicability for scenarios requiring immediate hazard detection and rapid response, such as dynamic HRI in industrial settings.

The main goal of OpenGraph is to provide a hierarchical representation that integrates semantic understanding from VLMs and LLMs, supports zero-shot object retrieval and structured queries, enables topological reasoning and global path planning, and improves semantic segmentation performance compared to previous methods. These capabilities make it well-suited for autonomous driving and outdoor navigation tasks, but its offline nature and outdoor focus limit its direct applicability to real-time indoor safety monitoring.

While ConceptGraphs and OpenGraph represent significant advances in integrating VLMs with

structured scene representations, both frameworks exhibit fundamental architectural and technical limitations that constrain their applicability to real-time, safety-critical applications. ConceptGraphs dependency on external API calls introduces latency and reliability concerns incompatible with time-sensitive hazard detection, while its batch-processing architecture prevents streaming sensor integration. OpenGraph's post-hoc graph construction and outdoor-focused design fundamentally limit its utility for immediate decision-making in dynamic indoor environments. These limitations are not merely deployment constraints but rather reflect core architectural decisions that prioritize offline analysis and domain-specific optimization over real-time adaptability and safety-critical responsiveness. In Section 2.9, a detailed comparative analysis of these frameworks against the requirements for real-time, edge-based safety applications is presented, demonstrating how the proposed approach addresses and mitigates these fundamental limitations.

2.9 Comparison with Existing Approaches

The preceding sections examined ConceptGraphs and OpenGraph, two notable frameworks that integrate VLMs with SG representations. Both approaches demonstrate significant advances in semantic understanding and structured scene representation, yet they exhibit distinct limitations that motivate the design of this approach. This section compares the presented method with these existing works across several critical dimensions relevant to real-time, edge-based deployment scenarios.

2.9.1 Real-Time Processing

A fundamental distinction of this work is its native support for real-time processing from live sensor streams. As discussed in the preceding section, both ConceptGraphs and OpenGraph are centered on pre-recorded or offline-processed data rather than live runtime perception. In contrast, the presented approach is designed to construct and update SGs in real-time during sensor acquisition, enabling immediate hazard detection, responsive adaptation, and concurrent scene analysis as environmental states evolve. This real-time availability of scene information also creates the basis for future runtime extensions in semantic reasoning and scene comprehension.

2.9.2 API Independence and Asynchronous Input Handling

A significant practical advantage of the presented system is API independence during runtime. ConceptGraphs relies on external GPT-4V calls for high-level scene understanding, whereas OpenGraph avoids runtime API calls by pre-encoding features offline. The presented framework executes locally and exposes standardized asynchronous REST endpoints (e.g., FastAPI) for input handling and scene-graph retrieval. This decoupled interface enables integration into diverse application contexts without direct platform coupling or specialized AS knowledge. It also supports deployment in resource-constrained or air-gapped environments and avoids token-cost and connectivity constraints associated with external API dependencies. ConceptGraphs, while a valuable research contribution, was not designed with accessibility or runtime data retrieval in mind. It lacks publicly accessible endpoints or standardized documentation for integrating SG queries and results into external applications during runtime.

2.9.3 Model Interchangeability and Extensibility

The proposed design prioritizes model interchangeability and extensibility. Instead of tightly coupling specific VLMs or relationship extractors, the architecture allows substitution of pre-trained models according to task and hardware constraints. ConceptGraphs and OpenGraph include multiple components, but their model choices are more tightly embedded and less exposed for dynamic runtime switching.

A key distinction is that model selection is exposed through documented API parameters, enabling model replacement without architectural changes. This design choice becomes increasingly important as VLM capabilities continue to advance. Frameworks that hardcode dependencies to specific models risk rapid obsolescence as newer and more capable alternatives emerge. By providing an open interface for model selection, the presented approach supports continuous adaptation to the evolving VLM ecosystem and enables comparative analysis of different models within the same scene-understanding pipeline.

The same modular design also simplifies extension. A key distinguishing characteristic is the framework's fundamentally linear architecture. New components, such as custom detectors, RAG modules, or domain-specific reasoning stages, can be inserted or removed through interfaces without disrupting the overall pipeline. Each module operates independently with explicit input and output interfaces, making the data flow transparent and easier to inspect throughout the processing chain. This architectural clarity supports debugging, understanding, and further extension. In contrast, existing frameworks prioritize computational efficiency and semantic coherence through tighter component coupling, which may suit their target domains but also makes adaptation and extension more difficult.

2.9.4 Ease of Deployment and Integration

A key design principle is usability without full platform coupling. The proposed framework can run on a server or edge device with an appropriate VLM and expose the same API interface in both cases. This allows AS or lightweight edge clients (e.g., Raspberry Pi class devices) to stream images to a server on the same network without framework-level modifications. In addition, the use of simple and widely supported software libraries supports operating-system-independent deployment across common development environments. In contrast, ConceptGraphs and OpenGraph are primarily research frameworks that do not provide documentation for integration into external applications.

2.9.5 Summary of Comparative Advantages

Table 1 summarizes the key distinctions across these dimensions. This approach addresses the identified limitations by combining real-time processing, API-independent operation, flexible model selection, modular architecture, and accessible deployment mechanisms. While ConceptGraphs and OpenGraph excel in their respective domains (semantic reasoning for retrieval tasks and large-scale outdoor representation), they are not optimized for the specific requirements of real-time, edge-based, safety-critical applications where immediate responsiveness and implementation flexibility are paramount.

Feature	**ConceptGraphs**	**OpenGraph**	**Proposed Framework**
Real-time Processing	No	No	Yes
External API Required	Yes (GPT-4V)	No	No
Model Interchangeability	Limited	Limited	Yes
Asynchronous API Endpoint	No	No	Yes
Functionality Extension	No	No	Yes
Indoor Focus	Yes	No	Yes
Safety Applications	No	No	Yes

Table 1: Comparative analysis of framework characteristics across ConceptGraphs, OpenGraph, and the proposed approach

2.10 Cloud and Edge computing

The increasing demand for computing power in deployed systems has led to different computing strategies, with cloud computing and edge computing as two dominant paradigms. As shown in figure 7, both rely on sensors or actuators, a network, and computing units. The main difference is where computation is executed. In cloud computing, processing occurs in centralized data centers, while edge computing places computation closer to the data source, for example on the device itself or in a local gateway.

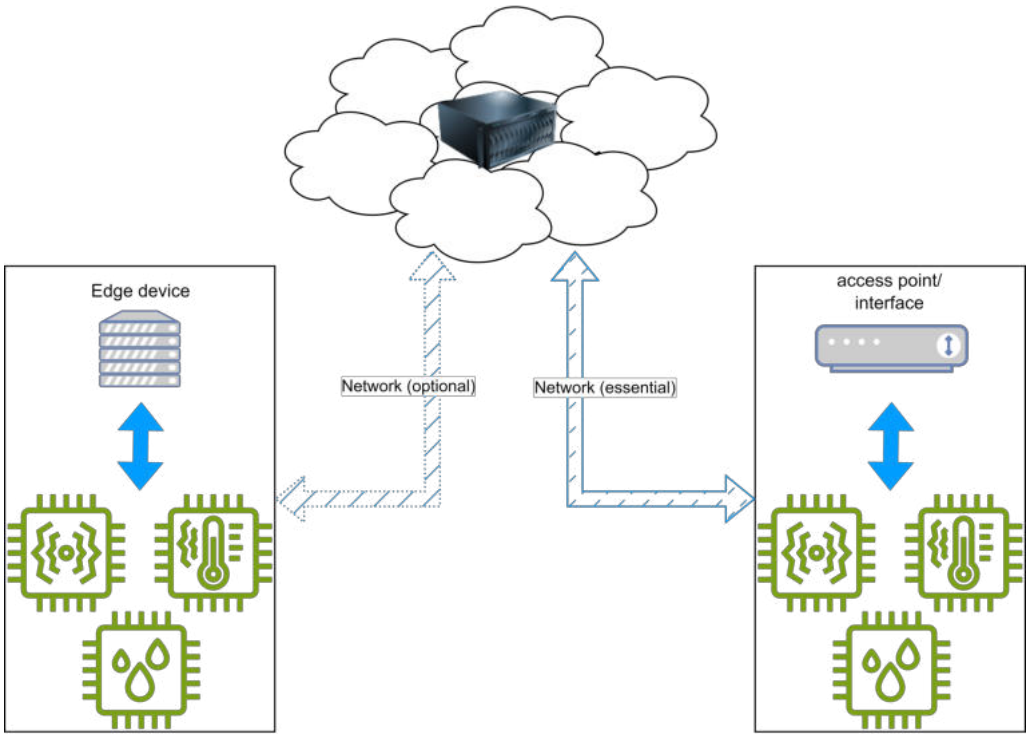


Figure 7: Comparison of cloud and edge computing architectures

The rise of Internet of Things (IoT) devices and applications has further intensified the need for efficient computing paradigms, as billions of connected sensors and actuators generate massive volumes of data that require processing (Jianli Pan and McElhannon, 2018).

2.10.1 Cloud computing

Cloud computing refers to the delivery of computing services over the internet, where data processing and storage occur in centralized, remote data centers (Figure 7, right). These data centers typically feature high-performance servers with substantial computational resources, enabling them to handle complex tasks. Their centralized architecture also enables elastic scaling through dynamic resource allocation (Satyanarayanan, 2017).

Cloud computing improves cost efficiency by pooling shared compute and storage resources in large data centers. This enables access to parallel processing and large-scale storage that would be prohibitively expensive for many individual organizations. The pay-per-use model converts capital expenditure into operational expenditure, and cloud platforms provide orchestration, automated scaling, and global service distribution (Jianli Pan and McElhannon, 2018; Satyanarayanan, 2017).

However, cloud computing also faces several challenges. The most significant is latency, as data must travel between edge devices and remote data centers, which can introduce round-trip delays ranging from tens to hundreds of milliseconds depending on geographic distance and network conditions. This Wide-Area Network (WAN) latency is constrained by physical propagation limits and

practical network conditions, so it can be reduced but not eliminated through provisioning alone. Such delays are problematic for time-critical applications such as AS or industrial control processes. Furthermore, cloud computing requires continuous internet connectivity, making systems vulnerable to network outages and bandwidth constraints. The need to transmit large volumes of raw sensor data to the cloud also creates substantial bandwidth costs and may overwhelm network infrastructure, particularly in scenarios with numerous IoT devices generating continuous data streams (Jianli Pan and McElhannon, 2018; Satyanarayanan, 2017; Shi and Dustdar, 2016). Privacy and security concerns also arise, as sensitive data must be transmitted over networks and stored on third-party infrastructure, raising issues of data sovereignty, compliance with regulations, and potential exposure to cyber attacks (Satyanarayanan, 2017; Shi and Dustdar, 2016).

2.10.2 Edge computing

Edge computing addresses cloud limitations by performing data processing closer to the data source. In this method, computational tasks are distributed to edge devices or local gateways rather than centralized in remote data centers (Figure 7, left without network). This placement reduces latency by avoiding long network paths for processing Shi and Dustdar (2016). Related concepts include fog computing, which extends cloud resources toward the network edge through hierarchical distribution (Yi, C. Li and Q. Li, 2015).

Beyond lower latency, edge computing provides operational benefits that are important in practice. Local processing reduces bandwidth usage by sending processed outputs instead of continuous raw streams (Jianli Pan and McElhannon, 2018). For instance, a surveillance camera can run local object detection and transmit alerts and metadata rather than full video. Edge computing can also improve privacy by keeping sensitive data local before transmission and can maintain partial autonomy during cloud outages, improving resilience (Satyanarayanan, 2017; Shi and Dustdar, 2016; Yi, C. Li and Q. Li, 2015).

Recognizing the growing demand for machine learning at the edge, manufacturers such as NVIDIA have developed specialized devices that support computationally intensive inference with constrained power budgets. The NVIDIA Jetson platform is one example, offering hardware for deploying deep learning and computer-vision workloads on edge systems (Jetson AI Lab, 2025). Such devices bridge the gap between highly constrained IoT nodes and cloud-scale infrastructure.

Despite these advantages, edge computing has inherent limitations. Edge devices typically have constrained computational resources, memory, and power supply compared to cloud data centers (Jianli Pan and McElhannon, 2018; Yi, C. Li and Q. Li, 2015). While some edge devices such as NVIDIA Jetson platforms may consume 30-50 watts of power (Jetson AI Lab, 2025), this is still significantly more limited than cloud servers. The distributed nature of edge infrastructure introduces significant management complexity, including challenges in software deployment, version control, security patch distribution, and monitoring across potentially thousands of heterogeneous devices (Jianli Pan and McElhannon, 2018). Additional challenges include ensuring interoperability between diverse edge devices and platforms, and addressing mobility management as devices move across network boundaries (Yi, C. Li and Q. Li, 2015). Edge devices may also lack the redundancy and reliability features of cloud data centers. The fundamental trade-off between computational capability and proximity to data sources remains a central challenge in edge computing architectures (Satyanarayanan, 2017; Shi and Dustdar, 2016).

2.10.3 Hybrid approaches

In practice, many modern systems adopt hybrid architectures that combine cloud and edge computing to leverage both paradigms (Jianli Pan and McElhannon, 2018; Yi, C. Li and Q. Li, 2015) (Figure 7, left with network). The central design question is no longer whether to use cloud or edge, but how to orchestrate workload placement across tiers according to latency, privacy, bandwidth, and compute constraints. A typical hierarchy includes device-level edge processing, local edge servers or gateways (fog layer), and cloud resources for heavy computation (Yi, C. Li and Q. Li, 2015). The edge-cloud architecture extends traditional cloud capabilities toward end devices (Jianli Pan

and McElhannon, 2018). Dynamic placement mechanisms can adapt to changing network conditions and application priorities, improving resource utilization and overall system performance (Satyanarayanan, 2017; Shi and Dustdar, 2016).

2.11 Ethical Considerations & Safety Compliance

The deployment of AS and VLMs in work environments introduces ethical challenges that extend beyond technical performance. These systems shape trust, safety culture, and worker autonomy through design choices and implementation practices. Addressing these concerns requires systematic examination of bias, transparency, accountability, and the broader societal impacts of AI-mediated decision-making in HRI.

2.11.1 Ethical Implications

When teams deploy systems without meaningful worker consultation, trust can erode or a false sense of safety can emerge, especially if users overrely on imperfect perception systems. Even well-intentioned systems can reinforce inequalities or unsafe practices if deployed without regard to local norms. Bias arising from training data and representational gaps remains central, as underrepresented populations or environments raise the risk of misinterpreting objects, gestures, or safety cues, which in turn leads to exclusion, misclassification, or unsafe automation. These harms intensify when datasets and development teams are geographically and culturally concentrated. Models trained predominantly in the United States, for instance, often embed standards and social norms specific to that context, which can overrepresent perspectives from developed countries while underrepresenting the thinking, practices, and conditions prevalent in developing nations (Hagerty and Rubinov, 2019).

Beyond bias, deployment affects worker autonomy and job design. Automation can free workers for higher-level tasks, yet may also reduce agency if roles become surveillance-heavy monitoring positions. Overreliance on automated perception can weaken human skills and blur moral responsibility. These concerns underscore the need for systematic examination of training data and model behavior, clear assignment of responsibility when AI-generated outcomes cause harm, and continuous assessment of effects on worker skills and identity (Bankins and Formosa, 2023; Osasona et al., 2024).

2.11.2 Responsible AI Deployment

Responsible deployment of AS and VLMs requires coordinated action across three dimensions: governance, design, and practice. Each dimension addresses distinct but complementary aspects of ethical AI integration.

Governance establishes the organizational and policy framework for responsible deployment. Organizations must clearly define who bears responsibility when AI systems cause harm, establish mechanisms for human override of automated decisions, and ensure shared accountability between system developers and operators. This governance structure ensures that ethical considerations remain enforceable rather than aspirational (Osasona et al., 2024).

Design focuses on building ethical considerations directly into system architecture and functionality. Systems must preserve human agency by avoiding full automation of critical decisions, provide explainable behaviors that allow operators to understand and verify system reasoning, and incorporate monitoring capabilities to track how HRI affects worker skills and safety culture over time (Bankins and Formosa, 2023).

Practice includes the ongoing activities required to maintain ethical alignment throughout the system lifecycle. Three interlocking activities support this: First, contextual ethnographic research uncovers local workplace risks and power dynamics that generic approaches might miss. Second, inclusive data practices actively expand representation across diverse populations and

environments in both training and validation datasets. Third, iterative participatory evaluation brings together workers, local stakeholders, and multidisciplinary experts to continuously assess and mitigate inequality and harm (Hagerty and Rubinov, 2019).

2.11.3 Explainability & Transparency

AI systems that operate as black boxes challenge transparency and explainability (Adadi and Berrada, 2018; Arrieta et al., 2020). Limited insight into model logic complicates verification and accountability in safety-critical settings (Arrieta et al., 2020). Explainable Artificial Intelligence (XAI) addresses these challenges by providing human-understandable insights into model behavior and supporting responsible deployment of AS systems (Adadi and Berrada, 2018; Arrieta et al., 2020). For VLMs, XAI methods include heatmaps that highlight salient image regions (for example in figure 8), graph-based visualizations that expose reasoning paths, and comparisons of graphs against raw outputs to assess interpretability (F. Mumuni and A. Mumuni, 2025; Selvaraju et al., 2017). Such techniques enable operators to audit model focus, verify safety-critical object recognition, and communicate system logic to workers and stakeholders.

Ante-hoc (explanation before generation) methods are designed with transparency as a core property, allowing users to directly trace how inputs map to outputs. However, these models often lack the representational power required for complex multi-modal tasks. Post-hoc methods, by contrast, provide explanations after training without altering the underlying model, making them widely used for deep neural networks and VLMs (Adadi and Berrada, 2018; Arrieta et al., 2020; F. Mumuni and A. Mumuni, 2025). This distinction reflects a broader trade-off between predictive accuracy and interpretability. Highly expressive architectures such as deep neural networks, transformer-based LLMs, and VLMs achieve superior performance by learning non-linear interactions across many latent variables, but they obscure the causal or logical structure underlying predictions. Interpretable models, including linear models, rule-based systems, or decision trees, provide clearer decision logic but typically cannot model complex multi-modal data distributions as effectively. XAI methods mitigate rather than eliminate this trade-off by approximating high-performance black-box behavior in a form that is cognitively accessible to users, with some loss of explanatory precision (Adadi and Berrada, 2018; Arrieta et al., 2020; F. Mumuni and A. Mumuni, 2025).

The appropriate interpretability level depends on application context and risk tolerance. High-stakes AS may justify reduced performance for stronger transparency and accountability, whereas performance-critical tasks may prioritize post-hoc explanations that preserve accuracy while offering insight. Additionally, VLMs and LLMs can serve as explanatory agents, generating natural-language rationales that enhance interpretability for non-technical operators and support their decision-making (F. Mumuni and A. Mumuni, 2025).

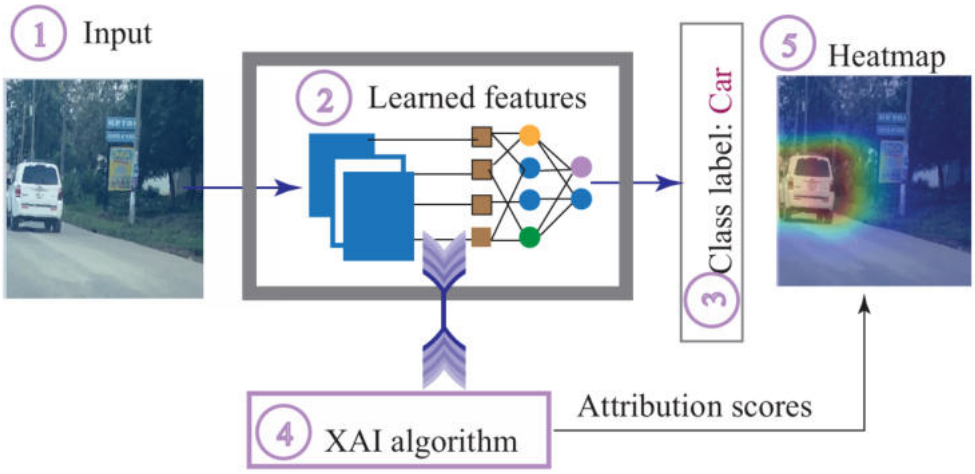


Figure 8: XAI for image recognition (F. Mumuni and A. Mumuni, 2025; Selvaraju et al., 2017)

3 Methodology

This section presents the methodological foundation of the proposed framework, from architectural and implementation decisions to the complete scene-understanding pipeline. It first motivates the modular, class-based system design, the communication layer based on FastAPI, and deployment on an edge device (NVIDIA Jetson Orin), followed by an evaluation of VLM input-output formats to select a suitable model configuration. Building on this, it details the framework workflow including preprocessing of image and IMU data, frustum-based scene-graph cutout generation, SG representation constraints, and bidirectional coordinate conversion between camera and world frames. Finally, it describes the graph combination strategy for robust temporal updates under detection uncertainty and discusses performance optimization and scalability trade-offs for practical HRI deployment.

3.1 Overall System Architecture

The overall system architecture is designed to be modular and extensible, therefore for the codebase a class-based approach is the most suitable option. This architectural decision provides several key advantages that align with the requirements of the framework:

- **Encapsulation and Maintainability:** Classes enable the grouping of data and the functions that operate on that data, establishing clearer boundaries within the codebase and promoting maintainable code. The name of each class explicitly indicates its intended purpose and functionality, making it straightforward to locate specific implementations, distinguish between different components, and add new features systematically. For example, if modifications to the SG handling are required, a developer would naturally navigate to the SG class and locate the relevant function without needing to understand the entire codebase.
- **Scalability through Instantiation:** As the computational power of modern GPUs continues to increase, the framework can scale efficiently without requiring extensive modifications to the codebase. Additional processing capacity can be leveraged simply by instantiating more objects of the specific classes. For example, multiple inference engines can be instantiated to handle larger workloads.
- **Extensibility via Inheritance:** The support for inheritance and abstract classes enables flexible extension of the framework. When different approaches for modifying object content are required, abstract classes can serve as templates that define the necessary interfaces and usage patterns for other modules. For instance, when implementing the conversion of prompt components to different formats, all object interfaces remain consistent while only the internal implementation of the content modification differs.
- **Modular Testing and Integration:** The class-based structure facilitates the independent testing of individual modules. For instance, components responsible for data acquisition or graph management can be tested in isolation before being integrated into the complete workflow. This modularity not only improves code reliability but also simplifies the addition of future classes and features. This approach has already proven useful during development, as it allowed testing of individual components such as the inference engine or the retrieval of additional information without running the entire system, thus speeding up development and debugging.
- **Accessibility and Developer Familiarity:** The class-based paradigm is one of the most widely understood concepts in software development, lowering the barrier for contribution and customization. Due to the nature of research, future contributors may be from different fields or have varying levels of coding experience, making it crucial to choose an architecture that is widely understood and accessible.

3.2 Communication Protocols

The communication protocol chosen for the proposed framework is based on RESTful APIs implemented using FastAPI. This choice is motivated by several key advantages that align with the requirements of run-time, edge-based scene understanding applications. During development, ROS (Robot Operating System) was considered but ultimately rejected as the communication protocol for this framework, for the reasons discussed below.

3.2.1 Comparison with ROS

While ROS excels at providing real-time and inter-process communication (its primary design goal), it imposes a steep learning curve for practitioners unfamiliar with robotics-specific paradigms. In contrast, REST APIs are universally understood and widely adopted across software development communities. Importantly, in many scene understanding applications, real-time processing of every sensor frame is unnecessary. Asynchronous processing of selected frames allows flexible resource allocation, and the API-based approach supports this by enabling developers to query scene understanding results on-demand, without requiring deep integration into complex frameworks.

ROS, furthermore, becomes complex to understand when not used correctly. This was observed on several occasions when attempting to understand and debug the Hydra, Khronos, and Kimera libraries. When running the system on a single machine, multiple nodes with different topics were deployed without adequate documentation of their actual functionality, necessitating examination of each topic and message to understand the data flow within the framework. This complexity is further amplified when deploying the system across multiple machines, where network configuration and topic remapping become necessary. In contrast, FastAPI provides a clear and well-documented interface for interaction, simplifying deployment across different environments without the need for complex network setups.

The current deprecation of ROS1 and the transition to ROS2 introduces potential compatibility issues, as many existing libraries are still based on ROS1 and migration can be non-trivial. This stems from the fact that ROS2 uses DDS (Data Distribution Service) as its underlying communication middleware, which differs from the custom TCP-based protocol used in ROS1, affecting how nodes discover each other and communicate. Combining both versions on a single system is not feasible without additional bridging tools, because ROS1 can only run on machines with Ubuntu versions up to 20.04 (Quigley et al., 2009), while ROS2 requires at least Ubuntu 22.04 (Macenski et al., 2022). In contrast, FastAPI relies on standard web protocols (HTTP/HTTPS), ensuring broad compatibility across different systems and programming languages without being tied to a specific middleware.

In conclusion, while ROS is powerful for AS systems requiring tight integration and real-time guarantees, its steep learning curve, complex multi-framework integration, platform constraints, and version transition issues make it poorly suited for a framework designed to support diverse edge devices and flexible on-demand scene understanding. FastAPI's simplicity, platform independence, and standardized HTTP interface directly address these requirements, making it the decisive choice for this application.

3.2.2 Limitations and Trade-offs

FastAPI's primary limitation is the lack of hard real-time guarantees. Request queuing and HTTP round-trip latencies introduce overhead that violates strict real-time deadlines typically required by control systems. For a AS performing real-time obstacle avoidance or emergency stops, this would be unacceptable.

However, this limitation has minimal impact on the proposed framework. Scene understanding serves as a semantic layer, not a run-time control layer, and the framework is designed to query scene understanding on-demand, for example when a AS encounters an ambiguous situation or for periodic high-level decision making, rather than processing every sensor frame. Response times on

the order of milliseconds are acceptable for these use cases. Additionally, the framework's architecture separates scene understanding (handled by FastAPI) from run-time control (handled by separate dedicated systems), eliminating the need for hard real-time guarantees in the communication layer.

3.2.3 Endpoint Design and Functionality

The endpoint architecture employs an asynchronous task-based model specifically chosen to decouple variable processing times from unpredictable client request patterns. Rather than blocking clients until scene understanding completes, clients submit requests and receive a task identifier immediately. This approach resolves key architectural challenges: variable processing times can extend to several seconds for complex scenes with multiple objects, making synchronous blocking unacceptable, and clients may disconnect or time out, requiring non-blocking responses.

The FIFO queueing model was chosen because each iteration must process the next image in strict sequence. The output of iteration n becomes the input context for iteration $n + 1$, so reordering tasks would break the dependency chain. Approaches such as priority queues or load-balancing were considered, but both would allow out-of-order execution and therefore violate the sequential processing requirement.

Task timestamps serve a critical function: they enable clients to determine whether cached scene understanding results remain valid. For example, if a AS queries the framework at time t_1 and receives results with timestamp t_0, it can later determine at time t_2 whether new results are available by comparing the returned timestamp. This design reduces redundant processing when the scene remains static.

Input parameters are intentionally minimized to maximize client accessibility. Base64-encoded images and simple numerical parameters ensure compatibility across programming languages without requiring dependency installation. Conversely, outputs are restricted to JSON, providing a single, consistent semantic representation for all downstream consumers.

3.3 Edge device

To facilitate run-time processing and inference, our framework is designed to operate on edge devices equipped with sufficient computational power. The choice of edge devices allows for reduced latency, enhanced privacy, and decreased dependency on cloud services. For this framework, we utilize a NVIDIA Jetson Orin, which was provided by the institute for this research. While alternative edge computing platforms exist, such as Intel Neural Compute Stick, Google Coral, or other Jetson variants (Nano, Xavier), the Jetson Orin represents an appropriate choice for this application. The platform is specifically designed for edge-based AI inference tasks, making it well suited for deploying VLMs locally. Its hardware specifications, including an integrated GPU and 64GB of shared memory, meet the substantial computational requirements of modern VLMs, which typically require significant memory for model weights and intermediate computations. Additionally, its balance of processing power, energy efficiency, and physical form factor aligns with the requirements for a deployable AS that must operate in real-world industrial environments.

3.3.1 NVIDIA Jetson Orin

The NVIDIA Jetson Orin runs on a Ubuntu-based Linux distribution, specifically tailored for the NVIDIA architecture, which provides a familiar environment for developers accustomed to working with Linux systems (Jetson AI Lab, 2025). It can easily be set up and integrated with various sensors and peripherals over LAN, wireless connections, or GPIO pins.

Some restrictions must be considered when deploying software on the Jetson Orin, as not all software packages and libraries available for standard Ubuntu distributions are compatible with the Jetson's ARM architecture. In some testing phases, certain Python libraries and frameworks

had to be built from source or modified to ensure compatibility. These libraries were mainly hosted by NVIDIA on their official repositories. On one side, this ensures that the libraries are optimized directly for the Jetson hardware, but on the other side, it can lead to limitations in terms of availability of the latest versions. This exact problem occurred during one of the testing phases, where a specific version of flashattention was required for the VLM models to run faster and more efficiently, but the required version was not available as the NVIDIA repositories were not reachable at that time.

Another problem that occurred during the testing phase was the practical usage of a virtual environment for Python. While virtual environments are a common practice in Python development to manage dependencies and avoid conflicts between different projects, it was found that certain libraries did not function properly when installed within a virtual environment on the Jetson Orin. For example, PyTorch and torchvision had to be built from source inside the virtual environment to ensure proper functionality, adding an additional layer of complexity to the development process.

Another option to deploy a VLM on the NVIDIA Jetson Orin would be to use NVIDIA's own container deployment (Jetson AI Lab, 2025). These containers are pre-configured Docker images that come with all the necessary dependencies and optimizations for the selected VLM and corresponding hardware. This approach simplifies the deployment process significantly, as it eliminates the need for manual installation and configuration of libraries and frameworks. However, using containers also comes with its own set of challenges (Tutorialpedia, 2026). Beyond the increased resource consumption that containers can introduce, which may impact performance on resource-constrained edge devices, the container images themselves are not always up-to-date and well maintained. For instance, certain flags provided by NVIDIA for the container build process were removing the ability to use images as API inputs, which was required for our framework to function properly. Furthermore, during the development period, NVIDIA changed their documentation and container build process, modifying the Docker commands required to build images with correct flags and removing certain models from their official container repository.

It must also be considered that using a desktop environment on the Jetson Orin can lead to increased resource usage, as graphical interfaces consume additional CPU and memory resources. In scenarios where maximum performance is required, it may be beneficial to operate the Jetson Orin in headless mode to free up resources for computational tasks. During testing, the desktop environment consumed around 16GB of the total 64GB of shared memory, which could have been used for processing tasks instead.

Overall, while the NVIDIA Jetson Orin provides a powerful platform for edge computing and AI applications, developers must navigate challenges related to software compatibility, dependency management, and resource optimization to fully leverage its capabilities.

3.4 Evaluating VLM models

Each VLM has its own specific strengths and weaknesses when it comes to the input and output formats it supports or it is trained for. Therefore, it is curcial to test different VLMs and evaluate which one is the most suitable for the specific use case of this framework. For this evaulation several criteria were taken into account, to find a score for each VLM. These criteria include:

- **Input format compatibility:** The VLM should be able to accept the input data format and understand the structure of the data.
- **Output format compatibility:** The VLM should provide output in a format that is compatible with the downstream tasks in the framework.
- **Repetition and hallucination:** The VLM should minimize repetition and hallucination in its outputs to ensure reliability, accuracy and usability of the results.
- **Location format:** The VLM should provide location information in the requested format, such as pixel coordinates or relative positions.

This evaluation includes three different VLMs: Llava1.6 Mistral 7B (L. Team, 2025), Qwen 2.5 VL 7B (Q. Team, 2024) and Phi 4 Vision Multimodal (Microsoft, 2025). These models were selected because NVIDIA provides optimized Docker containers with native integration to the Jetson Orin device, ensuring best performance and compatibility with the hardware specified in Section 3.3.1. All three are recent, open-source models suitable for edge deployment. Each model was tested with the same input data and outputs were compared against the mentioned criteria to identify the best performing model for this framework. It should be noted that for reproducibility purposes, users attempting to recreate this evaluation need to remove the `--chat-template-content-format=openai` flag from the docker-compose template provided by NVIDIA (Jetson AI Lab, 2025). This flag is designed only for chat completion tasks following the OpenAI format, which is not natively compatible with the specific VLM containers used in this evaluation, despite being included in the official VLM templates.

3.4.1 Input data and prompts

The input data for the evaluation consisted of images from the TUM dataset (Sturm et al., 2012). More precisely, images from the sequence "freiburg3_long_office_household" were used, as these images contain a variety of objects and scenes that are relevant for testing the VLMs capabilities. The dataset contains 2585 images with corresponding ground truth data of the trajectory and positions of the camera. For the evaluation, a subset of 88 images was selected, which were evenly distributed throughout the sequence to cover different angles and perspectives of the environment. The images have a resolution of 640×480 pixels and were not resized or modified before being fed into the VLMs, to ensure realistic test conditions and preserve the possibility of precise detail recognition. An example image from the dataset is shown in Figure 9. All the mentioned prompts are listed in the Appendix A. The criteria of the tables are explained in the following:

- **Right file format:** This criterion evaluates whether the VLM was able to generate the output in the requested file format (JSON, YAML, CSV). The score is calculated as the ratio of correctly formatted outputs to the total number of outputs.
- **No repetition:** This criterion evaluates whether the VLM was able to avoid repetition in its outputs. The score is calculated as the ratio of outputs without repetition to the total number of correct outputs.
- **Right location format:** This criterion evaluates whether the VLM was able to provide location information in the requested format (pixel coordinates or relative positions). The score is calculated as the ratio of correctly formatted location outputs to the total number of correct outputs.
- **Overall score:** This criterion provides an overall score for the VLM based on the previous three criteria. It is calculated as the ratio of all outputs that meet all three criteria to the total number of outputs.

Figure 9: freiburg3_long_office_household dataset example image (Sturm et al., 2012)

3.4.2 Llava 1.6

The first model evaluated was the Llava 1.6 Mistral 7B. This model was shown in the NVIDIA Jetson Lab Tutorials (Jetson AI Lab, 2025). It especially showed the realtime capabilities on the Jetson Orin platform, which makes it a promising candidate for edge device deployment. The realtime model could not be used for the evaluation, as the specific container was not usable due to compatibility issues with specific libraries. The container ran into errors when trying to build it on the Jetson Orin device, after 150GB of containers were downloaded and builded for over 10 hours. Therefore, the non-realtime version of the model was used for the evaluation. This version is the same container type as the other two models evaluated, which ensures a fair comparison between the models.

The evaluation results for each configuration are shown in Figure 10 and Table 2. The model was tested with two different prompts and two different quantization levels (8-Bit and 16-Bit), to evaluate the impact of the prompt on the output quality. The first prompt was an instruction to list all objects and relations in the image, in the format of a JSON file with pixel coordinates. The second prompt was identical, but format was specified to be in relative coordinates instead of pixel coordinates.

The first prompt was tested with prescaled images to 512×512. During the two test runs, it was noticed that the model could not handle scaled images very well, as the positions of the objects were not corresponding to the actual positions in the image. Therefore, for all further tests, the original image resolution of 640×480 pixels were kept. With llava 1.6, it was also noticed that the second prompt led to significantly better results regarding the location format. This is most likely due to the fact that the model is trained to identify objects with relative positions rather than absolute pixel coordinates (L. Team, 2025). Also the quantization level had an impact on the results, with the 16-Bit quantization performing slightly better than the 8-Bit quantization in terms of location accuracy. The correct file format was not significantly affected by the prompt or quantization, as the model was able to generate the correct format for the majority of the images.

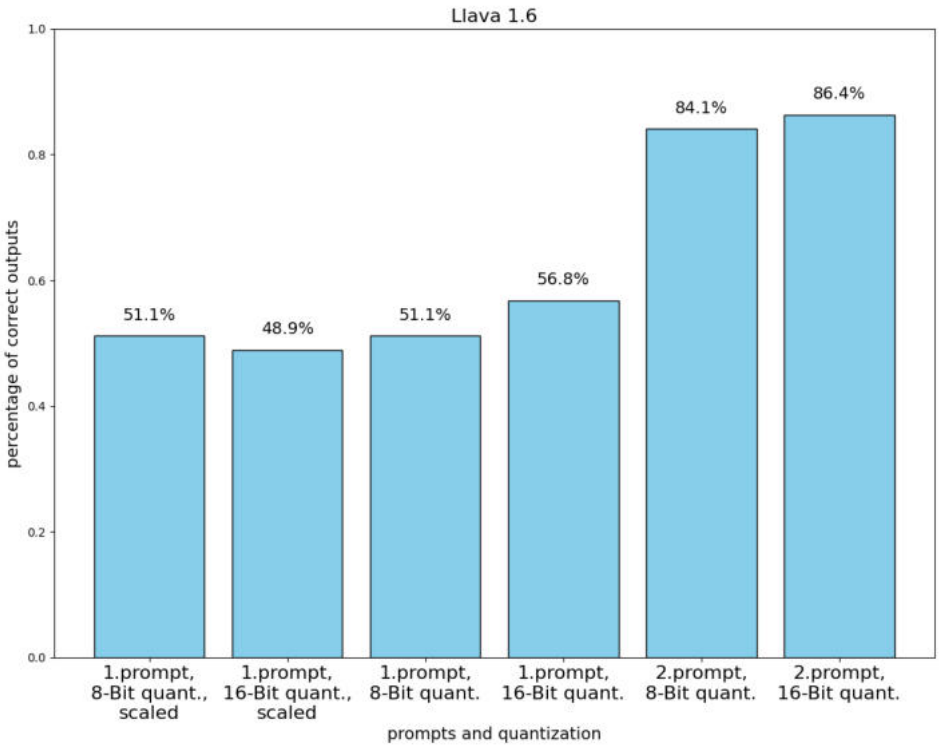


Figure 10: Llava 1.6 percentages for different prompt and quantization combinations

Prompt and quantization	right file format	no repetition	right location format	Overall score
1.prompt, 8-Bit quant., scaled	0.7841	1	0.6522	**0.5114**
1.prompt, 16-Bit quant., scaled	0.8864	1	0.5513	**0.4886**
1.prompt, 8-Bit quant.	0.875	1	0.5844	**0.5114**
1.prompt, 16-Bit quant.	0.8864	1	0.641	**0.5682**
2.prompt, 8-Bit quant.	0.9318	1	0.9024	**0.8409**
2.prompt, 16-Bit quant.	0.9091	1	0.95	**0.8636**

Table 2: Llava 1.6 evaluation results

3.4.3 Qwen 2.5

Qwen 2.5 VL 7B was the second model evaluated. The evaluation results for each configuration are shown in Figure 11 and Table 3. Similar to Llava 1.6, Qwen 2.5 was tested with the first and second prompt and two different quantization levels (4-Bit and 16-Bit). Additionally two additional prompts were tested with the 16-Bit quantization, to evaluate the impact of different prompt phrasings on the output quality. The third prompt had the same structure as the first two prompts, but the format was specified to be in YAML instead of JSON. The fourth prompt also prompted the VLM to provide the same structure, but the format of the output was a CSV format.

The first prompt has shown a better performance regarding to Llava 1.6, espacially in the terms of the location format accuracy. However, the second prompt did not lead to an improvement in the results, as it did with Llava 1.6. This indicates that Qwen 2.5 is more capable of handling absolute pixel coordinates compared to Llava 1.6. Although the quantization level influenced the results, the overall performance remained comparable to Llava 1.6. The third prompt with the YAML format led to the best results overall, indicating that Qwen 2.5 is more comfortable with this format compared to JSON or CSV. The fourth prompt with the CSV format led to the worst results, indicating that this format is not well suited for the use case of this framework. This is most likely due to the fact that two seperate CSV tables are needed to represent the objects and relations, which makes it more difficult for the model to generate the correct format. IT was noted that the model often forgot to fill in one of the two tables, leading to incomplete outputs.

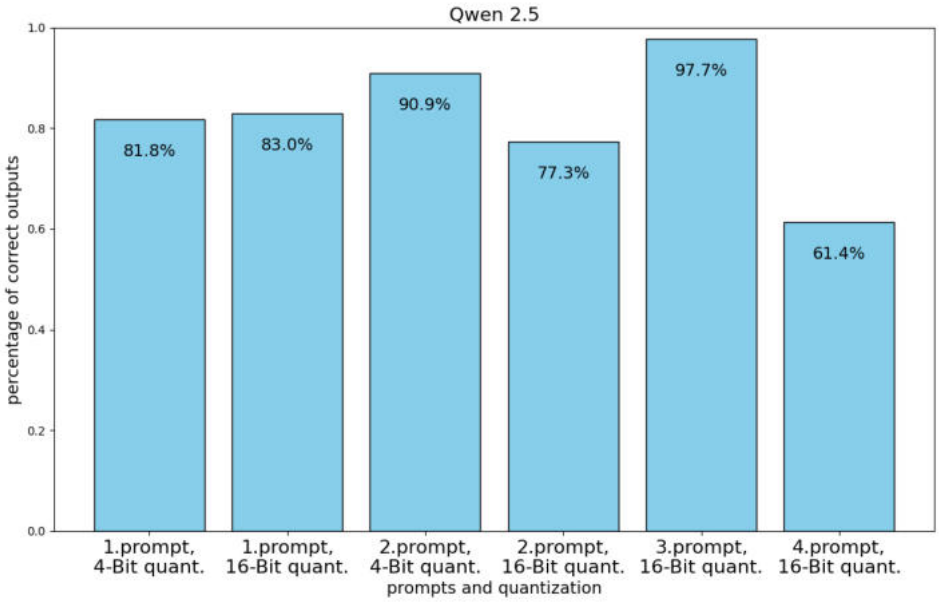


Figure 11: Qwen 2.5 percentages for different prompt and quantization combinations

Prompt and quantization	right file format	no repetition	right location format	Overall score
1.prompt, 4-Bit quant.	0.841	0.973	1	**0.8182**
1.prompt, 16-Bit quant.	0.83	1	1	**0.83**
2.prompt, 4-Bit quant.	0.92	1	0.988	**0.909**
2.prompt, 16-Bit quant.	0.784	1	0.986	**0.773**
3.prompt, 16-Bit quant.	1	0.9773	0.9773	**0.9773**
4.prompt, 16-Bit quant.	0.6364	1	0.9773	**0.6136**

Table 3: Qwen 2.5 evaluation results

3.4.4 Phi 4 Vision Multimodal

The last model evaluated was Phi 4 Vision Multimodal. The evaluation results for each configuration are shown in Figure 12 and Table 4. Similar to Llava 1.6, Qwen 2.5 the model should be tested with different quantization levels, but the container they provided only supported one type of quantization. Within the Docker environment the official Phi-4 Vision Multimodal container was used, which natively supports only 16-bit quantization, which corresponds to Microsofts standard deployment configuration of the model. Attempts from NVIDIA to alter the quantization level directly in the docke-compose configuration were ineffective because the underlying VLM does not provide support for alternative quantization modes. This raises concerns about whether the containers are systematically tested and well optimized before being published on the official website. Therefore only 16-bit quantization could be tested for this model.

Similar to Qwen 2.5, Phi 4 Vision Multimodal was tested with four different prompts. The second prompt led to the best results overall, and the first prompt also performed well, both indicating that the model handles JSON format most comfortably. The third prompt with YAML format led to worse results, indicating that the model is less comfortable with this format compared to JSON. The fourth prompt with CSV format led to the worst results, even worse to Qwen 2.5, indicating that this format is not well suited for the use case of this framework. It is notable that the repetition rate is significantly higher compared to Qwen 2.5, which also indiciates that CSV format doesnt prevent repetition by its nature, due to the fact that the tables do not have any end markers or brackets like JSON or YAML formats.

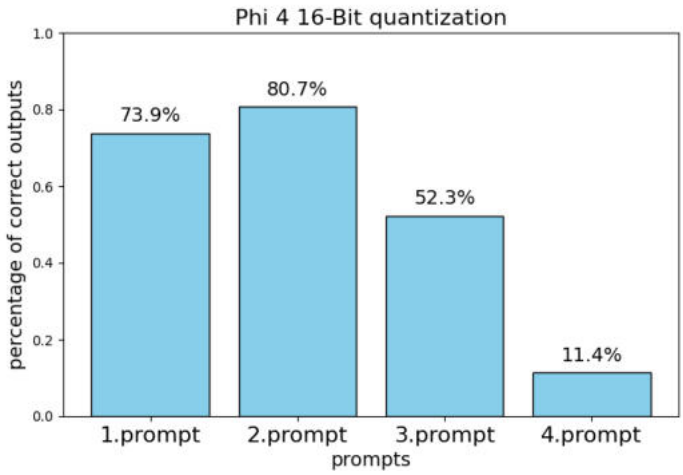
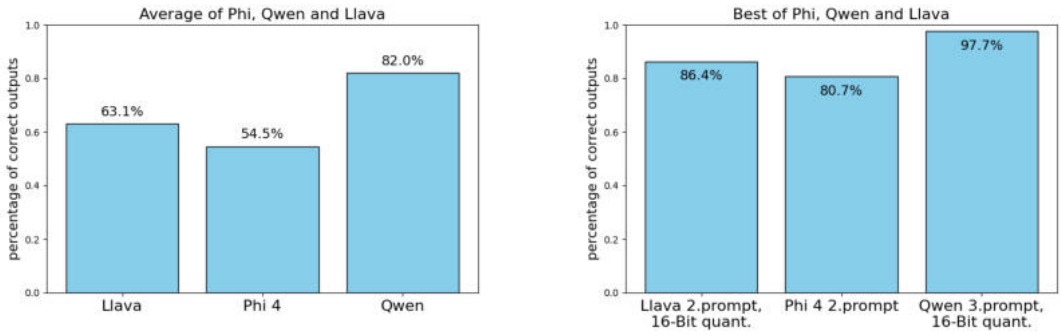


Figure 12: Phi 4 Vision Multimodal percentages for different prompt and quantization combinations

Prompt and quantization	right file format	no repetition	right location format	Overall score
1.prompt	0.75	1	0.9848	**0.7386**
2.prompt	0.8068	1	1	**0.8068**
3.prompt	0.5227	1	1	**0.5227**
4.prompt	0.1477	0.3	1	**0.1136**

Table 4: Phi 4 Vision Multimodal evaluation results

3.4.5 Quality comparison



(a) Average percentages comparison

(b) Best percentages comparison

Figure 13: Comparison of average and best percentages for each VLM

As shown in Figure 13b, comparing the best configurations across all three models reveals clear performance differences. LLaVA 1.6 achieves a maximum overall percentage of 86.4%, while Phi 4 Vision Multimodal reaches 80.7%. In contrast, Qwen 2.5 VL 7B achieves the highest overall percentage of 97.7%, demonstrating superior performance across all evaluation criteria. The average performance comparison in Figure 13a further confirms Qwen 2.5's consistency, showing greater robustness across different prompt variations and output formats. Based on this comparative analysis, **Qwen 2.5 VL 7B with YAML output format and 16-Bit quantization is selected** as the VLM for this framework.

3.4.6 Speed comparison

Furthermore, another factor that may influence the selection of the VLM is the inference time, specifically the number of tokens generated per second (tok/s). In this study, the quality of the output was prioritized over inference time. However, it remains relevant to examine the performance of different models in terms of inference time and tok/s, as these factors can impact real-time applications and overall system performance.

Although not scientifically evaluated, it was observed during testing that models with lower output quality often exhibited shorter inference times and higher token generation rates. This behavior is likely due to the reduced computational complexity, as these models process fewer or less intricate layers to generate outputs. Table 5 presents the observed inference times and tok/s for each model, based on the test set described earlier. The tok/s were recorded from the console output of the Docker container during the inference process. It is important to note that the time required to generate the first token is significantly higher than the average tok/s. This is because the model must process the input and generate the initial token before it can produce subsequent tokens at a faster rate (Horton et al., n.d.). This longer initial processing time is included in the calculation of the average tok/s.

The results indicate that the Phi4 model achieved the highest tok/s, which aligns with its lower

overall score and output quality. Conversely, Qwen 2.5, which demonstrated the best performance, had the lowest tok/s. Additionally, the tok/s were significantly higher for the 16-bit quantization compared to the 4-bit quantization of Qwen. This difference is likely attributable to the optimization and hardware support for 16-bit quantization on the NVIDIA Jetson platform, enabling faster processing and token generation (Jetson AI Lab, 2025). In contrast, the 4-bit quantization may require additional computational resources or lack optimization, resulting in slower tok/s. These observations further support the selection of the 16-bit quantization for this framework, as it offers both improved output quality and faster inference times compared to the 4-bit quantization.

Table 5: Tokens per second comparison

Model	**Tokens/s at 16 concurrent tasks**	**Tokens/s for one task**
llava 1.6	90 tokens/s	5,625 tokens/s
Qwen 4-Bit	30 tokens/s	1,875 tokens/s
Qwen 16-Bit	125 tokens/s	7,8125 tokens/s
Phi 4	350 tokens/s	21,875 tokens/s

3.4.7 Hallucination

Several types of hallucinations were observed during the evaluation of the VLMs. The most common hallucination involved the repetition of objects or relations in the output. This often caused structured formats such as JSON and YAML to become invalid, because once the maximum token limit was reached, the output was cut off and the final closing markers were missing. As a result, the entire structure could no longer be parsed. Although the repetition also appeared in CSV outputs, it did not lead to invalid files, since CSV does not use closing markers. This explains the high repetition rate of Phi 4 with the fourth prompt, because most of the repetition errors passed the format check. It also supports the observation that the repetition rate remained low for the other models and prompts, because a valid format was usually correlated with a reduced likelihood of repetition. In some cases, the output did not match the requested format at all and the model ignored the structural instructions entirely. Occasionally it produced random sentences separated by commas or new lines. An example of this behavior is presented below.

```
nodes, to,cont, and , and, with for pixels, for the row, take in,
and without, contain the,,, in,em,something and, and, and,one and,. of,,,
, and, and,
, in, stare, in,, for a, or, found,some,fast,, that, etc, of,m, but, and,
available, and, and has, then,one, and, and, concatenate, and
, that, its, one,then, and,one,dis, then, then, otherwise, one with,,,,:
nodes,offset, then, then, both,mut in the, and," and"""
```

Another form of hallucination appeared when some models outright refused to produce the requested format. Instead, they claimed to be unable to analyze images. This occurred frequently in Phi 4 Vision Multimodal, which often responded with statements such as “As a language model, I am unable to process images directly. If you provide a description of the image, I can generate the desired output based on that description.” This behavior seems to result from the model interpreting the task as purely text based because the prompt specified “Do not output any text other than the JSON file.” The instruction appears to activate a strict text reasoning mode that overrides the visual processing pathway.

A further hallucination type involved the generation of objects or relations that were not present in the image. This was not quantitatively evaluated, yet it was observed during data inspection that some models invented items such as a popcorn stand or a basketball hoop. In other cases they generated random names like “one three three seven sue brockle hue” or even just inserted links to scientific equipment websites. The challenge with this hallucination type lies in the fact that it affected only some objects or relations while the remainder of the output was correct. This made it difficult to measure and classify, since the overall structure could still be valid even though

individual elements were inaccurate. Concerning object positions, some outputs contained values outside the image boundaries, such as $x = 1.2$ or $y = -0.1$. This indicates that some models struggled to locate objects in a consistent manner. One possible explanation is that the model attempted to predict the center of an object that was only partially visible. Another explanation is insufficient training on accurate spatial prediction or formats.

The final hallucination type involved unexpected changes in language. In some cases the model switched from English to Italian or another language in the middle of the output. This behavior did not appear consistently in a specific model but occurred randomly across different outputs. The cause remains unclear but may be related to multilingual training data. Such language shifts are problematic because they can lead to misunderstandings and incorrect interpretations of the resulting information. This issue also creates the risk that the entire output is silently produced in a language different from the one requested. This could turn the whole graph into another language over time if multiple outputs are chained together.

To handle these hallucinations in operational deployment, the framework employs a skip-and-retry strategy when processing outputs from the model. When an image-prompt pair produces output that violates format or location constraints, the invalid result is discarded and processing continues with the next image in the sequence. This approach is computationally efficient because attempting to mitigate hallucination through alternative quantization levels or prompt rewriting would require model re-inference, matching or exceeding the cost of processing the next image. The framework logs flagged images for post-deployment analysis, but immediate reprocessing with modified parameters is avoided. With Qwen 2.5's 97.7% success rate on optimal configuration, approximately 97 of 100 images are expected to generate valid outputs without intervention, making this strategy practical for continuous operation.

3.5 Framework design

The proposed framework for enhancing safety in HRI environments integrates multiple components to effectively monitor potential hazards. The design is centered on the continuous assessment of risk indicators derived from run-time data, which informs adaptive safety measures. The framework consists of several key modules: data acquisition, SG generation, coordinate conversion, graph combination, iterative refinement, and evaluation. An overview of the framework design is shown in Figure 14.

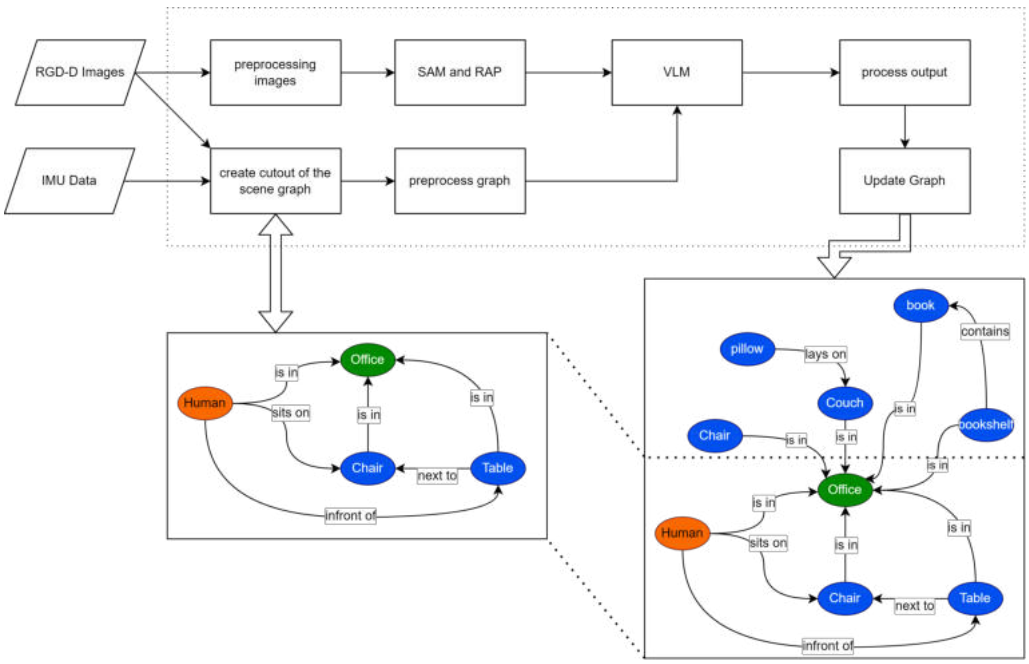


Figure 14: Overall framework design

The first step is to acquire and preprocess data from the sensor and camera. This data is then used to generate a cutout of the SG, which captures potentially relevant information about the environment. The cutout is then converted into the required format for the VLM. In addition, other preprocessing steps can be applied to the data, such as filtering or information reduction. Furthermore, the RGB image can be fed into the SAM model to segment all possible objects. These segmented images are then used to identify possible labels that may occur in the scene. Both sources of information are then combined to create a more detailed prompt for the VLM, which validates the possible objects in the scene. After VLM validation, the output is used to update the complete SG, for example by adding new nodes or edges. This process is repeated iteratively for each new frame. These steps are further detailed in the following sections.

3.6 Data preprocessing

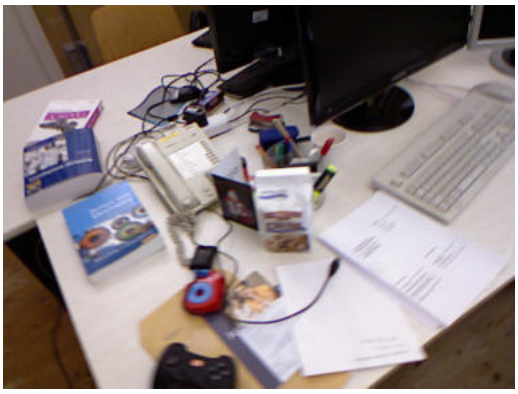
The data preprocessing step involves two main tasks: collecting relevant images and sensor data from the Inertial Measurement Units (IMUs), and creating a cutout of the SG that focuses on the area of interest. As shown in Figure 14, the collected images and IMU data provide the necessary context for understanding the current environment, while the SG cutout allows for tracking relevant objects and their relationships.

3.6.1 Collecting images and IMU data

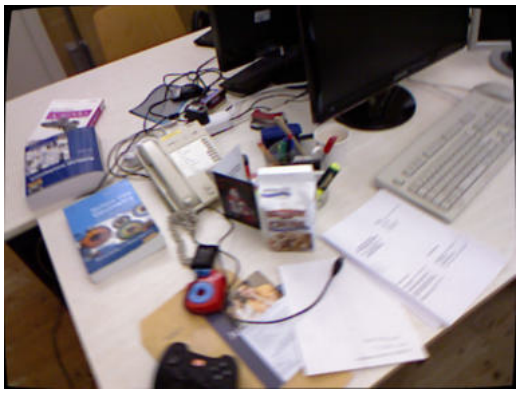
The images and IMU data are collected using sensors and cameras placed within the AS environment. The cameras capture visual information about the surroundings, including the positions and movements of both humans and AS's. This raw data requires preprocessing because the framework is designed to process data in specific formats. Images must conform to configured resolutions, and IMU data must follow standardized coordinate formats to enable accurate spatial transformations and SG operations. The IMU data provides additional context about the orientation of the AS itself and therefore directly influences the perspective from which the images are captured. The images can easily be converted to the required format using image processing libraries such as OpenCV or PIL in Python. These libraries offer a wide range of functions for image manipulation, including resizing, cropping, and format conversion (O. Team, n.d.), making it straightforward to prepare images for different camera outputs and VLM inputs. Also, it is often not necessary to use the highest-resolution images available, as VLMs can often perform well with lower-resolution images, which helps reduce computational load and processing time. The IMU data, however, might require additional preprocessing steps such as filtering or averaging to reduce noise and improve accuracy before being integrated with the image data.

The required IMU format in this framework is a simple array or list of numerical values. This list should contain `[tx, ty, tz]` for translation along the x-, y-, and z-axes, respectively, and `[qx, qy, qz, qw]` for the quaternion representation of orientation. In this standardized format, they denote the position and orientation of the optical center of the color camera, respectively, with respect to the world origin defined by the motion capture system.

Another important preprocessing step is image distortion correction. Cameras, especially wide-angle or fisheye lenses, can introduce distortions that affect the accuracy of object detection and scene understanding. Correcting these distortions during preprocessing is critical for VLM localization accuracy, as distorted images can cause the VLM to provide incorrect pixel coordinates or relative positions for objects, which directly propagate into spatial errors when converting these coordinates to world coordinates for SG construction. Figures 15a and 15b show the difference between a distorted and an undistorted image from the TUM RGB-D dataset (Sturm et al., 2012). In this specific example, the distortion is barely visible at the edges of the image, but in some cases, especially with wide-angle lenses, distortion can be more pronounced, as shown in Figures 16a and 16b.



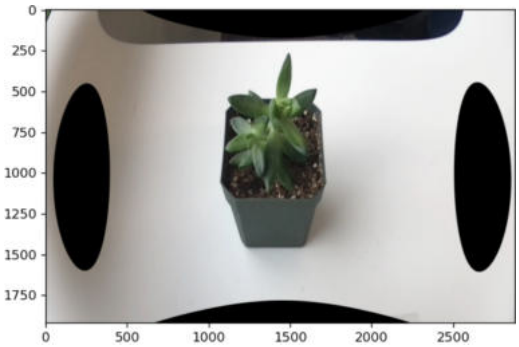
(a) Distorted Image (a)



(b) Undistorted Image (b)

Figure 15: Difference between distorted and undistorted images in TUM-Dataset (Sturm et al., 2012)

(a) Distorted Image (a)



(b) Undistorted Image (b)

Figure 16: Difference between distorted and undistorted images in PlantCV example (Gehan et al., 2017)

For distortion correction, the camera calibration parameters are used to undistort the images, ensuring that straight lines in the real world appear straight in the images. These parameters include the focal length (f_x, f_y), optical center (c_x, c_y), and distortion coefficients $(d0 - d4)$, which are typically obtained through a calibration process using known patterns, for example checkerboards (shown in Figure 17). This calibration process can be automated using libraries like OpenCV (O. Team, n.d.). Some vendors also provide a calibration file with these parameters included, so a separate calibration process is not necessary. The focal length and optical center are sometimes represented as the intrinsic matrix (Equation 1).

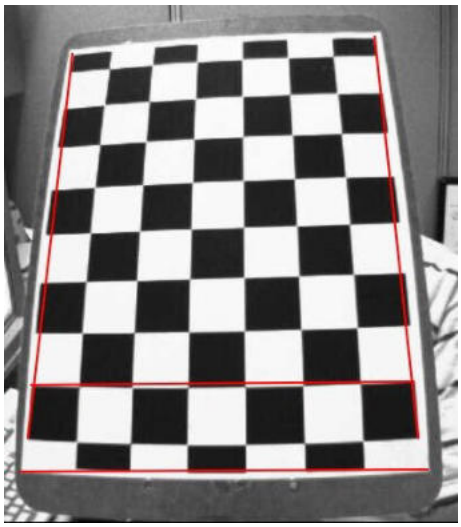


Figure 17: Checkerboard image by OpenCV (O. Team, n.d.)

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

Where:

- f_x, f_y = Focal lengths in pixel units.
- c_x, c_y = Optical center coordinates in pixel units.

3.6.2 Creating a cutout of the SG

To create a cutout of the SG, the first step is to determine the area of interest within the larger environment. Based on this spatial boundary, the corresponding nodes and edges are extracted from the full SG. The AS's IMU data provides the position and orientation needed to localize this area. By computing the camera frustum from the IMU pose, it becomes possible to identify which nodes fall inside the current field of view and should therefore be included in the cutout. A visual example of a camera frustum is shown in Figure 18.

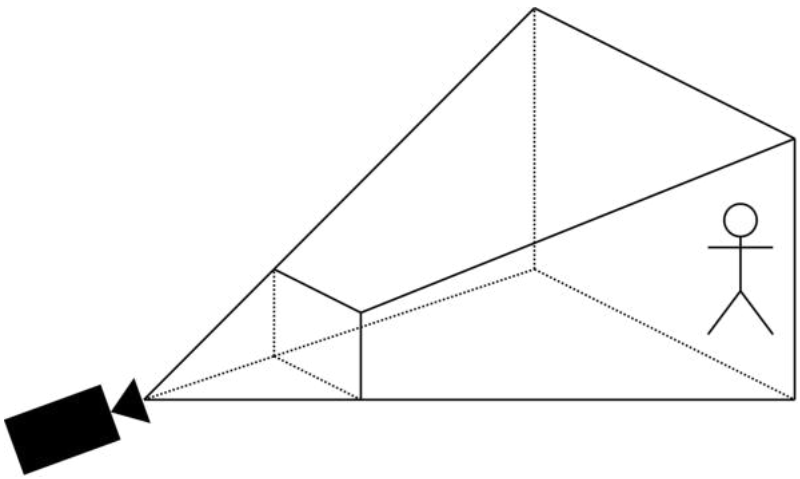


Figure 18: Camera frustum illustration

Using the IMU-derived translation and rotation, the point coordinates are transformed into the camera reference frame via the transformation matrix in Equation 2.

$$\underline{P}_c = \mathbf{R}^T * (\underline{P} - \underline{O}) \tag{2}$$

Where:

- $\underline{P}_c$ = Point in camera coordinate system
- $\underline{P}$ = Point in world coordinate system
- $\underline{O}$ = Camera optical center position in world coordinate system
- $\mathbf{R}$ = Rotation matrix from quaternion

After computing the camera transformation, the frustum planes and the image dimensions can be derived from the camera parameters (Equation 4). Any point in world coordinates can then be tested for visibility by transforming it into the camera coordinate frame (Equation 3) and evaluating its position relative to the frustum planes (Equation 5). A small margin can be added to the frustum boundaries to include nearby nodes that may provide useful contextual information, even if not fully visible. Furthermore, the distance of the point from the camera is also considered, as points that are too far away might not be relevant for the localization task. This depth information can be obtained from the depth image, which provides the distance of each pixel from the camera. By integrating this depth information into the frustum filtering process, it is possible to include only those nodes that are within the field of view and within a certain distance threshold.

This filtering process yields a focused and contextually meaningful cutout of the SG, ensuring that all nodes relevant for localization are retained. The computational cost of this method is $O(n)$, where n is the number of scene-graph nodes, since each node is evaluated once against the frustum conditions. This cost is negligible compared to VLM inference and remains practical for typical indoor AS environments, where SGs rarely contain thousands of objects. In scenarios with significantly larger graphs, spatial partitioning or hierarchical bounding structures could be introduced to reduce the number of nodes that require evaluation.

$$\Theta_x = arctan(\frac{|x_c|}{z_c}) \quad \text{and} \quad \Theta_y = arctan(\frac{|y_c|}{z_c}) \tag{3}$$

$$FOV_x = 2arctan(\frac{|W|}{2f_x}) \quad \text{and} \quad FOV_y = 2arctan(\frac{|H|}{2f_y}) \tag{4}$$

$$\Theta_x < \frac{1}{2}FOV_x * m \quad \text{and} \quad \Theta_y < \frac{1}{2}FOV_y * m \quad \text{and} \quad z_c > 0 \tag{5}$$

Where:

- f_x, f_y = Focal lengths.
- W, H = Image dimensions in pixel units.
- x_c, y_c, z_c = Coordinates of the point in camera coordinate system.
- Θ_x, Θ_y = Angles of the point relative to the camera's optical axis.
- FOV_x, FOV_y = Camera field of view angles in x and y directions.
- m = Margin factor to include nearby nodes.

3.6.3 Preprocess the SG

The SG needs to be passed to the VLM in a parsable format for further processing, such as JSON or YAML. The nodes and edges can be represented as key-value pairs, where each node is represented by its attributes and the edges represent the relationships between nodes. If different VLMs are used, the input format might need to be adjusted accordingly, as some models might require specific formatting or additional contextual information. For this purpose, the framework includes a module that handles this conversion, ensuring compatibility across different VLM configurations. After the VLM processes the input, the output must be parsed and integrated back into the SG by extracting relevant information and mapping it to the graph structure. Since locally running VLMs do not natively enforce structured output formats, parsing can fail when outputs are malformed or do not conform to the expected schema. The VLM evaluation in Section 3.4 examines the observed failure modes in detail and describes the skip-and-retry strategy employed to handle invalid outputs at runtime.

3.7 SG Representation and Constraints

The proposed framework represents the SG as a directed graph structure using NetworkX, a widely-adopted Python library for network analysis and graph manipulation. NetworkX provides an efficient data structure for dynamic graph operations, allowing nodes and edges to be incrementally added, updated, or removed through successive VLM analyses, which avoids the computational overhead of reconstructing the entire graph at each iteration. Furthermore, NetworkX's extensive library of graph algorithms enables sophisticated analysis of object relationships and scene topology when required. The library also integrates seamlessly with established visualization tools such as Matplotlib and Graphviz, facilitating qualitative inspection and debugging of the scene representation during development. Additionally, NetworkX's prevalence in the Python scientific ecosystem ensures compatibility with other frameworks and libraries commonly used in computer vision and scene understanding tasks.

For external communication, however, the graph object itself cannot be directly exposed through an API endpoint, because a native NetworkX object is not easily transferable or serializable in a robust, framework-independent way. Therefore, the framework includes an additional conversion function that maps the SG into a dictionary representation with the same logical structure, including one entry for all nodes and one entry for all edges. This dictionary-based representation can then be returned reliably as API output. For future research, a GraphML export could also be added, but this would require additional handling because GraphML is a specialized interchange format with its own structural constraints.

3.8 Formal Graph Definition

The SG representation is decoupled from the input format passed to the VLM, ensuring that the graph structure remains consistent and interchangeable regardless of the specific VLM input modality or preprocessing pipeline used. Consequently, if alternative input formats or VLM variants need to be evaluated, only the input conversion functions require modification. This design principle significantly reduces code maintenance overhead and facilitates rapid experimentation with different VLM configurations.

The node and edge parameters are as follows:

- Nodes:
  - id (unique identifier for the node, persistent across iterations)
  - node name/class label (e.g., "person", "chair", "table")
  - semantic_description (strict, mandatory, short, precise visual description of what the entity is, including shape, color, material, and key parts, must be evidence-based from the image and must not describe relations such as "next to table" or "in front of door")
  - semantic_pose (strict, mandatory, current physical state or posture of the entity, e.g., "hanging", "standing", "lying", "walking", "moving", "open", "closed", must be evidence based from the image and must not be empty)
  - position of the node in 3D space (coordinates from depth and position information)
  - risk metadata (e.g., "potentially hazardous", "requires attention")
  - appearance attribute (how frequently the object is detected)
  - confidence score for the object detection
  - layer type (e.g., object, human, room, building)
- Edges:
  - from node id (source node)
  - to node id (target node)
  - relationship label (e.g., "next to", "on top of", "inside")
  - distance between the two nodes (calculated from their positions)

These parameters enable the SG to capture both the semantic and spatial aspects of the environment. Due to the design choice of using NetworkX, additional attributes can be easily incorporated into the graph structure as needed. For example, temporal information (e.g., timestamp of detection) could be added in future research to further enhance the scene understanding capabilities of the framework.

In particular, the two mandatory semantic fields per node, *semantic_description* and *semantic_pose*, decouple visual identity from relational context, which is handled by the edges. This separation improves consistency of downstream reasoning. For example, a "chair" node would have a semantic description of "a four-legged object with a backrest and a seat, made of wood", while its semantic pose could be "standing" or "lying on the floor".

3.9 Coordinate Conversion

This section formalizes the coordinate conversion methodology used throughout the framework to map detections and relations to a consistent world coordinate system. Because the camera moves over time, object positions expressed in image pixel coordinates are not invariant. Consequently, all observations must be converted into the image space prior to prompting the VLM and, after inference, mapped back to the world frame to maintain a stable scene representation.

Frames and Notation The approach considers a world frame $\mathcal{W}$ and a camera frame $\mathcal{C}$. The camera pose at any time is defined by its optical center $\underline{O} \in \mathbb{R}^3$ and orientation given by a rotation matrix $\mathbf{R} \in SO(3)$ (from the quaternion as discussed previously). A point observed in the world is $\underline{P} \in \mathbb{R}^3$, and its coordinates in the camera frame are $\underline{P}_c \in \mathbb{R}^3$.

World Camera Transform The forward and inverse transforms between frames are

$$\underline{P}_c = \mathbf{R}^T (\underline{P} - \underline{O}) \quad \text{and} \quad \underline{P} = \mathbf{R}\, \underline{P}_c + \underline{O}, \tag{6}$$

which mirrors the formulation introduced earlier (see equation 2). This ensures that any pixel-localized estimate can be consistently expressed in $\mathcal{W}$, independent of camera position.

Camera Projection Model The mapping from $\underline{P}_c$ to image coordinates uses the pinhole camera model with intrinsics matrix $\mathbf{K}$ (cf. equation 1). Let normalized coordinates be (x_n, y_n) after distortion correction (see Section 3.6). With depth $z_c = [\underline{P}_c]_z$ the relations are

$$x_c = x_n\, z_c, \quad y_c = y_n\, z_c, \quad \underline{P}_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}, \tag{7}$$

and the projection to pixels (u, v) follows

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K} \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix}. \tag{8}$$

Before VLM: World to Current View Before calling the VLM, all nodes stored in world coordinates are transformed into the camera frame (equation 6) and projected to pixels (equation 8), yielding (u, v) positions that match the camera's pose at that moment. This keeps the world graph stable while presenting the correct current view to the VLM.

After VLM: View Back to World VLM outputs that reference image pixels are mapped back to world coordinates. Given a pixel (u, v) and depth z_c from the aligned depth sensor, the values are undistorted to (x_n, y_n). The back-projection with equation 7 yields $\underline{P}_c$, and the inverse transform in equation 6 recovers the world position. This keeps detections valid as the camera moves and lets new points merge consistently into the SG over time.

Consistency and Invariance Maintaining the SG in the world frame ensures temporal coherence across iterations and supports reasoning about object relationships that does not depend on the current viewpoint. This creates a clear separation between perception tied to the instantaneous camera pose and a persistent representation that remains stable over time. The conversion process assumes calibrated intrinsics, aligned depth measurements, and reliable pose estimates from the tracking system. Although individual projections may introduce small inaccuracies, each iteration re-anchors object positions using current pixel observations and depth values, preventing the accumulation of localization errors and stabilizing object indexing and relationship analysis across time.

3.10 Graph combination strategy

Object detection performance is affected by classification errors, particularly false positives and false negatives. Understanding these error modes is essential for designing robust SG integration mechanisms. Table 6 summarizes these cases and their implications for the framework.

Table 6: Confusion Matrix: Classification Outcomes and Definitions

Classification Outcome	Definition
True Positive (TP)	Object correctly detected and added to graph
True Negative (TN)	Object correctly not added to graph
False Positive (FP)	Non-existent object incorrectly added to graph
False Negative (FN)	Existing object incorrectly omitted from graph

Several complementary strategies are commonly employed to mitigate such errors. **Multi-view confirmation** requires objects to be detected across multiple camera views before integration, reducing transient FP at the cost of increased latency. **Temporal consistency filtering** favors objects that persist across consecutive scans, as short-lived detections are more likely spurious. **Geometric validation** enforces physical plausibility by rejecting detections that violate spatial constraints, such as floating objects or interpenetrating surfaces. **Minimum confidence thresholds** further prevent low-certainty detections from being incorporated into the SG.

While these techniques enhance robustness, the proposed framework primarily relies on an appearance score-based graph combination strategy to manage persistence and removal over time. After VLM processing, the SG before inference is compared with the output graph. Each node and edge is assigned a stable identifier to enable consistent tracking across observations. Graph integration follows an update-or-insert principle. Nodes with matching identifiers are updated, whereas unmatched identifiers are inserted as new nodes. Edges are similarly overwritten when existing or added when absent. When conflicting properties arise, the most recent attributes that remain spatially consistent are retained.

An appearance score is maintained for each node to represent its likelihood of persistence. When a node appears in both graphs with consistent properties, its appearance score is incremented. When a node is absent from the VLM output, its score is decremented. A node is removed only when its score falls below a predefined threshold, preventing premature deletion due to isolated missed detections. Objects repeatedly observed at consistent locations accumulate high scores and are treated as stable elements of the environment. Conversely, newly introduced or transient objects retain low scores and can be flagged for further verification. Through this mechanism, the framework inherently balances FP and FN based on multi-view and temporal confirmation.

3.11 Performance Optimization & Scalability

By providing the VLM additional information about the scene, we can reduce the number of wrong or ambiguous detections, which also reduces FP and FN. One approach is to use the RAP to retrieve information about the detected objects. One way to do this is to feed cutout images of the detected objects directly to the VLM with a possible corresponding label. For larger scenes with many objects, this can lead to a high token consumption, particularly if the VLM has a limited context window. To address this, we can instead provide only the retrieved labels and corresponding bounding boxes, which significantly reduces token consumption while still supplying the VLM with useful object information.

Additionally, providing the VLM with guidelines about the structure of the output can steer it toward the desired behavior. For example, if a person moves from one position to another, the VLM can be guided to update the existing node in the SG rather than creating a new one. Smaller models require more guidance, as they have less capacity to understand context and inter-object relationships. However, overly complex guidelines can themselves degrade performance, as the VLM may struggle to follow them. Therefore, it is important to strike a balance between sufficient

guidance and simplicity. It may be worth considering model-specific guideline sets tailored to each model's capabilities.

The overall runtime of the framework can be optimized by parallelizing its components, since different calculations are currently done in sequence for each frame. Currently, VLM inference on the NVIDIA Jetson can take up to two minutes depending on model, prompt, and input size, while the framework's own processing remains in the range of a few hundred milliseconds even as the graph grows. The computational bottleneck therefore lies exclusively in VLM inference, meaning the framework scales well to larger scenes and the pipeline overhead is negligible relative to overall system latency. If newer and faster models arise in the future, the framework could run into scalability issues, which can be addressed through such parallelization. Additionally, if hardware constraints are present, the system can be configured to skip certain components, such as the retrieval of additional information by the RAP. However, skipping components may result in a less accurate SG, as the VLM will not have access to the additional labels and information from cutout images. Therefore, when configuring the framework for larger scenes, the trade-off between performance and accuracy must be weighed to determine which components can be safely disabled.

4 Implementation

This section describes the practical implementation of the proposed framework. It first introduces the software architecture and the frame-wise processing pipeline, then explains how additional object information is retrieved, how view-dependent scene-graph cutouts are generated, and how VLM inference is performed. Afterwards, it details how new VLM outputs are merged into the global SG and how unknown objects are learned in parallel. Finally, debugging and testing support as well as dataset integration modes are presented.

4.1 Class Diagram

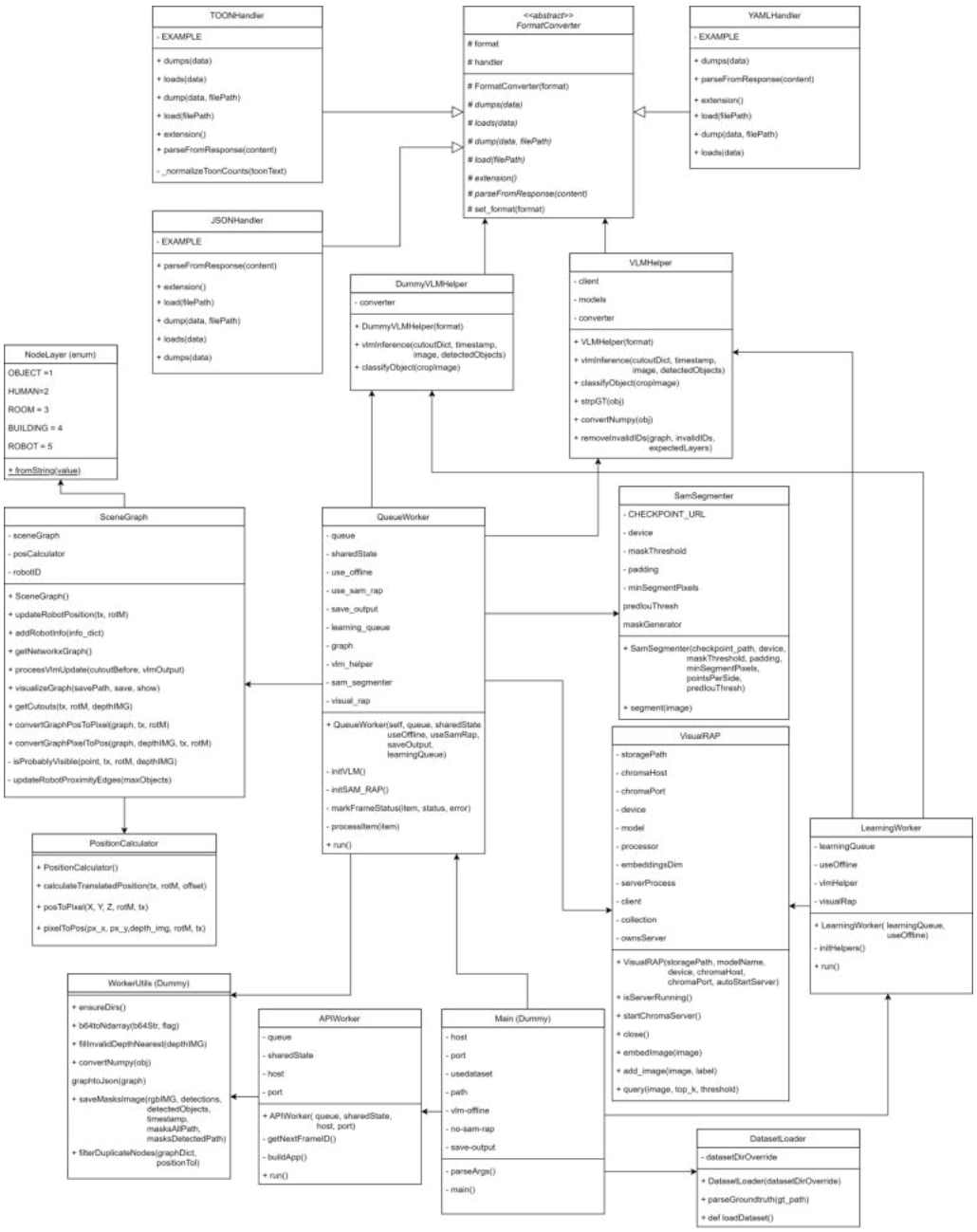


Figure 19: Class diagram illustrating the structure and relationships of the main components within the implementation framework.

Figure 19 highlights the key classes, their attributes, methods, and interactions, providing an overview of how the components collaborate within the framework. The `Main` class and the `WorkerUtils` class are dummy classes that serve only as placeholders for the actual implementation. As shown in the diagram, the framework is built around three `Worker` classes that act as central hubs for managing interactions between components. These workers are started and managed by the `Main` class, with each worker running in its own thread. This allows the system to handle multiple tasks concurrently. Additional workers can be added without affecting existing ones because they interact only through shared queues and dictionaries. An explanation of the main workers and their responsibilities is provided below:

- **API Worker:** The API endpoint worker serves as the interface between external clients and the internal processing framework. It handles incoming requests and forwards them to the appropriate queues for processing by other workers. It also manages responses, such as retrieving the current SG from the state dictionary, providing the current state of the system, and tracking specific frames.
- **Queue Worker:** The queue worker is responsible for retrieving tasks from the shared task queue and processing them accordingly. It checks for new tasks in the queue, processes one frame at a time, and updates the shared dictionary with the processed outputs. This is the main worker that handles the core functionality of the framework, such as image preprocessing, generating SG cutouts, position conversion, passing information to the VLM, and combining new output with the existing SG. It uses various utility functions from the `WorkerUtils` class to perform these operations. It coordinates the `SceneGraph`, `VLMHelper`, `VisualRAP`, and `SAMSegmenter` classes to achieve the desired results from the pipeline.
- **Learning Worker:** The learning worker handles tasks related to learning new information and labels encountered in the pipeline. For example, if the `SAMSegmenter` detects an unknown object, the learning worker can use the image cutout, prompt the VLM to generate a new label, and add it to the knowledge base for future reference. This enables the system to continuously learn and adapt to new objects and scenarios over time without interfering with the main processing tasks handled by the `QueueWorker`.

Furthermore, the diagram shows the different classes used by the workers. The `SceneGraph` class handles scene-graph operations such as creating cutouts, combining new output with the existing graph, manipulating nodes and edges, and providing the current graph state. It uses the enum class `NodeLayer` to distinguish graph layers and filter them when needed. For position calculations and coordinate conversions, it also uses the `PositionCalculator` class, which converts pixel coordinates to real-world positions based on camera parameters.

In the processing pipeline, the `QueueWorker` uses the `SAMSegmenter` class to segment objects, while the `VisualRAP` class is responsible for retrieving relevant or similar labels from the knowledge base. The `VisualRAP` class is also used by the `LearningWorker` to save new labels to the knowledge base, which can later be retrieved when similar objects are detected.

The `VLMHelper` class handles interactions with the VLM. It provides methods for passing frames with preprocessed information, such as image cutouts and relevant labels, to the VLM and retrieving the output, as well as generating new labels from image cutouts for the `LearningWorker`. A similar class, `DummyVLMHelper`, supports framework development and testing by returning dummy outputs when no real VLM is available. Both classes use the abstract `FormatConverter`, which converts VLM input and output formats and can be adapted through concrete implementations such as the `JSONHandler`.

4.2 Overall process for one frame

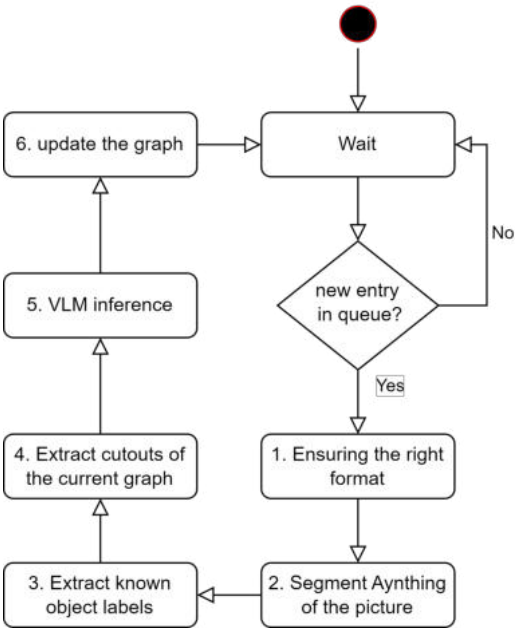


Figure 20: Overall process illustrating the steps the `QueueWorker` goes through for processing one frame.

The overall process is shown in Fig. 20. It illustrates the steps the `QueueWorker` goes through to process one frame. The process starts with the `QueueWorker` checking for new tasks in the shared task queue. If a new task is found, it retrieves the task and starts processing it. The first step is to ensure that the input frame is in the correct format for processing. This includes checking the format and abandoning the frame if it is invalid. If the frame is valid, the process proceeds to the next step, where the RGB image is passed to the `SAMSegmenter` for segmentation. The `SAMSegmenter` processes the image and generates a segmentation mask, which is then used to create cutout images for each detected object. These cutout images are then passed to the `VisualRAP` class to retrieve all known labels. Afterwards, a cutout of the SG is created by the `SceneGraph` class, which contains only the possible nodes and edges relevant to the current view. All this information is then gathered and passed to the `VLMHelper` class, which handles the interaction with the VLM. After the inference result from the VLM is retrieved, the new output is combined with the existing SG. The same sequence is repeated for each incoming frame.

Each frame is tracked by its unique frame ID, which is used to manage processing and ensure that the correct information is associated with each frame throughout the pipeline. The frame ID also tracks the processing state for each frame, such as whether the frame has been completed, failed at any point, is still being processed, or is still waiting in the queue. This allows better management of the processing pipeline and helps identify issues or bottlenecks in frame processing. The simple state diagram in Fig. 21 illustrates the different states a frame can be in during the processing pipeline and how it transitions between these states based on the processing outcomes.

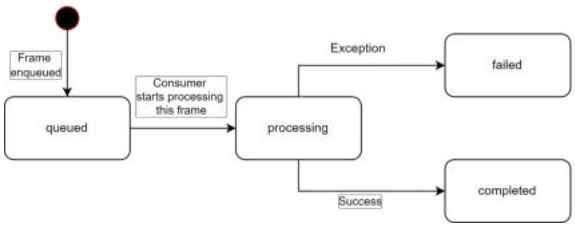


Figure 21: State diagram illustrating the different states a frame can be in during the processing pipeline.

4.3 Retrieving additional information

The first important step for each frame is to retrieve additional information about possible objects in the scene. Figure 22 shows the interaction between the framework components during this retrieval process. It starts with the `QueueWorker` passing the complete image to the `SAMSegmenter` class, which generates segmentation masks for the input image. As the segmentation model, we use SAM (Kirillov et al., 2023), which provides a list of segmented objects including the bounding box, segmentation mask, and confidence score for each detected mask. The `QueueWorker` then uses these masks to create cutout images for each detected object. These cutouts are passed to the `VisualRAP` class, which retrieves known labels for them. The RAP generates embeddings for each cutout image using the pre-trained CLIP vision encoder (Radford et al., 2021). These embeddings are compared to a predefined set of image and label embeddings generated with the same encoder. The vector database `ChromaDB` (C. Team, 2022) stores and manages these embeddings and returns the label with the highest similarity score for each cutout image. If the label is not **unknown**, it is added to a list of possible labels together with the corresponding confidence score and bounding box. Otherwise, it is added to the learning queue for further processing by the `LearningWorker`. After this step, the `QueueWorker` has a list of possible labels for each detected object to enrich the VLM input.

If hardware constraints are present, the system can be configured to skip this retrieval step and pass only the scene-graph cutout to the VLM. This reduces computational cost but may lower scene-graph accuracy because the VLM no longer receives the additional labels derived from the cutout images.

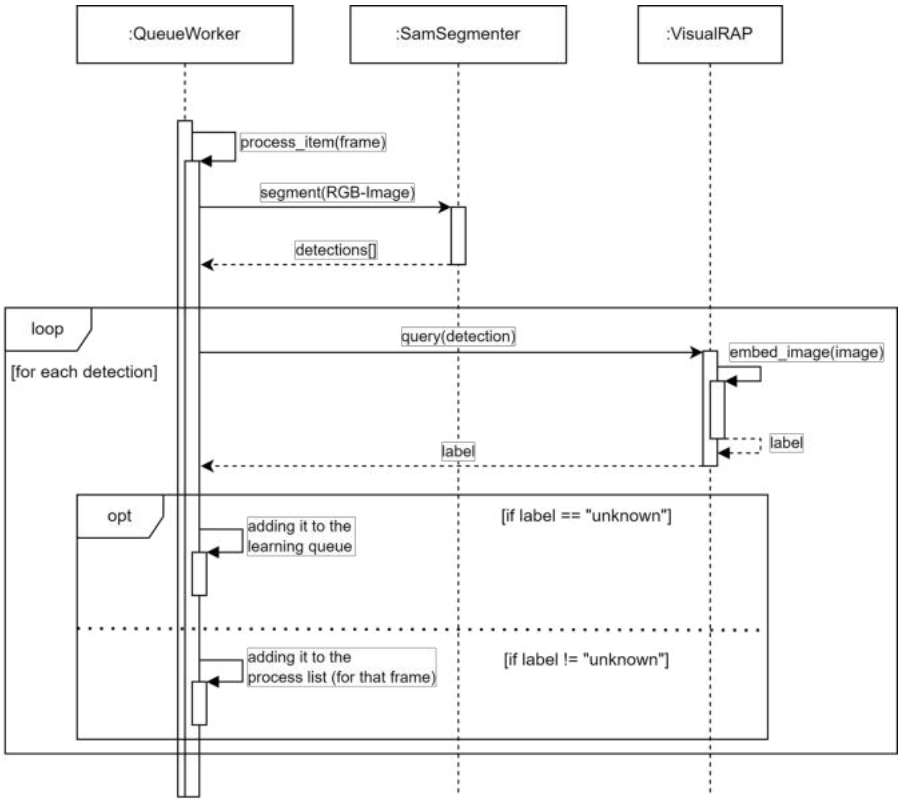


Figure 22: Sequence diagram illustrating the interaction between the different components of the framework for retrieving additional information.

4.4 Generating cutout of the SG

The second important step for each frame is to generate a cutout of the SG that contains only the nodes and edges relevant to the current view. The required function calls are illustrated in the sequence diagram in Fig. 23. The process starts when the `QueueWorker` passes the IMU data and depth image to the `SceneGraph` class by calling `get_cutouts(tx, rotM, depth_img)`. The `SceneGraph` class then determines for each node whether it is within the current view. To do so, it calls `is_probably_visible(pos_node, tx, rotM, depth_img)`, a function provided by the `PositionCalculator` class. After converting the node's real-world position to camera coordinates, it checks whether the node lies within the camera's field of view and uses the depth image to determine whether it is occluded or otherwise not visible. If a node is probably visible, it is added to the scene-graph cutout. Edges are added if both of their nodes are included in the cutout. At the end, all nodes are converted to camera coordinates so that the VLM can use the scene-graph cutout together with location information during inference.

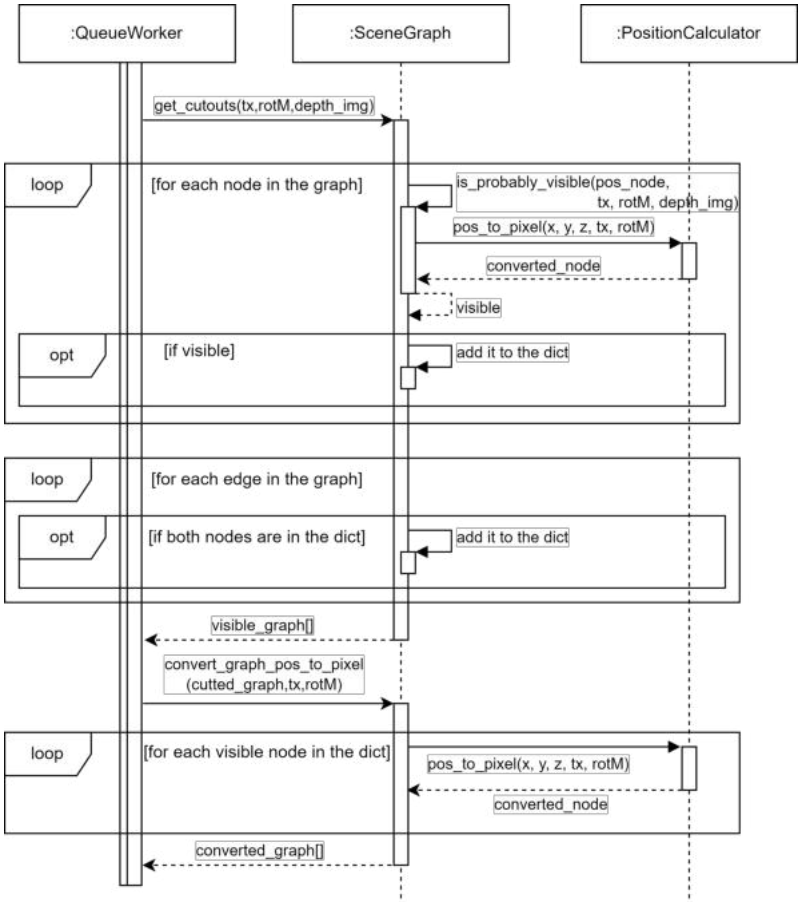


Figure 23: Sequence diagram illustrating the interaction between the different components of the framework for generating cutout of the SG.

4.5 Inference process

After gathering all necessary information, the next step is to pass it to the VLM for inference, as shown in Fig 24. For this purpose, the `QueueWorker` calls the `VLMHelper` class and passes the scene-graph cutout, the possible labels for each detected object, and the corresponding RGB image. `VLMHelper` prepares this information for the VLM, including formatting the SG by calling the `dumps(cutted_graph)` method of the `FormatConverter` class. Because `FormatConverter` is the parent class of all handlers, the same function call can be used for different formats while the active handler provides the concrete implementation. After the SG is converted into a parseable format, `get_example()` is called to obtain the format example for the current handler. This example defines how the VLM should output the new information. The output format is kept identical to the input format to reduce the chance of confusing the VLM. Next, the prompt is created by filling in the prompt template shown in Section A.5 in the appendix. Some placeholders have variable length depending on the current SG or the detected objects. Once the prompt is created, it is sent to the VLM via an OpenAI API call. Afterwards, `VLMHelper` uses a function provided by `FormatConverter` to parse the VLM output back into a scene-graph representation, which at this stage is a dictionary of nodes and edges. If parsing fails, the parser attempts to repair the format and parse it again. For example, if the VLM outputs JSON with additional prefixed text, the parser searches for the first and last curly brackets and parses only that part. If this succeeds, it returns the parsed dictionary. Otherwise, it returns a dictionary with a `"raw"` key containing the raw VLM output. The final dictionary is then returned to `QueueWorker` together with a `valid` flag. If the flag is true, `QueueWorker` updates the current SG with the new information from the VLM. Otherwise it marks the frame as failed.

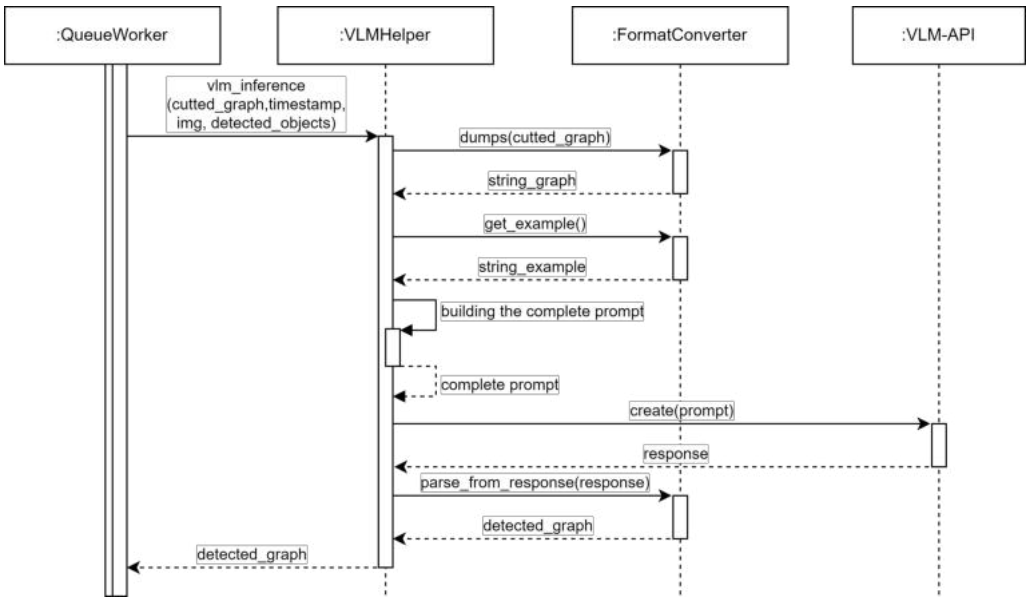


Figure 24: Inference process of the VLM illustrating the steps the `VLMHelper` goes through for processing the information and interacting with the VLM.

4.6 Combining SGs

The combination of the cutout SG and the complete SG is performed after the VLM inference process. For this purpose, the `SceneGraph` provides the function `process_vlm_update(cutout_Before ,vlm_Output)`. It takes the cutout SG before inference and the output of the VLM as input. The function then compares the cutout SG with the VLM output and updates the complete SG accordingly. The complete workflow is shown in Fig. 25. The function loops through each node in the VLM output and checks whether there is a matching node in the complete SG. If a matching node exists, it updates the node information in the complete SG using the information from the VLM output. Furthermore, it increases the appearance score of this specific node. If no matching node exists, it adds the new node to the complete SG. After all nodes from the VLM output are processed, the function loops through all nodes in the VLM input and checks whether any nodes are no longer detected by the VLM. For nodes that are no longer detected, it decreases their appearance score. If the appearance score of a node falls below a certain threshold, the node is removed from the complete SG. After all nodes are processed, the same procedure is applied to the edges of the SG. One additional processing step for edges is the calculation of the distance between nodes. This is done by using the node positions and calculating the Euclidean distance between them in the function `calculate_edge_distance(src, dst)`. This distance is then added as a new edge attribute, which can be used for further processing or visualization.

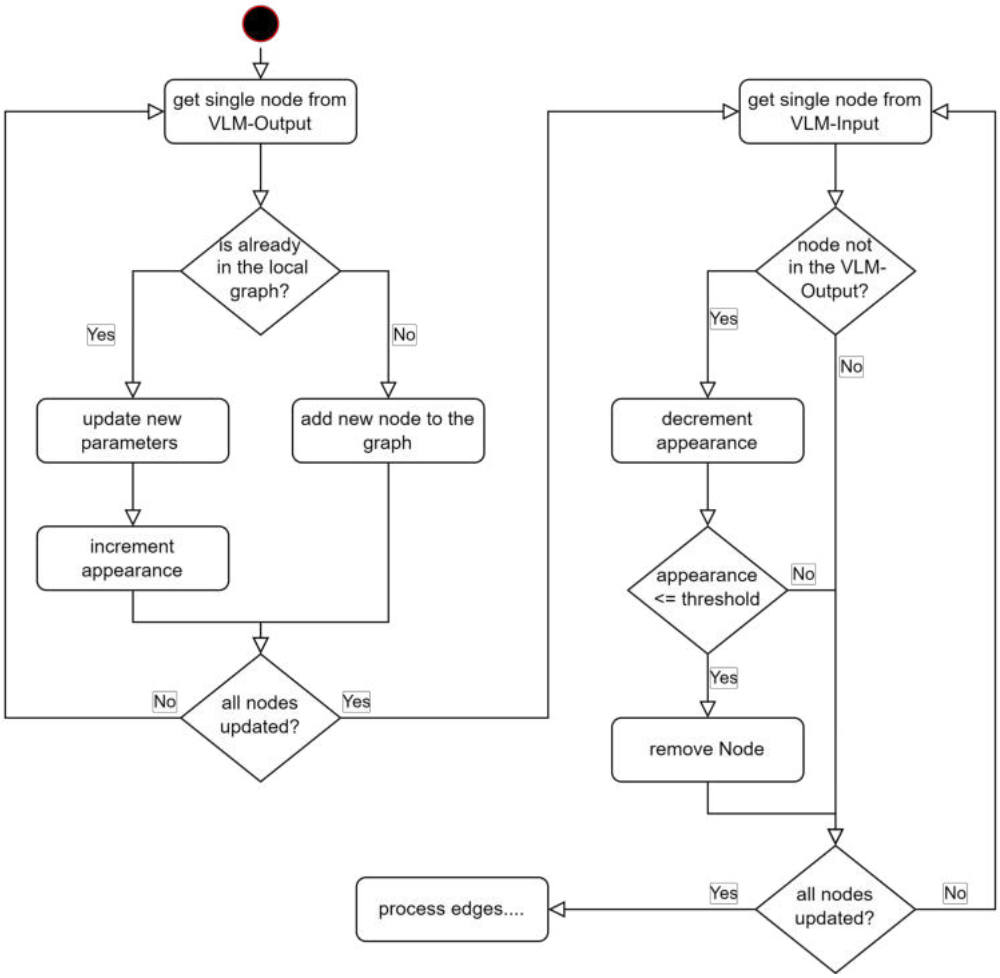


Figure 25: Combining SGs illustrating the steps the `SceneGraph` class goes through for processing the information and updating the complete SG.

At the end, the AS position is updated by the `update_robot_position(tx)` function of the `SceneGraph` class, called by the `QueueWorker`. Because the VLM cannot detect the AS position directly, the camera position is used as a reference point. If the AS is much larger than the

camera, this estimate can deviate noticeably from the true AS position. In that case, an additional transformation based on the AS size and camera mounting position could improve the estimate. In this work, however, the camera position is assumed to be sufficiently accurate.

4.7 Learning unknown objects

The Learning Worker is responsible for learning unknown objects that were not detected by the RAP. Such objects are added to a list of unknown objects, as described in Section 4.3. The Learning Worker periodically checks this list and processes each object individually. Figure 26 shows this sequence. The process starts with the `LearningWorker` retrieving the cutout image of an unknown object from the list. It then uses the VLM to generate a label for this cutout, and the resulting label-image pair is added back to the RAP for future reference. The RAP vector database runs as a lightweight background server, which enables concurrent access for both the `QueueWorker` and the `LearningWorker`. If the database were stored locally in the file system, both workers could not access it at the same time. For example, when the Learning Worker adds a new label, it would need to lock the database, write the new label, and reload the entire database. This would delay the `QueueWorker` pipeline and require additional lock management. Running the vector database as a lightweight server avoids that overhead and improves pipeline efficiency.

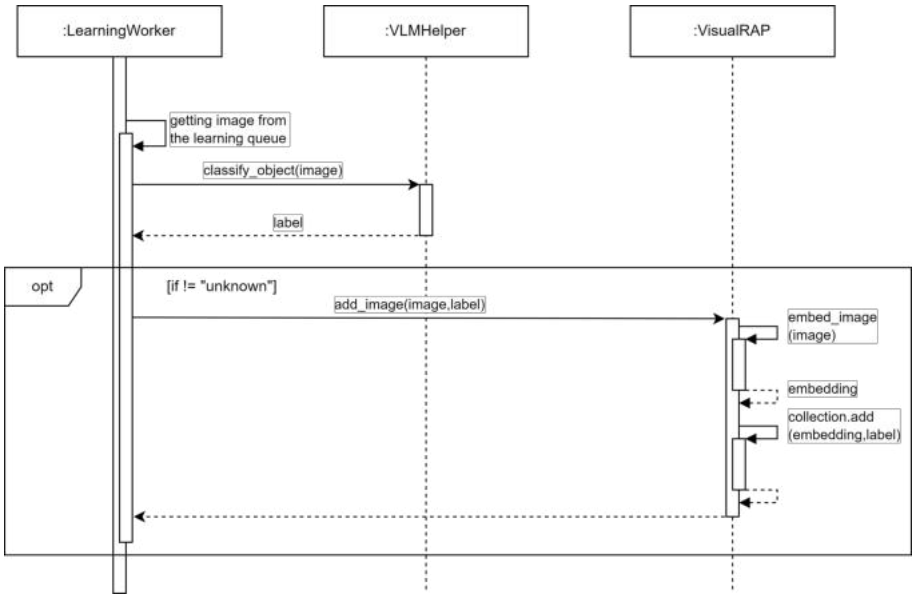


Figure 26: Sequence diagram illustrating the interaction between the different components of the framework for learning unknown objects.

4.8 Debugging and testing

The framework provides a debugging and testing flag that generates additional diagnostic outputs. When enabled, the framework saves intermediate outputs from different components. For example, it stores the VLM input, that is, the scene-graph cutout, together with the corresponding VLM output for each frame. This helps analyze VLM processing and identify issues or bottlenecks during inference, especially when the VLM shows limitations such as a restricted context window or hallucination effects. In addition, the complete SG is saved for each frame, making it possible to track how the graph evolves over time without requesting it repeatedly through the API. These saved outputs can also be used to generate custom datasets for further analysis without rerunning the entire framework. For visualization, the framework renders the graph as an image using matplotlib to provide a clearer view of the scene-graph structure and node-edge relationships.

4.9 Dataset Usage

There are two ways to feed a dataset into the framework. The first option is to place the RGB images, depth images, and corresponding camera poses in the dataset input folder. When the framework is started with the `--usedataset` flag, it automatically reads and processes this data at startup. A second option is available for testing from a different host or for simulating a more realistic scenario in which frames arrive over time instead of being available from the beginning. For this purpose, the predefined script `DatasetClient.py` in the testing folder can be used to send the dataset to the framework via the API endpoint. The script reads the RGB images, depth images, and corresponding camera poses from the input folder and can be configured to introduce a delay between images. This makes it possible to test different scenarios, such as tuning the delay to match the optimal processing time per frame. Individual frames can also be sent through the OpenAPI interactive documentation at `localhost:8005/docs`.

5 Results

This section summarizes the experimental findings and discusses their implications for the proposed framework. It first presents the test runs and observed system behavior, then analyzes VLM performance and practical limitations, and finally discusses implemented mitigation strategies together with future research directions.

5.1 Test runs

To evaluate system performance, two test runs were conducted under different conditions. The first run used a static environment to evaluate object detection and classification in a controlled setting. The second run included a moving person to evaluate real-time tracking, node updates in the SG, and the appearance-tracking mechanism that controls node insertion and removal over time. The results are presented in the following sections.

5.1.1 Experimental setup

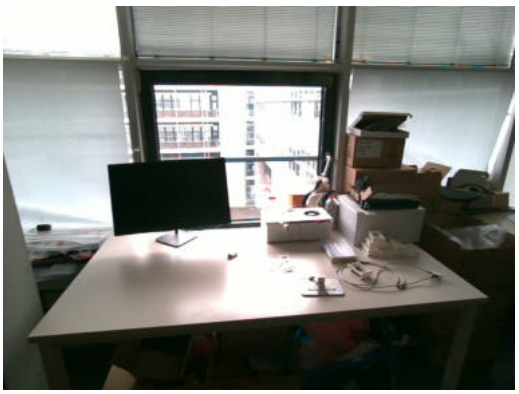
Both test runs were recorded in a standard indoor laboratory environment. The scene included objects that are uncommon in standard object detection datasets, such as a 3D printer and a robotic testing rig. Common objects such as chairs, tables, and monitors were also present. An Intel RealSense D455 RGB-D camera was used to capture color and depth information. The camera placement differed between the two test runs, as described in the corresponding sections. During data acquisition, RGB and depth frames were captured at 15 frames/s, and an additional reference video was recorded at 30 frames/s. The image resolution was 640×480 pixels, which provided a suitable trade-off between data quality and processing time. Each RGB frame was stored with a timestamp to support synchronization with depth data during processing. The recorded sequences were then used to generate simulated IMU data.

One limitation of the D455 is that it provides acceleration and angular velocity measurements, but not absolute camera position. Therefore, camera pose was estimated from RGB and depth data using visual odometry in Open3D. This method estimates relative camera translation and rotation from differences between consecutive frames. After data collection, it was observed that this pose estimation is sensitive to lighting conditions, scene texture, and camera motion. As a result, pose errors can propagate to the SG, particularly in sequences with significant camera movement. These constraints must be considered when interpreting the results. Despite this limitation, the experiments still provided relevant insights into the system's behavior.

After data recording and pose estimation, the system was evaluated on the recorded datasets. For evaluation, the dataset was pruned to 1 frame/s. Each selected frame and its corresponding odometry data were submitted through the API endpoint using a dummy sender class. During replay, frames were transmitted to the framework with a 1 s delay to emulate an online runtime scenario with sparse updates. The generated SG was stored at each iteration to analyze graph evolution over time and compare behavior across scenarios. The resulting observations are summarized in the following sections.

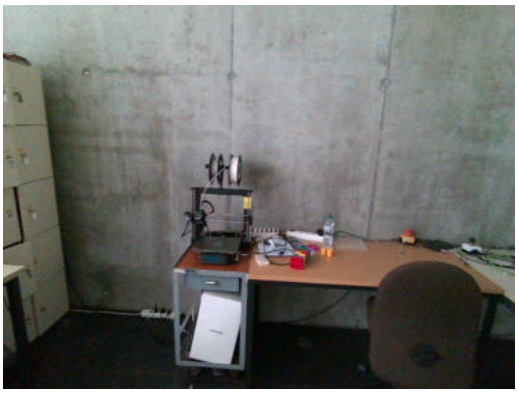
5.1.2 Test run in static environment

In the first run, the camera was mounted on the recording laptop. It was positioned to observe the area in front of the laptop. The laptop was then carried by a person walking through the laboratory to emulate a camera mounted on a AS. The sequence duration was approximately 30 s. Camera odometry was estimated using the method described above. The recorded data were then processed by the framework to generate SGs. Figure 27 and 28 show four snapshots at different iterations of this run. The first image pair shows the initial SG generated from the first frames. The second pair shows the graph after 6 s, where additional objects are detected and new nodes and edges are added. The third pair shows the graph after 12 s with further graph growth and relation updates. The last frame shows the graph at 14 s, where the camera has moved to a different area and new objects are observed.

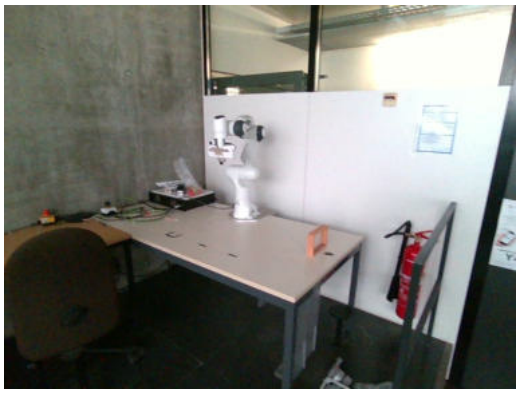


(a) Scene image at 2 s (a)

(b) Scene image at 7 s (b)

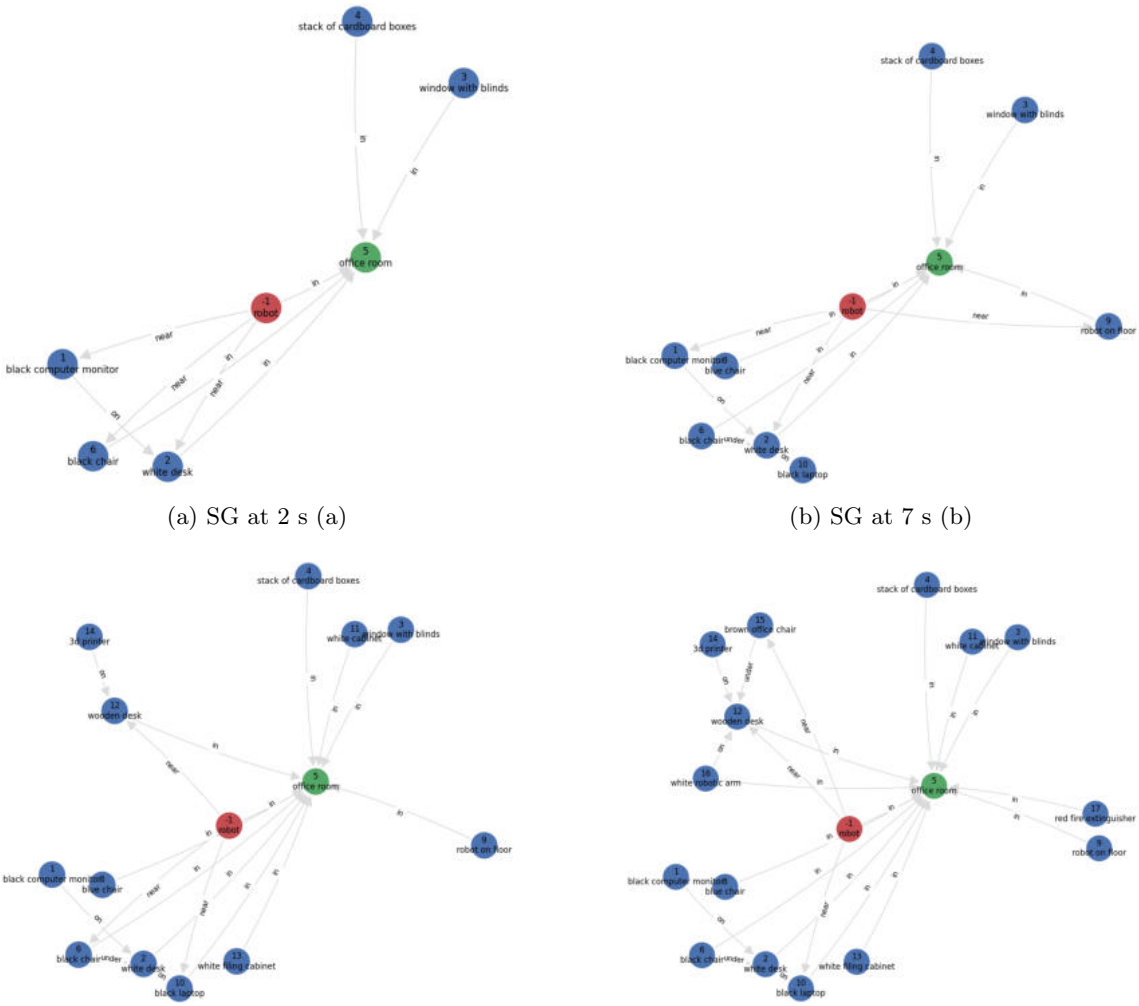


(c) Scene image at 11 s (c)



(d) Scene image at 14 s (d)

Figure 27: Scene images from the lab.



(a) SG at 2 s (a)

(b) SG at 7 s (b)

(c) SG at 11 s (c)

(d) SG at 14 s (d)

Figure 28: SGs from the lab.

Overall, the generated SG is largely accurate. In the first frame, major objects such as a chair, desk, monitor, and cardboard boxes are detected correctly. The relation between monitor and desk is also identified correctly. In the second frame, the camera moves toward another table, and additional objects are incorporated into the graph. A robotic dog is detected and classified as a robot, which is semantically acceptable at the selected abstraction level. In the third frame, more objects become visible, including a 3D printer on a wooden table. The label "wooden table" indicates that the VLM can provide more specific descriptors than generic class names. The relation between the 3D printer and the wooden table is correctly identified as "on". However, the 3D printer is not connected to the room node, which indicates an inconsistency in global scene reasoning.

In the fourth image pair, the camera reaches the corner of the room and reveals a Franka Emika robotic arm. The node is labeled as "white robotic arm", and a "red fire extinguisher" node is also added. The relation between robotic arm and wooden table is again classified as "on". This label is plausible but imprecise, since the arm is physically located on a neighboring table. This behavior is likely caused by the sensor drift, which produces an inaccurate cutout of the SG. The VLM then assigns a relation based on the available candidates in the cutout, which may not reflect the true spatial configuration.

In this static environment test, node classification achieved 95% precision, indicating that the detected and labeled nodes were classified correctly. Edge classification precision was approximately 90%, with several observed errors including inconsistent room node connections and misclassified

spatial relations. These edge errors were relatively sparse and likely resulted from faulty odometry data. Additional errors occurred when the same object was detected multiple times without being merged into a single node. This is likely also caused by sensor drift and pose-estimation inaccuracies. If a cutout fails to include an object that is actually present, the model may generate a new node for it, leaving duplicate representations after merging. This problem will likely be reduced when a proper IMU is used for pose estimation, as it can provide more accurate and stable camera pose data.

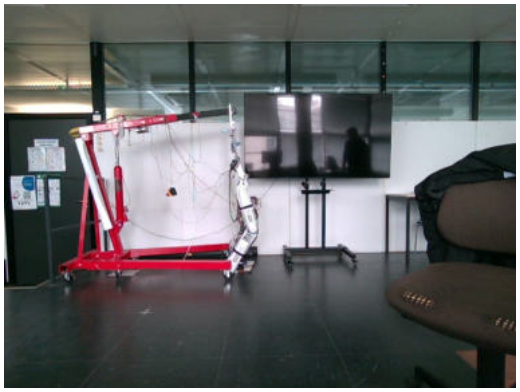
In summary, this run demonstrates that the system can generate a consistent SG from RGB-D input and can update it online as new frames arrive. The moving camera perspective in this test allowed the VLM to observe objects from multiple viewpoints, which improved reasoning about object identities and spatial configuration. The next run evaluates a dynamic scene with human motion but from a fixed camera viewpoint, which introduces additional challenges for temporal tracking and graph maintenance.

5.1.3 Test run with moving human

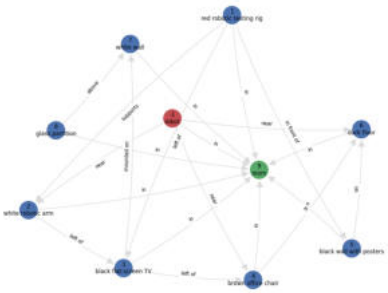
In the second run, the camera was mounted on a tripod placed on a table, which provided a stable viewpoint. The field of view covered a laboratory section in which a person moved through the scene. The person walked from one side of the capture area to the other, paused briefly, and then returned. This setup was designed to evaluate human tracking and corresponding SG updates. Figure 29 shows snapshots from different iterations. The first image pair shows the initial SG from the first frames. In the second pair at 6 s, the person is detected and represented by a new human node. In the third pair at 12 s, this node is removed after the person leaves the observed area.

The appearance tracking mechanism described in Section 4.6 increases the appearance score of persistent static nodes over time. Static objects are observed in most frames, which leads to a stable score increase. For the human node, the score is increased only when the person is detected. After the person exits the scene, the score is decreased at each iteration until node removal is triggered. The same temporal behavior is observed for related edges. These results confirm that the mechanism can maintain graph consistency by adding and removing dynamic entities according to evidence over time.

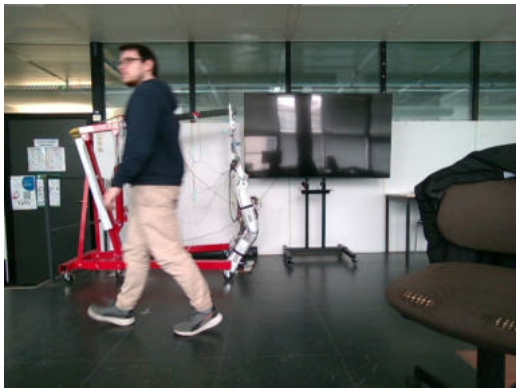
In this moving human test, node classification precision was measured at 80%. This lower precision compared to the static environment test results primarily from the fixed camera viewpoint, which limits the VLM's ability to reason about object identity and spatial configuration from multiple angles. Edge precision was also approximately 85% to 90%, with similar room node connection issues observed. These values are slightly lower than in the static case, partly because the smaller generated SG makes each misclassification more impactful on overall precision. For example, the relation between the television screen and the wall was incorrectly labeled as "mounted on", although the television was actually positioned on a separate stand. An additional source of error in this run is reflection-induced depth noise. In several frames, reflections on the TV screen produced implausible depth values, including distances of approximately $50m$ at specific pixels. This error propagated to node localization and could shift objects to incorrect positions in the SG. In subsequent iterations, such nodes could be excluded from the cutout as "too far away" even though the underlying object was still present. An example depth image of this artifact is shown in Figure 30. Despite these precision challenges, node and edge labels remain reasonably accurate. For example, the robotic leg test setup is identified as a "red robotic testing rig", which demonstrates contextual understanding of uncommon objects. Relation labels such as "supports" and "mounted on" provide richer semantic information than generic labels such as "on" or "next to". The layer assignments are also consistent, which supports stable multi-layer graph construction. The model creates a room node with the generic label "room" and classifies the human as "person", both of which are acceptable outputs for this scenario.



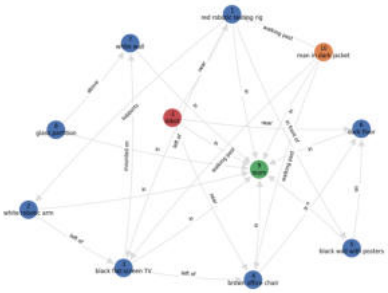
(a) Scene image at 2 s (a)



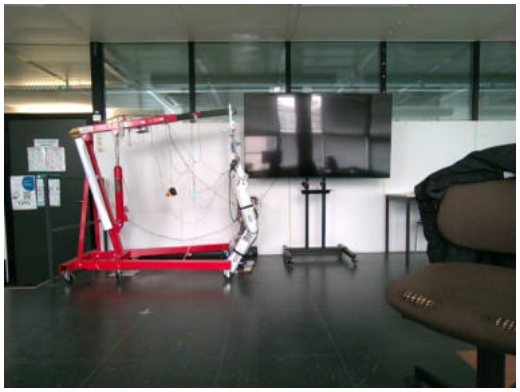
(b) SG at 2 s (b)



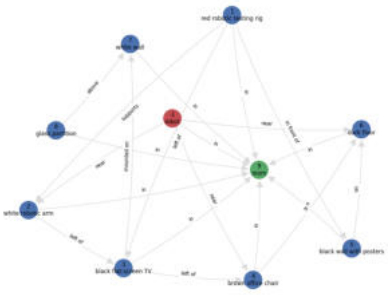
(c) Scene image at 6 s (c)



(d) SG at 6 s (d)

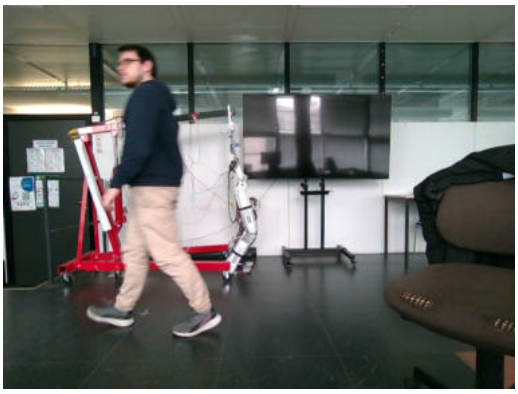


(e) Scene image at 12 s (e)



(f) SG at 12 s (f)

Figure 29: Scene images and graphs of a person walking around the lab.



(a) RGB image with reflection artifact (a)

(b) depth image with reflection artifact (b)

Figure 30: Depth and reflection artifacts in the scene.

It should be noted that precision measurements were feasible because ground truth node and edge labels could be manually verified. However, measuring recall (detecting all true objects in the scene) is not straightforward in this work, since the VLM detection granularity is not fixed. Unlike object detection methods such as YOLO with predefined label sets, the VLM can generate arbitrary object categories and relationship types, making it impossible to define exhaustive ground truth and thus impossible to reliably measure the true detection rate. This limitation must be considered when interpreting the completeness of the generated SGs.

Although some object labels drift due to motion blur and viewpoint changes (e.g., a robotic leg labeled as a "white robotic arm" or a door labeled as a "black wall with poster"), the overall graph structure still captures the spatial layout and interaction patterns in the scene. This preserves its utility for downstream scene-understanding tasks under dynamic conditions.

5.2 VLM performance

For both test runs, the VLM model was selected from the performance comparison presented in Section 3.4, where the Qwen series was identified as the best performing option. During framework development, NVIDIA released an updated container with newer versions of all VLM model series, which prompted the decision to upgrade from Qwen 2.5-VL to Qwen 3-VL for the test runs. This upgrade was motivated by the desire to evaluate the framework with current state-of-the-art VLM performance, while also validating a key design principle of the framework: VLM interchangeability, which allows model updates without requiring modifications to the rest of the framework.

5.2.1 Inference performance and output stability

The Qwen 3-VL model demonstrated significant improvements across multiple performance metrics compared to its predecessor. Output token generation rate increased from 7.8 tokens/s to 23 tokens/s, reducing inference time from approximately 3 minutes per frame to 1 minute per frame, while input token processing rate achieved approximately 1400 tokens/s. Output format stability also improved markedly. The Qwen 3-VL model produced consistently valid outputs throughout all test runs and eliminated the parsing failures that occasionally occurred with the Qwen 2.5-VL model. Location output was also completely stable, which is essential for downstream SG construction, and the model exhibited no spurious repetition, improving further output quality.

5.2.2 Prompt engineering and output format

Achieving reliable output required iterative prompt refinement, since the Qwen 3-VL model, being relatively compact, demands detailed guidance through explicit instructions and examples. Early

prompt versions included destructive operations such as "remove the parts of the JSON", but small models tend to ignore or misinterpret such commands. The solution was to use a marking mechanism where nodes requiring deletion are assigned an ID of -99, which the model reliably interprets and processes, thereby it significantly reduced both hallucination and undesired repetition in the output.

The final prompt is relatively long to provide sufficient guidance, yet output length remained stable across frames, with each image typically generating approximately 10 nodes in the SG and no tendency toward over-specification or spurious node proliferation. This suggests that the prompt constraints effectively limit output scope. Token efficiency is constrained by the use of JSON format (which was used for the testing phase), which requires numerous structural tokens for syntax and indentation. Alternative formats such as YAML or TOON would likely reduce token overhead and decrease overall inference time. Current average output length is approximately 800 to 900 tokens per inference, of which a significant fraction represents structural overhead rather than semantic content. Therefore an additional format converter was added which supports YAML, which can be changed through a flag in the config file.

5.2.3 Semantic understanding and limitations

The VLM demonstrates strong understanding of uncommon objects in the test scenes, with objects such as the "robotic dog" and "robotic testing rig" being correctly classified in a manner that indicates training on diverse datasets. During evaluation, hard FP were rare and limited to the misclassifications already documented in the test run results.

One additional failure mode observed during testing concerns reflective surfaces. In the moving human run, the VLM occasionally detected objects that were later traced to reflections in the large TV screen rather than to physical objects in the scene. In these cases, the model did not reliably distinguish reflected content from real entities, which can introduce false nodes and relations in the SG.

As already mentioned in the test run analysis, the VLM's recall is difficult to quantify due to the lack of fixed classes. As a result, it cannot be stated quantitatively whether the model failed to detect objects or merely deemed them irrelevant given the scene context and existing graph content. Confidence scores were provided by the model but proved unreliable in practice. Early iterations showed high confidence values, such as 90%, for spurious detections like humans in scenes where no persons were present. This behavior resulted from the model accepting the input SG as ground truth without independent verification. After prompt refinement, confidence scores stabilized somewhat, although their utility for downstream decision-making remains limited.

5.2.4 Framework implications and benchmark caveats

Overall, the Qwen 3-VL model provided sufficient performance to generate usable SGs for research and practical applications, while VLM interchangeability enabled straightforward model updates as newer versions became available. The primary limitation of the NVIDIA container deployment is substantial memory consumption, which restricts simultaneous execution of other processes on the same hardware and may impact future system integration or multi-model pipelines.

An additional practical limitation concerns the interpretation of vendor-reported latency values. NVIDIA reports figures such as 3 s to 5 s per frame, but these are typically achievable only under idealized conditions: short prompts, minimally structured input, and outputs of approximately one to two sentences. In this work, the prompt is intentionally long and structured, and also varies between iterations because each inference receives an updated cutout of the SG. A contributing factor is the prompt cache implemented by the container, which optimizes performance when the same prompt is repeated across calls. Under controlled demonstration conditions with a fixed prompt, cache hit rates can approach 100%, whereas in real-world use cases with dynamic prompts, the cache hit rate is substantially lower, which increases inference time relative to vendor benchmarks. These benchmarks should therefore be interpreted with caution, as they likely re-

flect optimized showcase conditions rather than end-to-end downstream workloads with dynamic, structured inputs.

5.3 Handling model limitations

In preliminary testing, additional model-related problems were identified before the final test runs were conducted. The main issues were incorrect or nonsensical VLM responses in specific contexts, likely caused by reliance on training-data patterns that do not always align with real-world scenarios. Smaller models also showed difficulty capturing complex relationships and subtle context, which can lead to suboptimal performance. To address these issues, three mitigation functions were implemented to filter irrelevant or incorrect responses and improve framework robustness. These functions were already in place during the test runs presented in this thesis. However, they also affect runtime performance, because aggressive filtering can occasionally remove relevant information.

The first method was implemented as an additional function in the `VLMHelper` class and is responsible for filtering irrelevant or incorrect responses generated by the VLM. The function takes the model output and applies predefined validation rules to determine whether the response is structurally valid. The first rule removes all nodes with ID `-99`, because smaller models often fail to perform destructive edits reliably, as already mentioned. The function also removes all nodes that do not contain a valid name label, because such nodes cannot be used for later analysis and are therefore not relevant for downstream tasks. Another check removes nodes that share the same ID but appear in different layers, which is structurally inconsistent. This issue occurred frequently during preliminary testing because the VLM sometimes replaced the full content of an existing node instead of creating a new node. For example, if a human was represented with ID 15 but the human was not actually present, the model sometimes changed node 15 to a different label, layer, and position but kept the ID instead of removing it. By checking for identical IDs across different layers, these incorrect responses can be filtered. However, this may also remove potential new detections. After node filtering is completed, the function checks whether all edges remain valid, because some edges may point to removed nodes. Any edge connected to a removed node is also deleted. This process ensures that the final output is usable and does not contain faulty information that could affect downstream analysis.

Another function in the `VLMHelper` class was implemented to reduce the input size for the VLM and thereby reduce overhead while improving framework performance. This function takes the complete SG cutout and filters information that is less relevant for model processing. The filtering strength can be adjusted by changing the dictionary of keywords, which makes the approach adaptable to specific use cases. Currently, information such as inter-node distance and overall confidence is removed from the input because it is not essential for this task. Node positions and labels remain because they are necessary for scene understanding and accurate response generation. Edges can also be removed because they are overwritten after the VLM generates its response, so including them in the prompt is not always required. They may add some context, but tests with smaller models showed no significant impact on output quality. Overall, this method helps mitigate model limitations by reducing prompt size and directing the model to the most relevant information.

A third method was implemented to address limitations caused by hallucinations and the model's tendency toward statistical continuation during structured output generation. During preliminary testing, the VLM occasionally produced multiple nodes for the same object, which introduced ambiguity and reduced SG quality. This behavior was especially common when several objects of the same type appeared in the scene. In such cases, the model often failed to distinguish individual instances reliably and continued generating additional nodes because that pattern had already appeared repeatedly in the output. Since the model operates through next-token prediction, it can statistically favor repeated object structures. Across multiple iterations, this led to an accumulation of redundant nodes, which amplified hallucinations and reinforced repetitive output behavior. To handle this issue, a function was implemented in the `WorkerUtils` class to identify and remove duplicate nodes, even when they are not perfectly identical. Each node is assigned to a bucket based on spatial position, normalized name, and layer. Nodes in the same bucket are treated as duplicates and merged using a union-find structure so that only one representative node remains.

After duplicates are detected, the function updates all edges to point to the retained node rather than removed nodes. It also removes redundant edges and prevents self-loops that may occur during merging. A dedicated rule is applied to room nodes so that only one room node remains, because the VLM frequently generated multiple room nodes although only one room should exist in the given context. By consolidating nodes and repairing graph connectivity, this method prevents the accumulation of repeated objects and reduces the model's tendency to continue repetitive patterns.

The first method proved effective in filtering incorrect responses and was therefore kept as a standard component of the framework. It is always enabled, although its strictness can be adjusted through parameters or implementation changes. The other two methods can be enabled specifically by the user because they are more aggressive and mainly useful for smaller models. For larger models, these methods may be unnecessary because such models generally handle data complexity better and generate more accurate responses without additional consolidation steps. These optional functions can be activated using the `--small-vlm` flag when running the framework. Overall, the methods improve framework robustness and help ensure that generated SGs remain suitable for downstream tasks.

5.4 Future Work

This section outlines the most relevant directions for extending the proposed framework beyond the current implementation. The following subsections focus on improvements in spatial consistency, model selection, and application-level safety analysis.

5.4.1 SLAM

As discussed in the test run analysis, one current limitation of the framework is its reliance on relatively accurate IMU data for camera pose estimation. A relevant direction for future work is therefore the integration of a visual-inertial SLAM system to improve pose accuracy and reduce sensor drift through loop closures. If drift accumulates while the camera is moving, revisiting a previously seen location can correct the estimated trajectory and align node detections to a more consistent spatial reference. This would improve SG accuracy and reliability for downstream tasks while also providing a robust environmental map for navigation and path planning. Because SG nodes already store real-world positions, they could be integrated directly into such a map to support a more intuitive 3D representation of the environment.

From an implementation perspective, the SLAM pipeline could be integrated before the SG update stage so that each node projection uses globally corrected pose information. A practical implementation path would include synchronized timestamp handling, coordinate-frame alignment between SLAM output and graph coordinates, and an interface that propagates loop-closure corrections to stored node positions. Future evaluation should then report trajectory consistency over long sequences, node localization error before and after loop closure events, and the runtime overhead introduced by the SLAM module. This would provide a quantitative basis for deciding whether the additional computational cost is justified for edge deployment scenarios.

5.4.2 Different VLM

Another important direction for future work is the evaluation of different VLMs within the framework. Although the current implementation uses a specific model, many alternatives are available with different strengths and computational requirements. Testing multiple VLMs may identify models that are better aligned with the framework tasks and deployment constraints, and regular reassessment is necessary because model quality is improving rapidly. Maintaining compatibility with recent VLM developments may enable real-time execution on edge devices for AS.

To make this comparison reproducible, future work should define a standardized benchmark protocol across candidate models. Such a protocol can include identical prompts, fixed scene subsets,

repeated inference runs, and common scoring criteria for graph correctness, hallucination rate, inference latency, memory consumption, and structured-output robustness. Identical prompts ensure that performance differences are attributable to model behavior rather than prompt formulation, while repeated runs expose output instability that may remain hidden in single-pass testing. Robustness tests should also assess how reliably each model handles noisy or incomplete inputs, for example under motion blur or partial occlusion, without producing malformed output. These results can support a model selection strategy that maps model classes to target hardware and operational constraints instead of relying on a single default model for all use cases.

5.4.3 Risk Assessment

One of the main goals of this thesis is to provide a framework for SG generation and analysis that can support risk assessment in different application domains. At present, risk is inferred only at a coarse level through semantic scene classification. A key future direction is therefore the development of a dedicated risk assessment module that analyzes generated SGs and produces more detailed hazard insight. Such a module could identify risk factors such as hazardous materials, dangerous object proximity, or cluttered workspaces, combine this information with additional sources such as historical incident records or real-time sensor streams, and support proactive planning through safer path suggestions or workspace rearrangement recommendations.

A useful next step is to formalize this functionality as a multi-stage pipeline with explicit intermediate outputs. The module could derive graph-level indicators such as obstacle density, visibility constraints, and blocked movement corridors, and then aggregate them into a calibrated risk score with interpretable confidence values. Temporal integration across consecutive graph snapshots would also allow trend estimation for early warning in evolving scenes. A possible starting point for this work is the approach proposed by (Ashour et al., 2022), which constructs a risk map based on semantic information. Validation should compare predicted risk levels with expert annotations or incident labels and report precision, recall, and calibration quality. This would turn the current descriptive scene understanding into a measurable and actionable decision-support component.

6 Summary

The implemented framework demonstrates that dynamic SG generation with a VLM can be used as a practical basis for safer HRI in changing environments. Across the test scenarios, the system continuously processed RGB-D frames, generated semantic graph updates, and maintained a coherent representation of objects and relations over time. The resulting graphs remained interpretable and sufficiently stable for downstream safety-oriented reasoning. In particular, the modular pipeline made it possible to process new observations online and to preserve useful contextual information between iterations.

In the static laboratory run, the framework produced strong semantic quality and reliable graph growth while the camera moved through the environment. Object labels were largely correct, including uncommon laboratory equipment, and spatial relations were mostly plausible at the selected abstraction level. In the dynamic run with a moving person, the temporal behavior of the graph was also consistent. Person-related nodes were added when the person entered the scene and removed after leaving it. This confirms that the appearance-tracking mechanism supports graph maintenance in scenarios with dynamic agents.

During development and evaluation, the framework design also proved practically beneficial. VLM interchangeability enabled model updates without restructuring the surrounding pipeline, while maintaining compatibility with the same processing flow. At the same time, the experiments revealed relevant constraints that must be considered in deployment, especially sensitivity to pose quality, reflection-induced depth artifacts, high memory usage of the containerized setup, and limited comparability of vendor latency benchmarks to long, structured prompts with iterative graph context.

Overall, the framework fulfills its intended core functions: online semantic scene understanding, continuous graph updates, and a usable foundation for risk-aware interpretation of AS environments. The implementation is structured for further extension, so additional modules such as SLAM-based global correction, stronger model backbones, or dedicated risk assessment can be integrated without redesigning the full system.

Bibliography

Adadi, Amina and Mohammed Berrada (2018). 'Peeking inside the black-box: a survey on explainable artificial intelligence (XAI)'. In: *IEEE access* 6, pp. 52138–52160.

Alayrac, Jean-Baptiste et al. (2022). 'Flamingo: a Visual Language Model for Few-Shot Learning'. In: *Advances in Neural Information Processing Systems* 35, pp. 23716–23736.

Amaya-Mejía, Lina María et al. (2022). 'Vision-based safety system for barrierless human-robot collaboration'. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 7331–7336.

Apraiz, Ainhoa et al. (2025). 'The user experience in industrial human-robot interaction: A comparative analysis of Unimodal and Multimodal interfaces for disassembly tasks'. In: *Robotics and Computer-Integrated Manufacturing* 95, p. 103045.

Armeni, Iro et al. (2019). '3d scene graph: A structure for unified semantics, 3d space, and camera'. In: *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 5664–5673.

Arrieta, Alejandro Barredo et al. (2020). 'Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI'. In: *Information fusion* 58, pp. 82–115.

Ashour, Reem et al. (2022). 'Semantic hazard labelling and risk assessment mapping during robot exploration'. In: *IEEE Access* 10, pp. 16337–16349.

Bankins, Sarah and Paul Formosa (2023). 'The ethical implications of artificial intelligence (AI) for meaningful work'. In: *Journal of Business Ethics* 185.4, pp. 725–740.

Bavle, Hriday et al. (2022). 'S-graphs+: Real-time localization and mapping leveraging hierarchical representations'. In: *arXiv preprint arXiv:2212.11770*.

Bricher, David and Andreas Müller (2025). 'Analysis of Deep-Learning Methods in an ISO/TS 15066–Compliant Human–Robot Safety Framework'. In: *Sensors (Basel, Switzerland)* 25.23, p. 7136.

Brockmann, Jan Thieß et al. (2023). 'The voraus-ad dataset for anomaly detection in robot applications'. In: *IEEE Transactions on Robotics* 40, pp. 438–451.

Brown, Tom B et al. (2020). 'Language Models are Few-Shot Learners'. In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901.

Cadena, Cesar et al. (2017). 'Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age'. In: *IEEE Transactions on robotics* 32.6, pp. 1309–1332.

Chu, Changyong, Weikang Yang and Yajun Chen (2024). 'Dynamic Fault Tree Generation and Quantitative Analysis of System Reliability for Embedded Systems Based on SysML Models'. In: *Sensors* 24.18, p. 6021.

De Luca, Alessandro and Fabrizio Flacco (2012). 'Integrated control for pHRI: Collision avoidance, detection, reaction and collaboration'. In: *2012 4th IEEE RAS & EMBS international conference on biomedical robotics and biomechatronics (BioRob)*. IEEE, pp. 288–295.

De Simone, Marco, Dario Guarascio and Jelena Reljic (2025). *The impact of robots on workplace injuries and deaths: Empirical evidence from Europe*. Working Paper. Scuola Superiore Sant'Anna, Laboratory of Economics and Management (LEM). DOI: 10.57838/sssa/r3g0-9g43.

Deng, Yinan et al. (2024). 'OpenGraph: Open-Vocabulary Hierarchical 3D Graph Representation in Large-Scale Outdoor Environments'. In: *IEEE Robotics and Automation Letters* 9.10, pp. 8402–8409.

Dettmers, Tim et al. (2023). 'Qlora: Efficient finetuning of quantized llms'. In: *Advances in neural information processing systems* 36, pp. 10088–10115.

Dittrich, Frank and Heinz Woern (2016). 'Robot activity adaptation for safe human-robot collaboration based on probabilistic risk modeling'. In: *2016 18th Mediterranean Electrotechnical Conference (MELECON)*. IEEE, pp. 1–6.

Feng, Chunyong et al. (2025). 'LiDAR SLAM for Safety Inspection Robots in Large Scale Public Building Construction Sites'. In: *Buildings* 15.19, p. 3602.

Gao, Yunfan et al. (2023). 'Retrieval-augmented generation for large language models: A survey'. In: *arXiv preprint arXiv:2312.10997* 2.1, p. 32.

Gehan, MA et al. (2017). 'PlantCV v2: Image analysis software for high-throughput plant phenotyping'. In: *PeerJ* 5, e4088.

Gu, Qiao et al. (2024). 'Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning'. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 5021–5028.

Hagerty, Alexa and Igor Rubinov (2019). 'Global AI ethics: a review of the social impacts and ethical implications of artificial intelligence'. In: *arXiv preprint arXiv:1907.07892*.

Horton, Maxwell et al. (n.d.). 'Kv prediction for improved time to first token, 2024'. In: *URL https://arxiv. org/abs/2410.08391* ().

Huck, Tom P et al. (2021). 'Risk assessment tools for industrial human-robot collaboration: Novel approaches and practical needs'. In: *Safety Science* 141, p. 105288.

Hughes, Nathan, Yun Chang and Luca Carlone (2022). 'Hydra: A real-time spatial perception system for 3D scene graph construction and optimization'. In: *arXiv preprint arXiv:2201.13360*.

International Federation of Robotics (2025). *World Robotics Press Conference 2025 – Presentation*. https://ifr.org/downloads/press_docs/PressConference2025_presentation.pdf. Accessed: 2025-10-22.

International Organization for Standardization (2011a). *Robots and robotic devices - Safety requirements for industrial robots - Part 2: Robot systems and integration*. ISO 10218-2:2011. Geneva, Switzerland.

— (2011b). *Safety of machinery - General principles for design - Risk assessment and risk reduction*. ISO 12100:2011. Geneva, Switzerland.

— (2013). *Safety of machinery - Risk assessment - Part 2: Practical guidance and examples of methods*. ISO/TR 14121-2:2013. Geneva, Switzerland.

— (2016). *Robots and robotic devices - Collaborative robots*. ISO/TS 15066:2016. Geneva, Switzerland.

Jetson AI Lab (2025). *Jetson AI Lab: Research and Development Platform*. Online resource. URL: https://www.jetson-ai-lab.com/ (visited on 10th Dec. 2025).

Ji, Jingwei et al. (2020). 'Action genome: Actions as compositions of spatio-temporal scene graphs'. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*.

Johnson, Justin et al. (2015). 'Image retrieval using scene graphs'. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3668–3678.

Katranis, Georgios et al. (2025). 'Dynamic Risk Assessment for Human-Robot Collaboration Using a Heuristics-based Approach'. In: *arXiv preprint arXiv:2503.08316*.

Kirillov, Alexander et al. (2023). 'Segment Anything'. In: *arXiv preprint arXiv:2304.02643*.

KUKA AG (2025). *Automobilindustrie Automatisierung*. https://www.kuka.com/de-de/branchen/automobilindustrie. Accessed: 2025-02-14.

Lee, Kangdon, Jaeho Shin and Jae-Yong Lim (2021). 'Critical hazard factors in the risk assessments of industrial robots: causal analysis and case studies'. In: *Safety and health at work* 12.4, pp. 496–504.

Lewis, Patrick et al. (2020). 'Retrieval-augmented generation for knowledge-intensive nlp tasks'. In: *Advances in neural information processing systems* 33, pp. 9459–9474.

Liu, Zhihao et al. (2020). 'Dynamic risk assessment and active response strategy for industrial human-robot collaboration'. In: *Computers & Industrial Engineering* 141, p. 106302.

Macenski, Steven et al. (2022). 'Robot Operating System 2: Design, architecture, and uses in the wild'. In: *Science Robotics* 7.66, eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

Mavsar, Matija, Mihael Simonič and Aleš Ude (2025). 'Human intention recognition by deep LSTM and transformer networks for real-time human-robot collaboration'. In: *Frontiers in Robotics and AI* 12, p. 1708987.

Microsoft (2025). *Phi-4 Multimodal Instruct*. URL: https://huggingface.co/microsoft/Phi-4-multimodal-instruct.

Mohammad, Aran, Moritz Schappler and Tobias Ortmaier (2023). 'Towards human-robot collaboration with parallel robots by kinetostatic analysis, impedance control and contact detection'. In: *arXiv preprint arXiv:2308.09633*.

Mumuni, Fuseini and Alhassan Mumuni (2025). 'Explainable artificial intelligence (XAI): from inherent explainability to large language models'. In: *arXiv preprint arXiv:2501.09967*.

OpenAI (2023). *GPT-4 Technical Report*. Accessed: 2025-10-31. URL: https://openai.com/research/gpt-4.

Osasona, Femi et al. (2024). 'Reviewing the ethical implications of AI in decision making processes'. In: *International Journal of Management & Entrepreneurship Research* 6.2, pp. 322–335.

Pan, Jianli and James McElhannon (2018). 'Future edge cloud and edge computing for internet of things applications'. In: *IEEE Internet of Things Journal* 5.1, pp. 439–449.

Primatech Inc. (2017). *Comparison of Process Hazard Analysis (PHA) Methods*. https://www.primatech.com/images/docs/comparison-of-pha-methods.pdf. Accessed: 2025-10-22.

Prof. Dr.-Ing. Andrey Morozov (2024). *Risk Assessment for Robotic Systems*.

Prof. Dr.-Ing. Dr. h. c. Michael Weyrich (2024). *Industrial Automation Systems*.

Quarta, Davide et al. (2017). 'An experimental security analysis of an industrial robot controller'. In: *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 268–286.

Quigley, Morgan et al. (2009). 'ROS: an open-source Robot Operating System'. In: *ICRA Workshop on Open Source Software*. URL: http://www.willowgarage.com/sites/default/files/icraoss09-ROS.pdf.

Radford, Alec et al. (2021). 'Learning Transferable Visual Models From Natural Language Supervision'. In: *Proceedings of the 38th International Conference on Machine Learning*.

Rao, K Durga et al. (2009). 'Dynamic fault tree analysis using Monte Carlo simulation in probabilistic safety assessment'. In: *Reliability Engineering & System Safety* 94.4, pp. 872–883.

Robla-Gómez, Sandra et al. (2017). 'Working together: A review on safe human-robot collaboration in industrial environments'. In: *Ieee Access* 5, pp. 26754–26773.

Rosinol, Antoni et al. (2021). 'Kimera: From SLAM to spatial perception with 3D dynamic scene graphs'. In: *The International Journal of Robotics Research* 40.12-14, pp. 1510–1546.

Satyanarayanan, Mahadev (2017). 'The emergence of edge computing'. In: *Computer* 50.1, pp. 30–39.

Schmid, Lukas et al. (2024). 'Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments'. In: *Proc. of Robotics: Science and Systems*.

Selvaraju, Ramprasaath R et al. (2017). 'Grad-cam: Visual explanations from deep networks via gradient-based localization'. In: *Proceedings of the IEEE international conference on computer vision*, pp. 618–626.

Shang, Xindi et al. (2021). 'Video visual relation detection via iterative inference'. In: *Proceedings of the 29th ACM international conference on Multimedia*, pp. 3654–3663.

Shi, Weisong and Schahram Dustdar (2016). 'The promise of edge computing'. In: *Computer* 49.5, pp. 78–81.

Sturm, Jürgen et al. (Oct. 2012). 'A Benchmark for the Evaluation of RGB-D SLAM Systems'. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Vilamoura, Algarve, Portugal.

Svenstrup, Mikael et al. (2009). 'Pose estimation and adaptive robot behaviour for human-robot interaction'. In: *2009 IEEE International Conference on Robotics and Automation*. IEEE, pp. 3571–3576.

Team, Chroma (2022). *ChromaDB: The AI-Native Open-Source Embedding Database*. Version used: specify your version here. URL: https://www.trychroma.com/.

Team, InternVL (2025). *InternVL 3.0: Expanding Multimodal Capabilities*. Accessed: 2025-10-31. URL: https://internvl.github.io/blog/2025-04-11-InternVL-3.0/.

Team, LLaVA (2025). *LLaVA: Visual Language Models with GPT-4*. Accessed: 2025-10-31. URL: https://llava-vl.github.io/.

Team, OpenCV (n.d.). *Camera Calibration Tutorial*. https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html. Image from OpenCV documentation. Accessed: 2025-11-29.

Team, Qwen (Sept. 2024). *Qwen2.5: A Party of Foundation Models*. URL: https://qwenlm.github.io/blog/qwen2.5/.

Tutorialpedia (2026). *Docker Container Runtime Performance Cost: Networking, CPU & Memory Overhead Explained + Mitigation Strategies*. https://tutorialpedia.org. Accessed 2026.

Villani, Valeria et al. (2018). 'Survey on Human-Robot Collaboration in Industrial Settings: Safety, Intuitive Interfaces and Applications'. In: *Mechatronics*. URL: https://iris.unimore.it/retrieve/e31e1250-94db-987f-e053-3705fe0a095a/Survey_HRC_mech_v4-compresso.pdf.

Wang, Wenbin et al. (2025). 'Retrieval-augmented perception: High-resolution image perception meets visual rag'. In: *arXiv preprint arXiv:2503.01222*.

Wei, Jason et al. (2022). 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. In: *arXiv preprint arXiv:2201.11903*.

Yanco, Holly A and Jill Drury (2004). 'Classifying human-robot interaction: an updated taxonomy'. In: *2004 IEEE international conference on systems, man and cybernetics (IEEE Cat. No. 04CH37583)*. Vol. 3. IEEE, pp. 2841–2846.

Yi, Shanhe, Cheng Li and Qun Li (2015). 'A survey of fog computing: Concepts, applications and issues'. In: *Proceedings of the 2015 workshop on mobile big data*. ACM, pp. 37–42.

Zacharaki, Angeliki et al. (2020). 'Safety bounds in human robot interaction: A survey'. In: *Safety science* 127, p. 104667.

Zhang, Kaiyuan, Yuchen Yan and Yunyi Jia (2025). 'Mixed-Reality (MR) Enhanced Human–Robot Collaboration: Communicating Robot Intentions to Humans'. In: *Robotics* 14.10, p. 133.

Zheng, Pu et al. (2022). 'Human arm motion prediction for collision avoidance in a shared workspace'. In: *Sensors* 22.18, p. 6951.

Zhou, Kaiyang et al. (2022). 'Learning to Prompt for Vision-Language Models'. In: *International Journal of Computer Vision* 130.9, pp. 2337–2348.

Appendix

A VLM Prompts

A.1 1. Prompt

```
You are a scene graph expert. Given a single image,
generate a JSON describing all detected objects and their
relations Use the format:
{nodes:[{id:1,name:'object1',meta:'safety or danger info',px_x:x,px_y:y},...],
edges:[{from:1,to:2 rel:'relation'}]}
Provide possible room or place nodes if applicable.
Each id is unique. px_x and px_y are center pixel coordinates.
Only use possible coordinates.
Include all possible object-object relations.
'meta' should reflect potential safety relevance or danger.
Output only the complete JSON in English. Dont add any other informations.
```

A.2 2. Prompt

```
You are a scene graph expert. Given a single image,
generate a JSON describing all detected objects and their
relations Use the format:
{nodes:[{id:1,name:'object1',meta:'safety or danger info',x,y},...],
edges:[{from:1,to:2 rel:'relation'}]}
Provide possible room or place nodes if applicable.
Each id is unique. x and y are center pixel coordinates.
Pixels are realtive specification, Only use possible values betwen 0 and 1.
Include all possible object-object relations.
'meta' should reflect potential safety relevance or danger.
For relations only use possible IDs, relations describe the spatial connection.
Output only the complete JSON in English. Dont add any other informations.
```

A.3 3. Prompt

```
Return a single YAML document describing detected places, objects and relations.
Output only YAML.
Structure:
image_id: "<string>"
place:
id: <int|null>
name: "<string|null>"
objects:
    id: <int>
    name: "<string>"
    confidence: <0.0-1.0>
    meta: "<one sentence possible danger or risk related informations>"
    center:
    x: <0.0-1.0>
    y: <0.0-1.0>
relations:
    from: <int>
    to: <int>
    rel: "<relation>"
    confidence: <0.0-1.0>
```

```
Output only the YAML document.
```

A.4 4. Prompt

```
Produce exactly two newline-separated CSV blocks and nothing else.
First block: nodes.csv (header + rows). Then one blank line. Then edges.csv.
nodes.csv header: id,name,meta,cx,cy,confidence
edges.csv header: from,to,rel,confidence
Coordinates must be between 0 and 1 with at least three decimals.
ids unique. meta in one sentence possible danger or risk related informations.
If no edges exist, edges.csv has only the header row.
Output only the two CSV tables separated by one blank line.
```

A.5 Complete prompt for the VLM

```
You are a scene graph expert.

## Verification First:
- For every node in the provided scene graph, confirm that it is clearly
  represents a visible entity in the image.
- If the entity represented by the node is NOT visible, CHANGE the id to
  -99 to indicate it is not visible. Do not reuse the old id for any other node.
- This applies to humans, objects, rooms, and building elements.
- Do not pass through nodes unchanged without verifying visibility
  of the represented entity.
- DO NOT OVERWRITE NODES WITH NEW ENTITIES from the image.
  Only ADD new nodes at new IDs if a visible ENTITY is missing.
  Therefore you MUST expand the graph if needed.
- ALL NEW entities MUST be appended with new IDs to the graph,
  they MUST START at the specified next available ID for any new nodes you add.

## Hard Deletion rule (no discretion)
- For every input node:
    - You MUST choose exactly one:
    - visible and accurate entity in the image -> keep node with
      same id in the graph
      (you can adjust name, attributes, position, relations, NOT LAYER)
    - not visible entity -> CHANGE id to -99
      (do not reuse the old id for any other node)
- No node may remain unchanged unless it is explicitly verified visible.

Given a single image, first verify whether the proposed scene graph
accurately represents the visual content of the image.
If the graph does not match the image, ADD, or ADJUST (change attributes,
positions, or relations) nodes and edges so the scene graph correctly
reflects which entities are visible, and what is not visible, in the image.

Provide the final corrected graph strictly as {output_format_name},
describing all visible and not visible entities as nodes and their relations.

Here is the possible scene graph (nodes and edges). Some positions
or attributes may be inaccurate and must be corrected if they do not
match the entities in the image. Some of the nodes may NOT appear as entity
the image. Do NOT assume a node is present as entity in the image just
because it is in the graph.
{cutout_str}
```

```
{detection_context}

Use the format:
{format_example}

## Core Rules (must follow)
- Possible entities are represented by nodes in the scene graph.
- ALWAYS use the same IDs for the same node when the visuals,
  name/layer, and approximate position of the entity match.
- IDs are the primary identifier for nodes, not names. Do not change
  the ID of a entity that is clearly visible, even if you adjust its
  name or position.
- Only if the entity which is represented by the node is clearly not
  visible or clearly does not match the image should you CHANGE the id to -99.
- If a node is not visible in the image, the ID must be CHANGED to -99.
- Names can be adjusted if the current ones are inaccurate,
  but IDs MUST be preserved if it is likely the same node.
- NEVER create duplicate nodes for the same node.
- Do NOT add new nodes unless a visible node is missing in the graph.
- If an entity is visible but is inaccurate, ADJUST its node position,
  attributes, or relations.
- Only ADD up to 3 new nodes total.
- Do not add new room nodes unless the existing room clearly no longer
  matches the scene.
- Update positions/metadata and relations as needed.
- ALWAYS change the id of nodes to -99 if necessary to maintain accuracy.
- INCLUDE at least one room node, which represents the space,
  but do not create new room nodes unless the existing one is clearly
  inaccurate or not visible.

## Node Adjustment & ID Rules (strict)
- Do not change the ID of any node that is clearly visible as an entity
  in the image.
- You may adjust its name, position, attributes, and relations, but the layer
  stays the same unless clearly incorrect.
- Only change the id to -99 if its not visible as an entity in the image.
- If a node is changed to id -99 because it is not visible as an entity,
  its old ID cannot be reused for any other node.
- Only assign a new ID when adding a completely new node that
  does not match any existing node in name, layer, or position.
{next_id_text}
- You MUST START new nodes at the specified next available ID and
  increment by 1 for each new node.
- DO NOT just OVERWRITE existing nodes with new nodes.
  Only ADD new nodes at new IDs.

## Layer Types (required for each node)
- OBJECT: Physical items, furniture, tools, equipment
- HUMAN: People, humans, persons, dogs, animals
- ROOM: Rooms (represents space only, not surfaces like walls/floor/ceiling)
- BUILDING: Building structures, floors, exterior elements

## Naming Guidelines
- Always use specific, descriptive node names.
- Descriptive naming is for clarity, correction, or highlighting salient
  attributes, not for creating duplicates.
- Include salient qualifiers when clearly visible:
```

```
    - Examples: "red fire extinguisher", "open silver laptop",
      "wooden dining table", "CO2 gas canister"
- Avoid generic names like "object", "item", "device" when more
  detail is visible.
- Do not create a new node just because you want to add more detail
  to the name (e.g., "on white desk"). Instead, adjust the existing
  nodes name if it is inaccurate or too generic.
- Avoid including other objects context unless it is essential
  for identification:
    - "red fire extinguisher" -> correct
    - "red fire extinguisher on white desk" -> do NOT make a new node
      if it already exists

## Semantic Metadata (strict, mandatory)
- Every node MUST include "semantic_description" and "semantic_pose".
- "semantic_description" must be a short, precise visual description
  of what the entity is (shape, color, material, key parts), not a
  relation description.
- Prefer examples like: "grey robot dog with four articulated legs"
  or "red cylindrical fire extinguisher".
- Avoid relation-heavy for this node metadata descriptions like:
  "next to table", "in front of door", "on left side of room".
- "semantic_pose" must describe the current physical state or
  posture of the entity (for example: "hanging", "standing",
  "lying", "walking", "moving", "open", "closed").
- These fields must be evidence-based from the image and must not be empty.

## Human Handling (strict, mandatory)
- All human nodes must be CHANGED to id -99 if there is no clear
  visual evidence of a human
  in the image. This is mandatory.
- Keep the same ID if a human is likely the same person, even if
  position changes.
- Update positions freely if humans moved.
- Use descriptive human names (e.g., "man in blue jacket",
  "woman with glasses").
- Never duplicate a human with the same description.
- If more humans are visible than in the graph -> ADD missing human
  nodes (up to 3 new nodes max).
- If fewer humans are visible than in the graph -> CHANGE id to
  -99 to excess human nodes.
- If a human is close to a entity which is represented by a
  node -> create edge.
- If an edge exists but the human is no longer close to the
  connected entity -> REMOVE edge.
- If there is no clear visual evidence of a human in the image,
  CHANGE id to -99 to all human nodes.

## Forbidden Structure & Surface Nodes (strict)
- NEVER create nodes for visual entities like:
    - Walls, floors, ceilings, windows as structural surfaces
    - Colors or textures of room surfaces (e.g., "white wall",
      "black floor", "wooden ceiling")
    - Architectural background elements unless they are standalone
      physical objects (e.g., "door", "stairs" are allowed if visible)
- ROOM nodes must represent space only - do not convert surfaces
  into ROOM or OBJECT nodes
- If any such nodes exist in the input scene graph, they MUST be
```

```
  CHANGED to id -99, and their old IDs cannot be reused.
- Do not convert surfaces into OBJECT nodes under any circumstances

## Risk Metadata (strict, mandatory)
- For every visible node, you MUST assess and describe potential
  safety risk metadata based on what is visually present.
- Use explicit hazard categories when applicable.
    - examples are: [fire hazard, trip hazard, electrocution hazard,
      collision hazard, cut/puncture hazard, chemical hazard, biohazard,
      falling object hazard], but you are NOT limited to this list and
      should use any other clearly plausible hazard if present.
- Assign a risk severity level for each node: low, medium, or high.
- Risk must be evidence-based from the image. Do not invent hazards
  that are not visually plausible.
- Do not default to "no hazard" for all nodes.
- Use "no hazard" only when the node has been explicitly checked and
  no plausible danger is visible.
- Ensure the node risk description clearly contains both hazard type
  and severity level (for example: "trip hazard (medium)").
- If multiple hazards are present for a node, include the primary
  hazard first and keep severity explicit.
- Keep focus on this section when generating the risk field and
  do not override these risk rules elsewher

## Additional Constraints
- ALWAYS CHANGE the id of nodes to -99 if they are not visible
  as entity in the image.
- Each node must:
    - Have a non-empty risk field that follows the Risk
      Metadata section above
    - Include a confidence score
    - Have at least one relation
- All nodes must be connected (no isolated nodes).
- Relations must reference valid IDs.
- Before outputting the graph, scan all nodes again and CHANGE
  the id of any node that is not
  clearly visible as an entity in the image to -99. Humans, objects,
  rooms, and buildings must
  all be verified.
- Ignore any diagnostic/gt flags in the input.

## FINAL SANITY CHECK (MANDATORY BEFORE OUTPUT)
- Ensure all nodes which are not clearly visible as an entity
  in the image have ID -99
- Ensure no ID was reused, except for ID -99 for invisible entities
- Ensure No node from the graph was deleted without being
  CHANGED with id -99. Removal is only allowed by CHANGING to id -99.
- Ensure no duplicates exist

## Output Rules
- Output ONLY the completed corrected graph in {output_format_name}.
Do NOT include explanations, comments, or extra text.
```